Syrian Arab Republic
Ministry of Higher Education and Scientific Research
Manara University-Faculty of Engineering
Robotic and Intelligent Systems Department

Design, Modeling, and Control of an Autonomous Mobile Manipulator for Pick-and-Place Tasks Using Advanced Control Strategies

Prepared by:

Beilassan Hdewa
Lana Alwazzeh
Zain Alabidin Shbani

Supervised by:

PhD. Mohamad Kheir Abdullah Mohamad, PhD. Essa Alghannam

2025-2026

ملخص

يقدّم هذا العمل تطويراً متكاملاً لمنظومة روبوتية متنقلة ذات ذراع مناورة، مصممة لتنفيذ مهام الالتقاط والوضع ضمن البيئات المنظمة. يجمع النظام المقترح بين منصة حركة متعددة الاتجاهات وذراع روبوتية بثلاث درجات حرية من نوع RRR ضمن بنية موحّدة تتيح تحقيق التنسيق بين التنقل والمناولة.

من الناحية الميكانيكية، تم تطوير منصة معيارية تتمتع بالاستقرار البنيوي اعتماداً على أسلوب تصنيع هجين يجمع بين العناصر الهيكلية المقصوصة بالليزر والمكونات المصنّعة بالطباعة ثلاثية الأبعاد. يوفّر تكوين النظام قدرة على الحركة متعددة الاتجاهات باستخدام عجلات ميكانيوم، إلى جانب تزويده بمؤثر نهائي مخصص لضمان التعامل الآمن مع الأجسام. كما تم تحليل توزع الكتلة وتحديد موقع مركز الثقل للتحقق من استقرار المنظومة أثناء الحركة وتزامن عمل الذراع. ولتأمين أساس تحليلي دقيق، تم بناء نموذج حركي كامل باستخدام صياغة دينافيت–هارتنبرغ، مما أتاح توصيف العلاقة بين فضاء المفاصل وفضاء المهمة بدقة. واستكمالاً لذلك، تم اشتقاق نموذج ديناميكي غير خطي باستخدام منهجية أويلر–لاجرانج، مع الأخذ بعين الاعتبار تأثيرات العطالة والاقتران الحركي وقوى كوريوليس والقوى النابذة والجاذبية، إضافة إلى ديناميكيات المشغلات والاحتكاك وتغير الحمولة والاضطرابات الخارجية. تمت دراسة أداء التحكم من خلال تصميم وتقييم ثلاث استراتيجيات مختلفة شملت متحكم PID التقليدي كأساس لتتبع المسارات، ومتحكم LQRالأمثل لتحسين الأداء، ومتحكم تكيفي من نوع Fuzzy–PID لتعزيز المتانة في ظل قيود المشغلات وتغيرات الحمل. وقد أتاح هذا التحليل المقارن تقييم الدقة والاستقرار وقابلية التكيف ضمن ظروف تشغيل واقعية. تحققت خاصية التشغيل الذاتي من خلال دمج إدراك البيئة المعتمد على مستشعر LiDAR مع خوارزمية الحقول الكامنة الاصطناعية للملاحة وتجنب العوائق، إلى جانب بنية إدارة مهام قائمة على آلة الحالات المنتهية لتنفيذ تسلسل مهام الالتقاط والوضع.

أظهرت نتائج المحاكاة قدرة النظام على تحقيق تتبع دقيق للمسارات وسلوك ديناميكي مستقر وتنفيذ موثوق للمهام. ويسلط هذا الإطار المتكامل الضوء على فاعلية الجمع بين التصميم الميكانيكي والنمذجة التحليلية واستراتيجيات التحكم المتقدمة ضمن منظومة واحدة، بما يسهم في تطوير المناولات الروبوتية المتنقلة الذكية لتطبيقات الأتمتة الصناعية والروبوتات الخدمية.

Abstract

In this project, the integrated development of an autonomous mobile robotic manipulator for structured pick-and-place operations is presented. The developed robotic manipulator is composed of an omnidirectional mobile platform and a 3-DOF RRR robotic arm, integrated in a single architecture.

From the mechanical point of view, the developed platform is characterized by the use of a structurally stable platform, achieved through the application of the hybrid manufacturing approach, where laser cutting is used in combination with 3D printing technology. The structure of the developed platform allows for the implementation of the omnidirectional motion of the robotic manipulator through the use of Mecanum wheels, while the platform is designed to support the use of an end effector for the safe handling of objects. The distribution of the masses of the developed platform, as well as the evaluation of the center of mass, were performed in order to ensure the stability of the platform while in motion. For the establishment of an analytical framework, the complete kinematic model was developed through the Denavit-Hartenberg formulation, thereby providing an accurate mapping between the joint space and the task space. Based on this, the nonlinear dynamic model was developed through the application of the Euler-Lagrange method, thereby incorporating inertial coupling, Coriolis and centrifugal forces, gravity, actuator dynamics, friction, payload changes, and disturbances. For the evaluation of the control performance, the implementation of three unique control methods was performed, namely, the application of the traditional PID control, optimal control through the LQR approach, and an adaptive fuzzy-PID control strategy for the enhancement of the control performance. Through the application of the multi-layered control strategy, the accuracy, stability, and adaptability of the control performance were assessed. The autonomous capability was realized through the integration of environment perception based on LiDAR technology and Artificial Potential Field navigation, and finite state machine-based task management.

The simulation results show accurate trajectory tracking, stable dynamic response, and reliable task execution. The proposed framework illustrates the benefits of integrating mechanical design, analytical modeling, and sophisticated control within a unified system and contributes to the advancement of intelligent mobile manipulators in industrial automation and service robotics.

Contents

List of Figures

List of Tables

1. Introduction

Continuous advances in the field of robotics have profoundly changed the way in which complex operations are carried out in industrial, medical, and service environments. As robotics continues to transcend from fixed and repetitive operations, the integration of mobility with dexterous manipulation has come to play a vital role in the development of autonomous and flexible operations for mobile manipulator systems. This is because the ability to interact with structured and semi-structured environments through the use of a mobile manipulator system will not only allow the robot to move through space but will also allow it to manipulate objects in a purposeful way.

Mobile manipulator systems are a combination of two important areas in robotics: mobile robotics and robotic manipulator systems. Mobile manipulator systems are equipped with the ability to move through space through the use of mobile robotics systems, while the ability to execute precise operations is achieved through the use of robotic manipulator systems. This has led to a number of challenges in the precise mathematical modeling of mobile manipulator systems, particularly in ensuring stability and precision in the performance of operations with varying payloads.

This research work describes the modeling, analysis, and control of a mobile robotic manipulator with a three-degree-of-freedom (3-DOF) RRR robotic arm on a mobile platform. The work begins with a detailed kinematic analysis using the Denavit-Hartenberg convention for the forward and inverse kinematic solutions. These solutions define the relationship between the joint variables and the end-effector motion, which serves as the basis for trajectory planning and control in the task space. Next, the dynamic model of the manipulator is developed using the Euler-Lagrange approach, which captures the nonlinear and coupled dynamics of the system, including inertial, Coriolis, centrifugal, and gravitational forces.

Based on the mathematical model, a variety of control methods are designed and compared. Traditional PID control is first considered for trajectory tracking, followed by optimal LQR control that incorporates a state-space design approach to optimize control performance and effort. Additionally, an adaptive fuzzy-PID control method is developed to improve robustness against uncertainties, actuator saturations, and changing operating conditions. The performance of these control designs is tested through simulation experiments involving obstacle avoidance, mapping, and the performance of a pick-and-place task in a realistic robotic setup.

The proposed framework makes a contribution to the design and control of autonomous mobile robotic manipulators. The combination of analytical modeling and advanced control techniques in this research work will enable the development of robotic systems that can operate in a safe, accurate, and reliable manner.

1.1. Problem Statement

Mobile manipulators are robots that navigate and manipulate objects, but the combination of these two capabilities brings its own set of problems like nonlinear dynamics, actuator limitations, tracking errors, and obstacle avoidance. If these problems are not addressed properly through modeling and control, then mobile manipulators may not be able to accomplish autonomous pick and place tasks successfully.

1.2. Research Question

How can a mobile robotic manipulator be modeled and controlled to achieve accurate, stable, and autonomous pick-and-place performance in the presence of nonlinear dynamics, actuator constraints, and environmental obstacles?

1.3. Aims of The Project

Main Objective

To design and implement a complete modeling and control framework for an autonomous mobile manipulator capable of executing pick-and-place tasks safely and accurately.

Specific Objectives

1. formulate an exact kinematic model of the robotic arm using the Denavit–Hartenberg notation.
2. obtain the nonlinear dynamic model of the robotic arm considering inertia, gravity, friction, actuator dynamics, and payload.
3. formulate diverse control techniques like PID control, LQR control, and adaptive fuzzy PID control.
4. assess the performance of each controller in terms of precision, robustness, and control actions.
5. incorporate trajectory generation and task space control for autonomous execution of pick and place operations.
6. incorporate obstacle avoidance techniques using Artificial Potential Fields, and LiDAR-based mapping techniques for autonomous execution.
7. validate the proposed system using simulations and assess the performance under actuator limitations and disturbances.

1.4. Significance of the Study

The significance of this study is that it will help in the development of autonomous mobile manipulators that can accomplish manipulation tasks safely and accurately. This research will help in the development of intelligent robotic systems that can work in real-world environments

2. Kinematic Analysis and the Role of a 3-DOF RRR Manipulator

In many industrial and technological environments, robotic arms perform tasks such as assembly, welding, painting, material handling, and medical procedures. The success of these systems depends critically on being able to predict and command the motion of the end effector with high precision. This prediction is governed by kinematics, which relates joint variables to the position and orientation of the end effector in space.

A serial manipulator with three degrees of freedom, all implemented as revolute joints (an RRR manipulator), provides a particularly instructive architecture. Three independent joint angles describe its configuration. Although this number of degrees of freedom is modest compared to many industrial arms, it already allows complex spatial motion and serves as a clean model for developing and analyzing forward and inverse kinematic techniques. It avoids some of the algebraic complexity of higher-DOF manipulators while retaining the essential nonlinear trigonometric structure that characterizes robot kinematics.

In such a system, forward kinematics refers to the computation of the end-effector pose (position and orientation) from known joint angles. Inverse kinematics refers to the computation of the joint angles needed to realize a desired end-effector position, and, in more general settings, orientation as well. For the 3-DOF RRR manipulator under consideration, both problems admit explicit closed-form expressions based on homogeneous transformation matrices and a Denavit–Hartenberg (DH) [1]

Figure 2—1: A 3-Dimension Model of the 3-DOF Manipulator

2.1. Geometry, Parameters, and Coordinate Systems

The mechanical configuration includes two rigid links that are connected in series by three revolute joints. The system also includes a rotational base that can turn around a vertical axis (it can turn around the global $y-$axis). This joint is followed by another revolute joint that connects the first link to the base. A third revolute joint is connected to the distal part of link 1 and the head of link 2. An end-effector is attached to the distal part of link 2.

The main physical dimensions are summarized by three scalars:

The length of link 1 is

$$L_1 = 0.5 \text{ m},$$

the length of link 2 is

$$L_2 = 0.5 \text{ m},$$

and the height of the base is

$$H_1 = 0.1 \text{ m}.$$

Four coordinate systems are introduced: a base frame $[x_0, y_0, z_0]$ a frame attached to joint 1$[x_1, y_1, z_1]$, a frame attached to joint 2 $[x_2, y_2, z_2]$, and a frame attached to the end effector $[x_3, y_3, z_3]$. These frames follow the Denavit–Hartenberg convention and are arranged to make the relative transformations between successive links simple and systematic.

Three joint variables fully describe the configuration of the manipulator:

- The base angle θ_1, which gives the rotation of the base disk.
- The angle of the first arm joint θ_2, between the base structure and link 1.
- The angle of the second arm joint θ_3, between link 1 and link 2.

The variable θ_1 selects a vertical plane in which links 1 and 2 move. The combined values of θ_2 and θ_3 then determine how far the arm reaches within that plane and at what elevation relative to the base the end effector is located. [1]

Category	**Description**	**Value**
Physical Dimensions	The Length of Link 1	$L_1 = 0.5$ m
	The Length of Link 2	$L_2 = 0.5$ m
	The Height of Base	$H_1 = 0.1$ m
Coordinate Systems	The Coordinate System of Base	$[x_0, y_0, z_0]$
	The Coordinate System of Joint 1	$[x_1, y_1, z_1]$
	The Coordinate System of Joint 2	$[x_2, y_2, z_2]$
	The Coordinate System of End Effector	$[x_3, y_3, z_3]$
Variables	The Angle of the Base	θ_1
	The Angle of Joint 1	θ_2
	The Angle of Joint 2	θ_3

Table 2-1: The Parameter of Manipulator

2.2. Homogeneous Rotation and Translation Matrices

Homogeneous coordinates are used to represent rotations and translations in a unified framework. A homogeneous transformation is a (4×4) matrix whose upper-left (3×3) block is a rotation matrix, whose upper-right (3×1) block is a translation vector, and whose last row is $[0\ 0\ 0\ 1]$. Products of such matrices represent successive rigid-body transformations.

To simplify notation, the trigonometric abbreviations

$$C_\alpha = \cos\alpha\,, \qquad S_\alpha = \sin\alpha$$

are used for any angle α.

A rotation about the x −axis by angle α is described by the homogeneous matrix

$$\mathrm{Rot}_{x,\alpha} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 1)}$$

A rotation about the y −axis by angle β is

$$\mathrm{Rot}_{y,\beta} = \begin{bmatrix} \cos\beta & 0 & \sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 2)}$$

A rotation about the z −axis by angle γ is

$$\mathrm{Rot}_{z,\gamma} = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 & 0 \\ \sin\gamma & \cos\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 3)}$$

Translations along the coordinate axes, expressed in homogeneous form, are

Translation along the x −axis by distance a:

$$\mathrm{Trans}_{x,a} = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 4)}$$

Translation along the y −axis by distance b:

$$\mathrm{Trans}_{y,b} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} & \boldsymbol{b} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix} \qquad \text{(Equation 5)}$$

Translation along the z −axis by distance (c):

$$\mathrm{Trans}_{z,c} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 6)}$$

These six basic matrices constitute the building blocks used in the DH construction of link transformations. [2]

2.3. Denavit–Hartenberg Convention and Parameters

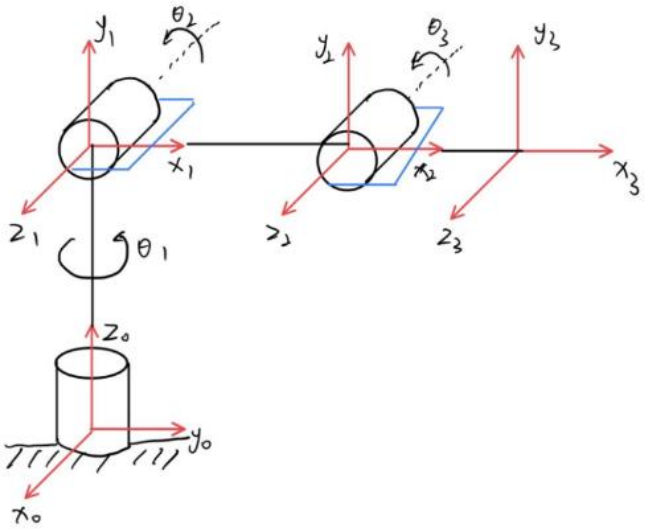


Figure 2—2: The Coordinate Systems of Manipulator

Link	$\mathbf{a_i}$	$\boldsymbol{\alpha_i}$ **(deg)**	$\mathbf{d_i}$	$\boldsymbol{\theta_i}$
1	H_1	90	0	θ_1
2	L_1	0	0	θ_2
3	L_2	0	d_3	θ_3

Table 22-: DH Parameters of Manipulator

The Denavit–Hartenberg convention provides a standardized way to assign coordinate frames to a serial manipulator and to characterize the relative pose of consecutive frames using four parameters. For link i, these parameters are $a_i, d_i, \alpha_i, \theta_i$.

The DH transformation from frame $i-1$ to frame i is expressed as

$$\boldsymbol{A_i} = \mathrm{Rot}_{z,\theta_i}\,\mathrm{Trans}_{z,d_i}\,\mathrm{Trans}_{x,a_i}\mathrm{Rot}_{x,\alpha_i} \qquad \text{(Equation 7)}$$

The geometric meaning of each parameter is as follows. The distance a_i is measured along axis x_i from the intersection of axes x_i and z_{i-1} to the origin o_i. The distance d_i is measured along axis z_{i-1} from origin o_{i-1} to the intersection of axes x_i and z_{i-1} For a prismatic joint, d_i would vary with the joint displacement; in the present manipulator, all joints are revolute, so each d_i is constant. The twist angle α_i is the angle about axis x_i from axis z_{i-1} to axis z_i. The joint angle θ_i is the angle about axis z_{i-1} from axis x_{i-1} to axis x_i; for a revolute joint, θ_i is the joint variable. [1]

For the 3-DOF RRR manipulator, the DH table is

Link 1:

$$a_1 = H_1, \quad \alpha_1 = 90^\circ, \quad d_1 = 0, \quad \theta_1 = \theta_1,$$

link 2:

$$a_2 = L_1, \quad \alpha_2 = 0°, \quad d_2 = 0, \quad \theta_2 = \theta_2,$$

link 3:

$$a_3 = L_2, \quad \alpha_3 = 0°, \quad d_3 = d_3, \quad \theta_3 = \theta_3.$$

The value $(\alpha_1 = 90°)$ indicates that the (z_1) axis is orthogonal to (z_0), so the first joint reorients the chain from the base axis into a plane in which the remaining links will move. The values $(\alpha_2 = 0°)$ and $(\alpha_3 = 0°)$ imply that (z_2) and (z_3) are parallel, so links 2 and 3 form a planar chain in that rotated plane. The parameter (d_3) introduces a constant offset along the final joint axis, for example due to an end-effector mounting distance.

2.4. Explicit Link Transformations (A_1), (A_2), and (A_3)

Using the DH parameters and the fundamental transformations, the three link matrices are written explicitly. To keep the notation compact, the abbreviations

$$C_i = cos\,\theta_i\,, S_i = sin\,\theta_i\,, C_{ij} = cos(\theta_i + \theta_j)\,, S_{ij} = sin(\theta_i + \theta_j)$$

are used.

The transformation from the base frame to frame 1 is

$$A_1 = \begin{bmatrix} \cos\theta_1 & 0 & \sin\theta_1 & H_1\cos\theta_1 \\ \sin\theta_1 & 0 & -\cos\theta_1 & H_1\sin\theta_1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 8)}$$

The transformation from frame 1 to frame 2 is

$$A_2 = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & L_1\cos\theta_2 \\ \sin\theta_2 & \cos\theta_2 & 0 & L_1\sin\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 9)}$$

The transformation from frame 2 to frame 3 (end effector) is

$$A_3 = \begin{bmatrix} \cos\theta_3 & -\sin\theta_3 & 0 & L_2\cos\theta_3 \\ \sin\theta_3 & \cos\theta_3 & 0 & L_2\sin\theta_1 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{(Equation 10)}$$

In a complete model, translations by the link lengths H_1, L_1 and L_2 appear in the appropriate entries of these matrices; the OCR snippet preserves the rotational structure and the offset d_3 which are central

to the orientation analysis. Conceptually, A_1 applies a rotation about the base axis and a twist, A_2 rotates link 1 in its plane, and A_3 rotates link 2 and shifts the end effector along the third joint axis.

2.5. Forward Kinematics and the Overall Transformation (T_3^0)

Forward kinematics constructs the transformation from the base frame to the end-effector frame:

$$T_3^0 = A_1 A_2 A_3.$$

Carrying out the multiplication yields

$$T_0^3 = \begin{bmatrix} C_{23}C_1 & -S_{23}C_1 & S_1 & H_1C_1 + d_3S_1 + L_1C_1C_2 + L_2C_1C_2C_3 - L_2C_1S_2S_3 \\ C_{23}S_1 & -S_{23}S_1 & -C_1 & H_1S_1 - d_3C_1 + L_1S_1C_2 + L_2S_1C_2C_3 - L_2S_1S_2S_3 \\ S_{23} & C_{23} & 0 & L_1S_2 + L_2S_{23} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{Equation 11}$$

The upper-left 3×3 block is the rotation matrix describing the orientation of frame 3 with respect to frame 0. The fourth column contains the position of the end effector relative to the base.

For further clarity, the entries of T_3^0 are denoted by r_{ij}. The forward kinematic equations for the rotational part are

$$r_{11} = C_{23}C_1,$$

$$r_{12} = -S_{23}C_1,$$

$$r_{13} = S_1,$$

$$r_{14} = H_1C_1 + d_3S_1 + L_1C_1C_2 + L_2C_1C_2C_3 - L_2C_1S_2S_3,$$

$$r_{21} = C_{23}S_1,$$

$$r_{22} = -S_{23}S_1,$$

$$r_{23} = -C_1,$$

$$r_{24} = H_1S_1 - d_3C_1 + L_1S_1C_2 + L_2S_1C_2C_3 - L_2S_1S_2S_3,$$

$$r_{31} = S_{23},$$

$$r_{32} = C_{23},$$

$$r_{33} = 0,$$

$$r_{34} = L_1S_2 + L_2S_{23},$$

$$r_{41} = 0, \quad r_{42} = 0, \quad r_{43} = 0, \quad r_{44} = 1.$$

The entries r_{11}, r_{21}, r_{31} describe the projection of the end-effector x_3 −axis onto the base axes (x_0, y_0, z_0). The entries r_{12}, r_{22}, r_{32} describe the projection of the y_3 −axis, and r_{13}, r_{23}, r_{33} describe the projection of the z_3-axis. For example, $r_{13} = \sin(\theta_1)$ shows directly how thez_3-axis acquires a component along the base x_0 −axis purely through the base rotation θ_1. The combinations

$\cos(\theta_2)\sin(\theta_3)$ and $\sin(\theta_2)\cos(\theta_3)$ in r_{31} and r_{32} show how the inner joint angles bend the arm relative to the vertical direction.

Thus, forward kinematics provides a nonlinear mapping

$$(\theta_1, \theta_2, \theta_3) \longmapsto T_3^0,$$

from which end-effector position and orientation are obtained. In robotic control, this mapping is used to predict the behavior of the arm given a set of joint commands. [1] [3]

2.6. Geometric Structure of the Inverse Kinematics

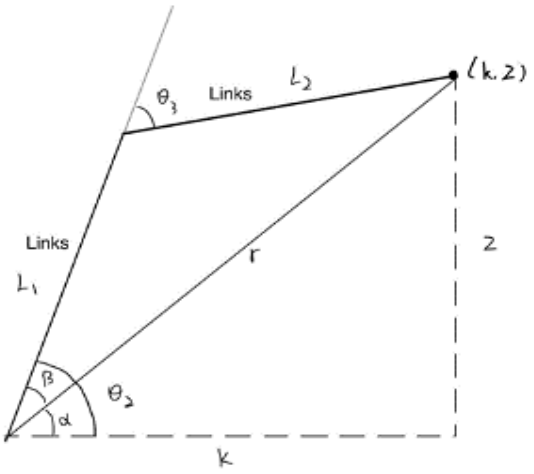


Figure 2—3: 2D Representation of 3-DOF RRR Manipulator

Inverse kinematics reverses the mapping: given a target position (x, y, z) for the end effector, the goal is to find joint variables $(\theta_1, \theta_2, \theta_3)$ that place the end effector at that point (for the present manipulator, orientation is left implicit, as three revolute joints do not in general allow arbitrary orientation in three dimensions).

The structure of the manipulator makes the first step straightforward. The base joint rotates the entire arm about the base axis. Therefore, the azimuth of the target in the horizontal plane determines the base angle independently of the inner joints.

Let (x, y, z) be the Cartesian coordinates of the desired end-effector position relative to the base frame. Define a horizontal distance k by $(k^2 = x^2 + y^2)$, and define

$$r = \sqrt{k^2 + z^2} = \sqrt{x^2 + y^2 + z^2} \qquad \text{(Equation 12)}$$

The quantity r represents the Euclidean distance from the base origin to the target point. The base angle is then

$$\theta_1 = \tan^{-1}\left(\frac{y}{x}\right), \qquad \text{(Equation 13)}$$

which is the azimuth of the projection of the target point onto the $(x–y)$ plane.

With θ_1 determined, the remainder of the problem becomes planar. The arm can be viewed in a plane whose horizontal coordinate is $\left(k = \sqrt{x^2 + y^2}\right)$ and whose vertical coordinate is z. In this reduced $(k–z)$ plane, link 1 and link 2 form a two-link planar manipulator that must reach from the shoulder joint to the point (k, z) at distance r from the shoulder. The inverse kinematics of a two-link planar arm are well known and can be derived via the law of cosines and basic trigonometry. [3]

2.7. Elbow Joint Angle (θ_3) Via the Cosine Rule

Consider the triangle formed by link 1, link 2, and the straight line from the shoulder joint to the end-effector point. The sides of this triangle have lengths L_1, L_2, and r, where:

$$r = \sqrt{x^2 + y^2 + z^2}.$$

Using the cosine rule for a triangle with sides a, b, and c, where ccc is opposite angle γ:

$$c^2 = a^2 + b^2 - 2ab\cos\gamma,$$

one can relate $L_1, L_2, \boldsymbol{r}$ and the internal angle at the elbow. In this manipulator, the internal elbow angle and the joint variable θ_3 differ by a supplementary relation, so the cosine rule appears in the form

$$r^2 = L_1^2 + L_2^2 - 2L_1 \cdot L_2 \cdot \cos(180^\circ - \theta_3) \qquad \text{(Equation 14)}$$

The identity $\cos(180^\circ - \theta_3) = -\cos\theta_3$ transforms equation (14) into

$$r^2 = L_1^2 + L_2^2 + 2L_1L_2\cos\theta_3\,.$$

Solving for θ_3 leads to an inverse cosine expression. In the OCR text, this appears as

$$\theta_3 = \cos^{-1}\left(\frac{x^2 + y^2 + z^2 - L_1^2 - L_2^2}{2 \cdot L_1 \cdot L_2}\right) \qquad \text{(Equation 15)}$$

The structure matches the typical pattern: an inverse cosine of a ratio involving squared distances and squared link lengths, scaled by $2L_1L_2$. The minus sign in front of $(\cos^{-1})$ fixes a particular configuration branch, such as an "elbow-down" posture. In general, two configurations (elbow-up and elbow-down) may correspond to the same target point, distinguished by the sign or complementary value of θ_3.

From a geometric viewpoint, θ_3 controls the bend between the two links. When $|\theta_3|$ is small, the arm is nearly straight, maximizing reach; when $|\theta_3|$ is large, the arm is highly bent, bringing the end effector closer to the base. Equation (15) ensures that the chosen bend matches the desired distance $(\boldsymbol{r})$. [2]

2.8. Auxiliary Angles (α) and (β), and Shoulder Angle (θ_2)

Determining the shoulder angle θ_2 requires relating the direction from the shoulder to the end effector to the internal geometry of the two-link chain. Two auxiliary angles, α and β are introduced to achieve this.

The angle α is defined as the elevation of the line from the shoulder joint to the end effector relative to the horizontal in the $(k–z)$ plane. Considering the right triangle with vertical side z and hypotenuse r, the relation

$$\sin\alpha = \frac{z}{r}$$

leads to

$$\alpha = \sin^{-1}\left(\frac{z}{r}\right), \qquad \text{(Equation 16)}$$

The angle β describes how the second link modifies the direction of this line. The contribution of link 2 to the motion of the end effector can be decomposed into horizontal and vertical components relative to the direction of link 1. The vertical component is proportional to $(L_2 \sin\theta_3)$, and the effective horizontal extension is $(L_1 + L_2\cos\theta_3)$. The ratio of these components yields

$$\beta = \tan^{-1}\left(\frac{L_2 \cdot \sin\theta_3}{L_1 + L_2 \cdot \cos\theta_3}\right) \qquad \text{(Equation 17)}$$

The shoulder angle must account for both the elevation to the target and the internal bend configuration, so

$$\theta_2 = \alpha + \beta.$$

Substituting (16) and (17) gives

$$\theta_2 = \sin^{-1}\left(\frac{z}{r}\right) + \tan^{-1}\left(\frac{L_2 \cdot \sin\theta_3}{L_1 + L_2 \cdot \cos\theta_3}\right) \qquad \text{(Equation 18)}$$

The first term in (18) points the first link approximately toward the target in the plane, while the second term corrects for the effect of the elbow's bend on the exact position of the end effector. [2]

2.9. Complete Inverse Kinematic set and Variable (r)

The inverse kinematic results can be collected as a set of equations relating the joint variables to the target coordinates and link lengths. In the table of inverse kinematic results:

$$\theta_1 = \tan^{-1}\left(\frac{y}{x}\right),$$

$$\theta_2 = \sin^{-1}\left(\frac{z}{r}\right) + \tan^{-1}\left(\frac{L_2 \cdot \sin\theta_3}{L_1 + L_2 \cdot \cos\theta_3}\right),$$

$$\theta_3 = \cos^{-1}\left(\frac{x^2 + y^2 + z^2 - L_1^2 - L_2^2}{2 \cdot L_1 \cdot L_2}\right),$$

These expressions reveal explicitly how each joint angle depends on the position coordinates and on the link parameters.

In other words, the inverse trigonometric functions in this case need to be computed while taking into account the branch considerations and the joint limits. For example, the $\tan^{-1}$ function can be written as a two-argument arctangent $\mathrm{atan2}(y, x)$ to make sure that θ_1 is in the correct quadrant. Also, the range of $\cos^{-1}$ function forces θ_3 to be in $[0, \pi]$.

The existence of a solution requires that the target point be inside the workspace of the manipulator. Intuitively, the distance (r) must satisfy geometric inequalities associated with a two-link chain:

$$|L_1 - L_2| \leq r \leq L_1 + L_2,$$

otherwise, no real value of (θ_3) satisfies the cosine relationship. Points too near the base (inside the "inner hole") or too far away are unreachable.

2.10. Interpretation of Forward and Inverse Kinematics in Motion Control

The forward kinematic model, embodied in equations $(1)-(11)$ and in the (r_{ij}) expressions, establishes a deterministic mapping from joint space to task space. For any joint configuration within mechanical limits, it provides the corresponding end-effector position and orientation. This model is used in simulation, where joint trajectories are proposed and their resulting Cartesian paths are examined for feasibility and safety (for example, avoiding collisions or singular configurations). It is also fundamental to many control laws that compute Jacobians and perform linearization based on the partial derivatives of the forward kinematic map.

The inverse kinematic relations $(12)-(18)$ provide the complementary mapping from desired positions to joint angles. In many applications, tasks are specified in terms of Cartesian coordinates, such as "move the end effector to (x, y, z) or "follow this spatial path." In these cases, the inverse kinematic equations must be solved repeatedly along the trajectory. Having explicit closed-form expressions reduces computational effort and improves reliability compared to purely numerical iterative methods.

Forward kinematics is comparatively simple and always yields a unique answer for a given joint configuration (assuming an ideal rigid mechanism). Inverse kinematics is more delicate: equations are nonlinear, multiple solutions may exist, or no solution may exist for points outside the reachable set. The geometric formulation used here reduces algebraic complexity by exploiting the rotational symmetry at the base and reducing the problem to a planar two-link system in the $(k–z)$ plane. This approach lowers computational burden without sacrificing correctness for the considered manipulator. [4]

2.11. Advantages and Limitations of the Geometric Approach

A geometric inverse kinematic solution, which relies on trigonometric identities and the law of cosines, is particularly appropriate for manipulators that can be geometrically analyzed by decomposing complex problems into two-dimensional ones. In the case of the 3-DOF RRR manipulator, the difference between the base rotation θ_1 and the planar configuration of links 1 and 2 is, of course, implicit in the geometry. This immediately points to equations (13) through (18) with their obvious geometric interpretations and expressions.
The use of a geometric solution is very advantageous but also introduces limitations. The relative simplicity of the solution is, of course, due in part to the geometry of the manipulator itself, which consists of one base rotation and two joints in a plane. More complicated geometries involving joints that are not centered, multiple stages with non-zero twists, or more degrees of freedom may not be as easily solved.

Moreover, geometric inverse kinematics always considers an ideal rigid body. In practical robotic arms, there are errors in the structure, such as elastic joints, mechanical slip, backlash, and manufacturing errors. These errors can cause a discrepancy between the computed joint angles and the actual end-effector position. Although geometric inverse kinematics remains important as a nominal model, other calibration and compensation methods, possibly including optimization and parameter identification, are also required to achieve high accuracy.

Forward kinematics, in theory, does not consider external forces, payloads, or interactions with the environment. However, dynamic effects such as inertia, friction, and compliance can cause a discrepancy between the actual and kinematic positions when a payload is attached. In control theory, these effects are handled by integrating kinematic and dynamic models and feedback control laws. [4]

2.12. Relation to Broader Developments in Robotics

The kinematic modeling of a 3-DOF RRR manipulator fits into a broader landscape of research on robotic mechanisms and control. Various works have addressed related problems for more complex or different architectures. For example, Huang and co-authors have proposed cooperative control strategies for three-degree-of-freedom parallel mechanisms, using synchronized and differential motion control schemes that outperform classic PID control in certain settings. [5]

Liu and co-authors have studied kinematic calibration for heavy-load manipulators, employing optimization algorithms to refine geometric parameters and thus improve accuracy in industrial tasks [6]. In marine engineering, Liu and colleagues have designed adaptive control systems for ship propulsion assisted by sails, using advanced sliding-mode control to manage uncertainty and disturbances. Sheng and collaborators have developed decoupling strategies for stable 3-DOF systems, providing improved control performance by effectively separating coupled dynamics. [7]

Standard references such as Spong, Hutchinson, and Vidyasagar's work on robot modeling and control provide a general theory of kinematics, dynamics, and control, within which the 3-DOF RRR manipulator serves as a concrete example. Craig's introduction to robotics systematizes the use of DH parameters and homogeneous transformations in forward and inverse kinematics [2]. Siciliano and co-authors survey modeling, motion planning, and control of robotic systems in depth, and Latombe's treatment of robot motion planning demonstrates how kinematic models underlie path-planning algorithms in complex environments. [8]

More recent studies by Johnson and Murphey advocate the integration of machine-learning techniques into robotic control to predict motion errors and adapt control parameters online [9]. Franklin and colleagues emphasize adaptive feedback control and real-time mechanisms for improving robustness and adaptability of robots operating in uncertain or changing environments. Such works point toward future directions where analytical kinematic models are combined with data-driven components and advanced control schemes to yield more capable and resilient robotic systems. [10]

2.13. Overall Synthesis

The kinematic description of the 3-DOF RRR manipulator rests on a clear geometric foundation. The physical dimensions (L_1, L_2, H_1), the joint variables $(\theta_1, \theta_2, \theta_3)$, the DH parameters $(a_i, d_i, \alpha_i, \theta_i)$, and the homogeneous transformations built from rotation and translation matrices together define the mapping between joint space and task space. The forward kinematic model, expressed through the matrices (A_1, A_2, A_3) and the resulting transformation (T_3^0) with entries (r_{ij}), provides a complete description of the end-effector pose as a function of the joint angles.

The inverse kinematic model, based on geometric reduction to a planar problem in the $(k–z)$ plane, introduces the auxiliary quantities k and r, and the angles α and β The resulting expressions $(13)–(18)$ give explicit formulas for $(\theta_1, \theta_2, \theta_3)$ in terms of the target coordinates (x, y, z) and the link lengths. These equations capture the essential nonlinear trigonometric relationships that govern how a relatively simple three-joint structure can reach a wide variety of positions in three-dimensional space.

Such a complete kinematic formulation is crucial for understanding and designing motion control strategies, for performing path planning, and for integrating the manipulator into larger automation systems. Although idealized, the model provides a foundation upon which dynamic effects, uncertainty, and advanced control methods can be layered. The concepts and equations associated with this 3-DOF RRR manipulator—every variable, parameter, and transformation presented here—illustrate the general methodology by which the motion of robotic manipulators is analyzed, predicted, and ultimately controlled. [11]

3. Dynamic model of the 3-DOF robotic manipulator

The dynamic model establishes the explicit relationship between the actuator-generated joint torques and the resulting motion of a three-link articulated manipulator with three revolute joints. This model is necessary for simulation and for the development of control algorithms. The Euler–Lagrange approach is adopted because it provides a systematic derivation of the equations of motion from energy expressions and is widely used for robotic manipulators.

To apply this method, the kinetic and potential energies of each link are evaluated, as illustrated in Figure 4. The kinetic energy includes rotational and translational components and depends on joint positions and velocities, $K_E(q, \dot{q})$ The potential energy is associated with conservative forces, mainly gravity. The Euler–Lagrange equation for an (n)-DOF system is:

$$\frac{d}{dt}\left(\frac{\partial L(q,\dot{q})}{\partial \dot{q}_i}\right) - \frac{\partial L(q,\dot{q})}{\partial q_i} = \tau_i \qquad \text{(Equation 19)}$$

where $i = 1,2, \dots, n, \tau_i$ is the generalized torque/force at joint i, q_i is the generalized coordinate (joint position), and $\dot{q}_i$ is the joint velocity. The physical parameters used in the model are provided in Table 3-1.

Category	Parameter Description	Symbol	Value	Unit
Geometry	Base vertical length	l_1	0.10	m
Geometry	Link 2 length	l_2	0.50	m
Geometry	Link 3 length	l_3	0.50	m
Mass	Mass of link 1	m_1	0.50	kg
Mass	Mass of link 2	m_2	0.50	kg
Mass	Mass of link 3	m_3	0.50	kg
Center of Mass	Center of mass location of link 1	l_{c1}	0.05	m
Center of Mass	Center of mass location of link 2	l_{c2}	0.25	m
Center of Mass	Center of mass location of link 3	l_{c3}	0.25	m
Physical Constant	Gravitational acceleration	g	9.81	m/s^2

Table 3-1: List of parameters of robot manipulator

Since the manipulator has three degrees of freedom, applying (19) yields three coupled nonlinear equations of motion. The joint torques are given by:

by:

$$\begin{aligned}\tau_1 = &\left(\frac{1}{2}m_1 r_1^2 + \frac{1}{4}m_2 l_2^2 C_2^2 + \frac{1}{4}m_3 l_3^2 C_{23}^2 + m_3 l_2^2 C_2^2 + m_2 l_2 l_{23} C_2 C_{23}\right)\ddot{q}_1 \\ &+ \left(-\frac{1}{4}m_2 l_2^2 \sin(2q_2) - m_3 l_2^2 \sin(2q_2)\right. \\ &\left. - m_3 l_2 l_3 \sin(2q_2 + q_3) - m_3 l_{c2}^2 \sin(2(q_2 + q_3))\right)\dot{q}_1\dot{q}_2 \\ &+ (-m_3 l_2 l_3 C_2 S_{23} - m_3 l_{c3}^2 \sin(2(q_2 + q_3)))\dot{q}_1\dot{q}_3\end{aligned} \qquad \text{(Equation 20)}$$

Where:

- r_1 is a radius. It quantifies how the mass m_1 (associated with the base rotation mechanism, Link 1, or a combined structure) is distributed relative to the vertical axis of Joint 1. It directly determines the moment of inertia of that mass about that axis.

$$\tau_2 = \left(\frac{1}{4}m_2 l_2^2 + m_3(l_2^2 + l_{c3}^2 + l_2 l_3 C_3)\right)\ddot{q_2} - m_2 l_2 l_3 S_3 \dot{q_2}\dot{q_3} + m_3(l_2 l_{c3} C_3 + l_{c3}^2)\ddot{q_3} - m_3 l_2 l_{c3} S_3 \dot{q_3} + \left(\frac{1}{2}m_2 l_{c2}^2 \sin(2q_2) + m_3 \frac{l_2^2}{2}\sin(2q_2) + l_2 l_{c3} m_3 \sin(2q_2 + q_3) + \frac{1}{2}l_{c3}^2 \sin(2(q_2 + q_3))\right)\dot{q_1^2} + l_{c2} m_2 g C_2 + l_2 m_3 g C_2 + l_{c3} m_2 g C_{23} \quad \text{(Equation 21)}$$

$$\tau_3 = \frac{1}{3}m_3 l_3^2 \ddot{q_3} - m_3(l_2 l_{c3} C_3 + l_{c3}^2)\ddot{q_2} + m_3 l_{c3} S_{23}(l_2 C_2 + l_{c3} C_{23})\dot{q_2^2} + m_3(l_2 l_{c3} S_3 \dot{q_2}\dot{q_3} + g l_{c3} C_{23}) \quad \text{(Equation 22)}$$

These expressions show that the manipulator dynamics are nonlinear and coupled through trigonometric terms and products of joint velocities and accelerations. [2]

3.1. Vector Form of the Manipulator Dynamics

The coupled nonlinear dynamics can be written compactly in vector form as:

$$\tau = M(q)\ddot{q} + B(q)\,[\dot{q}, \dot{q}] + C(q)\,[\dot{q}]^2 + G(q) \quad \text{(Equation 23)}$$

where τ is the actuator torque vector, $M(q)$ is an $(n \times n)$ symmetric positive definite inertia matrix $M(q) = M(q)^T > 0$, $B(q)$ is the Coriolis torque matrix, $C(q)$ is the centrifugal torque matrix, and $G(q)$ is the gravity vector $(n \times 1)$. The joint vector is $q \in R^{n \times \mathbb{1}}$, $[\dot{q}, \dot{q}]$ denotes the velocity-product vector, and [12]

$$[\dot{q}]^2 = \left[\dot{q_1^2}, \dot{q_2^2}, \dot{q_3^2}\right]^T. \quad \text{(Equation 24)}$$

3.2. Friction and Disturbance Modeling

To reflect the physical behavior more realistically, friction is included. Friction affects position-control accuracy, especially at low speeds. A simplified classical friction model consisting of viscous and Coulomb terms is used: [13]

$$F(\dot{q}) = F_V \dot{q} + F_S \operatorname{sgn}(\dot{q}) \quad \text{(Equation 25)}$$

where $F(\dot{q})$ is the friction force/torque, F_S is the direction-dependent Coulomb friction coefficient, F_V is the viscous damping coefficient, and $\dot{q}$ is the system velocity.

External disturbances and unmodeled effects are grouped as a global uncertainty term (τ_d). Including friction and disturbances modifies (23) into:

$$\tau = M(q)\ddot{q} + B(q)\,[\dot{q},\dot{q}] + C(q)\,[\dot{q}]^2 + G(q) + F(\dot{q}) + \tau_d \qquad \text{(Equation 26)}$$

3.3. Overall System Model Including Actuators

Many control designs treat torque as the input; however, actuator dynamics significantly affect the overall response. In this formulation, the control input is chosen as the armature voltage of the DC motor rather than the torque. Therefore, both mechanical manipulator dynamics and actuator electrical dynamics are combined.

3.3.1. Worst-Case Configuration and Assumptions

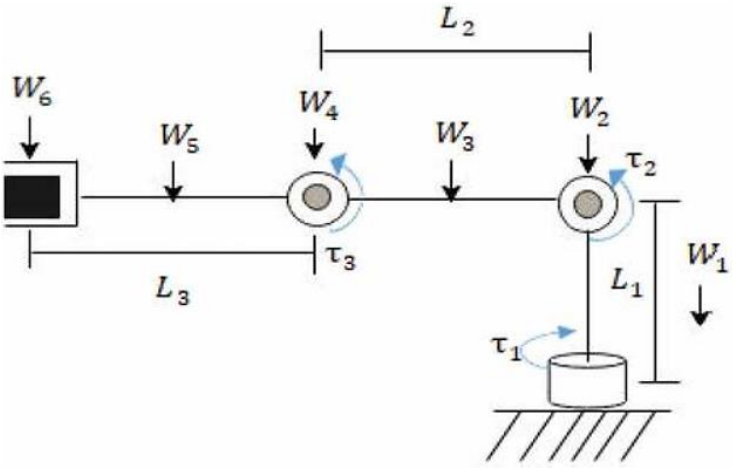


Figure 3—1: Force calculation of joints

The analysis considers the worst-case static gravity configuration, in which links 2 and 3 are fully extended and collinear in the horizontal plane while gravity acts vertically downward. This configuration maximizes the moment arms of all masses and therefore produces the maximum static holding torques. Only gravitational effects are considered at this stage; dynamic, inertial, Coriolis, and frictional effects are intentionally excluded because actuator sizing must first satisfy static holding requirements.

The base joint (joint 1) rotates about a vertical axis, so gravity does not generate a moment about that axis in this configuration. Consequently, the static gravity torque at joint 1 is zero. [14]

3.3.2. Torque at Joint 3 (Elbow) and Motor Selection

At joint 3, gravity torque is generated by the weight of link 3 and the payload. The motor mounted at joint 3 does not contribute to the gravity torque at that same joint because its center of mass lies on the joint's rotation axis, resulting in a zero-moment arm.

The static gravity torque at joint 3 is therefore

$$\tau_3 = g\left(m_3\, l_{c3} + m_p\, l_3\right)$$

Using the given parameters with $m_p = 1.0\,\text{kg}$

$$\tau_3 = 9.81(0.50 \cdot 0.25 + 1.0 \cdot 0.50) = 9.81 \cdot 0.625 \approx 6.13\,\text{N·m}$$

This value represents the minimum continuous holding torque required at joint 3. A PMDC motor with an integrated gearbox is an appropriate choice for this torque range due to its high torque density, simplicity, and controllability. A compact PMDC geared motor capable of delivering a continuous output torque of approximately 8–10 N·m provides sufficient margin. Motors in this class typically have a mass on the order of 0.8–1.2 kg. A representative and realistic choice is

$$m_{m3} \approx 1.0\,\text{kg}$$

This motor mass must now be included in the upstream torque calculation. [14]

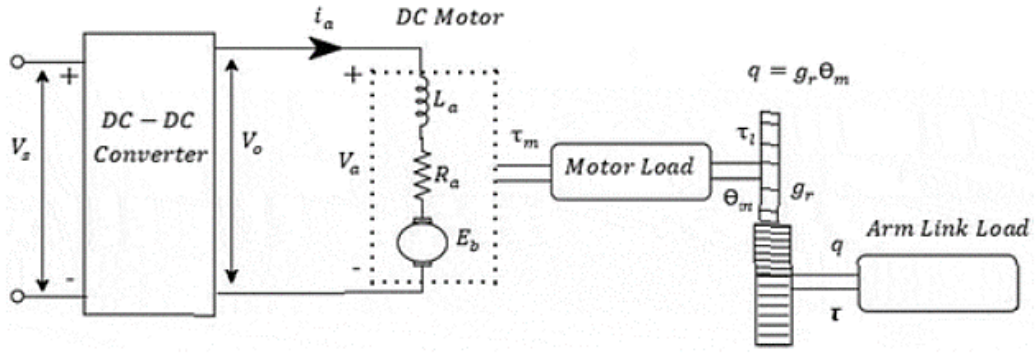


Figure 3—2: Equivalent circuit of a chopper fed DC motor with gear and mechanical load

3.3.3. Torque at Joint 2 Including Link, Payload, and Motor Mass

Joint 2 must support the gravitational moments generated by link 2, link 3, the payload, and the motor mounted at joint 3. The motor mass at joint 3 acts at a horizontal distance l2l_2l2 from joint 2 and therefore contributes directly to the shoulder torque.

The static gravity torque at joint 2 is

$$\tau_2 = g\big(m_2\, l_{c2} + m_3\,(l_2 + l_{c3}) + m_p\,(l_2 + l_3) + m_{m3}\, l_2\big)$$

Substituting numerical values

$$\tau_2 = 9.81(0.50 \cdot 0.25 + 0.50 \cdot 0.75 + 1.0 \cdot 1.00 + 1.0 \cdot 0.50)$$

$$\tau_2 = 9.81 \cdot 2.00 \approx 19.62\,\text{N·m}$$

This torque already includes the structural mass, payload, and the realistically sized PMDC motor selected for joint 3. Joint 2 therefore requires a motor-gearbox combination capable of delivering a continuous holding torque of at least $20\ N \cdot m$, with additional margin for dynamics and losses.

3.3.4. Joint 1 Consistency Check

In this geometry, joint 1 rotates about a vertical axis. All gravitational forces act parallel to that axis, producing zero moment arm. Therefore,

$$\tau_1^{(\text{gravity})} = 0$$

This remains valid regardless of link masses, payload, or motor masses. [15]

3.3.5. Interpretation for Motor and Gearbox Selection

The computed torques represent worst-case static holding torques, which must be met by the continuous rated torque of the motor-gearbox combination. The rated torque is the torque that can be delivered indefinitely without thermal overload, whereas the holding torque represents the minimum torque required to prevent motion under gravity. Peak torque ratings are not appropriate for static sizing because they are only sustainable for short durations. [15]

The gearbox ratio is selected so that

$$\tau_{\text{output}} = \eta\, N\, \tau_{\text{motor}}$$

where N is the gear ratio and η is gearbox efficiency.

Joint 3 (target ≈ 8–10 N·m continuous)

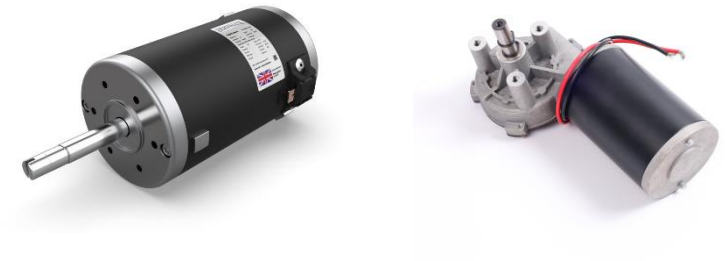


Figure 3—3: PMDC motor similar to the M63BDC125-24

A compact solution that lands inside this range is:

- **Motor:** M63BDC125-24 (rated torque $\tau_m = 270$ mNm $= 0.27$ N · m)
- **Gearbox:** MCP4 worm-wheel gearbox, 50:1 ratio (listed as available for M63BDC family)

Approximate continuous output torque (with a conservative worm efficiency $\eta \approx 0.8$):

$$\tau_{\text{out,3}} \approx \eta\, g_r\, \tau_m = 0.8 \times 50 \times 0.27 \approx 10.8\ \text{N·m}$$

This fits the 8– 10 N · m requirement.

For M63BDC125-24, the motor torque constant K_m is calculated based on the rated torque and the rated current using the formula:

$$K_m = \frac{\tau_m}{I_{\text{rated}}}$$

The rated current is 4.9 A (assuming this from typical PMDC motor datasheets), K_m would be calculated as:

$$K_m = \frac{0.27}{4.9} = 0.0551\ \text{N·m/A}$$

The motor inertia J_m for M63BDC125-24 can be approximated based on typical PMDC motor designs:

- For a motor like the M63BDC (with a 63 mm frame), a reasonable guess for J_m could be in the range of 10^{-4} kg·m^2

Viscous friction, β_m, would typically be provided under the mechanical characteristics in the datasheet. If it's not provided, it can be estimated or approximated based on similar motors in the same family. For M63BDC125-24, a typical viscous friction coefficient can be assumed as:

- $\beta_m = 0.01$ N·m·s/rad (a reasonable value for this motor type).

The M63BDC125-24 is a compact motor with relatively low torque compared to the larger motors used in other joints. For this motor, the friction parameters are:

- Viscous friction coefficient F_V:
    - The motor is small and has relatively low friction. A typical value for PMDC motors of this size and type is:

$$F_V \approx 0.01 \text{ N·m·s/rad}$$

- Coulomb friction coefficient F_S:
    - The Coulomb friction for this motor is also relatively low, typical for small PMDC motors. The assumed value is:

$$F_S \approx 0.2 \text{ N·m·s/rad}$$

These values reflect the low friction characteristics of the M63BDC125-24 motor, making it suitable for use in joints with lower torque requirements, such as Joint 3. [16] [16]

Joint 2 (target $\geq$ 20 N · m continuous)

Figure 3—4: Motors similar in class and size to the M90BDC199-24, which corresponds to a larger-frame PMDC motor suitable for higher torque applications

A stronger solution using the gearbox table (MCP2 compatible with M90BDC/M100BDC/M110BDC) is:

- **Motor:** M90BDC199-24 (rated torque 955 $mNm = 0.955\ N \cdot m$)
- **Gearbox:** MCP2 worm-wheel gearbox, 30: 1 ratio

$$\tau_{out,2} \approx 0.8 \times 30 \times 0.955 \approx 22.9 \text{ N·m}$$

This satisfies the $\geq 20\ N \cdot m$ requirement with margin.

For M90BDC199-24, the motor torque constant K_m is similarly calculated using the rated torque and rated current.

The rated current from the datasheet is 11.2 A, we calculate K_m as:

$$K_m = \frac{0.955}{11.2} \approx 0.0853\ \text{N·m/A}$$

Similar to the M63BDC125-24, the motor inertia J_m for M90BDC199-24 needs to be extracted from the datasheet or approximated. For M90BDC motors with a 90 mm frame, the typical motor inertia could be in the range of:

$$J_m \approx 2 \times 10^{-4}\ \text{kg·m}^2$$

As with the M63BDC125-24, the viscous friction β_m for M90BDC199-24 would need to be provided in the datasheet, or it could be estimated. Assuming similar values as typical for PMDC motors:

- $\beta_m = 0.01\ \text{N·m·s/rad}$

The M90BDC199-24 is a larger motor with significantly higher torque capability, making it suitable for joints with higher load demands, such as Joint 2. The friction parameters for this motor are:

- Viscous friction coefficient F_V:
    - Given the increased size and power of the motor, the viscous friction is expected to be higher. A typical value for motors of this size is:

$$F_V \approx 0.02\ \text{N·m·s/rad}$$

- Coulomb friction coefficient F_S:
    - Similarly, the Coulomb friction for this motor is assumed to be higher due to its greater size and the larger forces it will encounter during operation:

$$F_S \approx 0.3\ \text{N·m·s/rad}$$

These values reflect the increased frictional resistance that is common for larger motors such as the M90BDC199-24, which are used in joints with greater torque and load-bearing requirements.

Joint 1

A numerical holding-torque requirement for joint 1 was not provided in the latest constraints. In most 3R arms, joint 1 typically needs torque comparable to or higher than joint 2 if it carries the full arm inertia during motion; therefore, two consistent options exist: [15]

A. standardize joints 1 and 2:
   Use the same actuator as joint 2 for joint 1:
   M90BDC199-24 + MCP2 30:1
B. higher margin for joint 1:
   Select a higher-ratio MCP2 option (e.g., 60:1) with the same motor family if speed reduction is acceptable:

3.3.6. DC motor electrical and mechanical equations

The electrical and mechanical dynamics of the permanent-magnet DC motor are:

$$V_a = Ri_a + L\frac{di_a}{dt} + K_b\frac{d\theta_m}{dt} \qquad \text{(Equation 27)}$$

where V_a is the armature voltage, R and L are armature resistance and inductance, K_b is the back-EMF constant, i_a is the armature current, and θ_m is the motor angular position.

The electromagnetic torque is proportional to armature current:

$$\tau_m = K_m i_a \qquad \text{(Equation 28)}$$

where τ_m is the generated motor torque and K_m is the diagonal matrix of motor torque constants. [15]

Since the control input is V_a, and the inductance L is neglected due to its small magnitude, combining (27) and (28) yields the voltage-level model:

$$\begin{aligned} V_a = {} & RK_m^{-1}[g_r M(q) + J_m g_r^{-1}]\ddot{q} + RK_m^{-1}[g_r B(q)(\dot{q},\dot{q}) + g_r C(q)(\dot{q})^2] \\ & + RK_m^{-1}[\beta_m g_r^{-1} + R^{-1}K_m K_b g_r^{-1}]\dot{q} + RK_m^{-1} g_r G(q) \\ & + R^{-1}K_m[g_r F(\dot{q}) + g_r \tau_d] \end{aligned} \qquad \text{(Equation 29)}$$

where g_r is the gear ratio, J_m is the motor inertia, and β_m represents motor viscous friction. [15] [4]

3.4. Final complete nonlinear dynamic model (explicit acceleration form)

After substituting the selected physical and geometric parameters and rearranging the system equations into explicit acceleration form, the nonlinear dynamic model of the 3-DOF RRR robotic manipulator is expressed as:

$$\begin{aligned} \ddot{q} = \frac{1}{RK_m^{-1}[g_r M(q) + J_m g_r^{-1}]} & (V_A - RK_m^{-1}[g_r B(q)(\dot{q},\dot{q}) + g_r C(q)(\dot{q})^2] - RK_m^{-1}[\beta_m g_r^{-1} + R^{-1}K_m K_b g_r^{-1}]\dot{q} \\ & - RK_m^{-1} g_r G(q) - R^{-1}K_m[g_r F(\dot{q}) + g_r \tau_d]) \end{aligned} \qquad \text{(Equation 30)}$$

3.4.1. Explanation

The goal was to write the joint accelerations explicitly:

$$\ddot{q} = f(q, \dot{q}, V_a)$$

instead of the implicit torque form. In other words: given the current joint angles q, velocities $\dot{q}$, and the motor voltages V_a, the equation outputs the accelerations $\ddot{q}$.

That is why the left-hand side is:

$$\begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \\ \ddot{q}_3 \end{bmatrix}$$

For a 3-DOF manipulator, the standard Euler–Lagrange result is represented as Eq. (26)

Many texts also separate Coriolis/centrifugal as velocity-product and velocity-square terms; this final model does exactly that.

So conceptually, everything on the right of the following equation is:

$$\boxed{\ddot{q} = M(q)^{-1}\big(\tau - C(q,\dot{q})\dot{q} - G(q) - F(\dot{q})\big)}$$

an input term (here: voltage mapped to torque), minus motion-dependent opposing terms (Coriolis, centrifugal), minus gravity, minus friction, all multiplied by the inverse inertia to isolate accelerations.

Normally the input is τ. In this model the input is the armature voltage vector:

$$\begin{bmatrix} V_{a1} \\ V_{a2} \\ V_{a3} \end{bmatrix}$$

This comes from the DC motor equations (27, 28)

Then, after neglecting L (small inductance assumption) and mapping motor speed ω_m and motor torque τ_m to joint variables through gearing, you can eliminate i_a and express the *input* in terms of voltage. That yields a voltage-level "equivalent dynamics" structure where applied voltage is the control input and everything else appears as opposing terms. That is why Eq. (30) has V_{a1}, V_{a2}, V_{a3} as the driving term.

3.4.1.1. The Inertia Matrix $M(q)$

The inertia matrix $M(q)$ is defined by collecting the coefficients of the joint accelerations $\ddot{q}_1, \ddot{q}_2, \ddot{q}_3$ in the three torque equations τ_1, τ_2, τ_3. Therefore, the element $M_{ij}(q)$ equals the coefficient multiplying $\ddot{q}_j$ in the equation corresponding to τ_i.

Inertia terms associated with joint 1

The torque equation for joint 1 contains an acceleration term proportional to $\ddot{q}_1$ and does not contain any terms proportional to $\ddot{q}_2$ or $\ddot{q}_3$. Consequently,

$$M_{12}(q) = 0, \qquad M_{13}(q) = 0.$$

The coefficient multiplying $\ddot{q}_1$ defines $M_{11}(q)$:

$$M_{11}(q) = \frac{1}{2}m_1 r_1^2 + \frac{1}{4}m_2 l_2^2 \cos^2 q_2 + \frac{1}{4}m_3 l_3^2 \cos^2(q_2 + q_3) + m_3 l_2^2 \cos^2 q_2 + m_2 l_2 l_{c3} \cos q_2 \cos(q_2 + q_3),$$

where r_1 is retained as an unknown base-radius parameter. [4]

Using the numerical parameters, the individual contributions are

$$\frac{1}{2}m_1 r_1^2 = 0.25\, r_1^{2,}$$

$$\frac{1}{4}m_2 l_2^2 = \frac{1}{4}(0.5)(0.5^2) = 0.03125,$$

$$\frac{1}{4}m_3 l_3^2 = 0.03125,$$

$$m_3 l_2^2 = (0.5)(0.5^2) = 0.125, \qquad m_2 l_2 l_{c3} = (0.5)(0.5)(0.25) = 0.0625.$$

Hence,

$$M_{11}(q) = 0.25\, r_1^2 + 0.15625\, \cos^2 q_2 + 0.03125\, \cos^2(q_2 + q_3) + 0.0625\, \cos q_2 \cos(q_2 + q_3).$$

Inertia terms associated with joint 2

The torque equation for joint 2 contains acceleration terms proportional to $\ddot{q}_2$ and $\ddot{q}_3$. These coefficients define $M_{22}(q)$ and $M_{23}(q)$:

$$M_{22}(q) = \frac{1}{4}m_2 l_2^2 + m_3(l_2^2 + l_{c3}^2 + l_2 l_{c3} \cos q_3),$$

$$M_{23}(q) = m_3(l_{c3}^2 + l_2 l_{c3} \cos q_3).$$

therefore

$$M_{22}(q) = 0.03125 + (0.125 + 0.03125) + 0.0625 \cos q_3 = 0.1875 + 0.0625 \cos q_3,$$

$$M_{23}(q) = 0.03125 + 0.0625 \cos q_3.$$

Inertia term associated with joint 3

The coefficient of $\ddot{q}_3$ in the third torque equation defines

$$M_{33}(q) = \frac{1}{3}m_3 l_3^2.$$

With $m_3 = 0.5$ and $l_3 = 0.5$,

$$M_{33}(q) = \frac{1}{3}(0.5)(0.5^2) = \frac{1}{3}(0.5)(0.25) = 0.0416667.$$

The inertia matrix is symmetric, so

$$M_{32}(q) = M_{23}(q).$$

Then the final $M(q)\ is$:

$$M(q) = \begin{bmatrix} 0.25r_1^2 + 0.15625C_2^2 + 0.03125C_{23}^2 + 0.0625C_2C_{23} & 0 & 0 \\ 0 & 0.1875 + 0.0625C_3 & 0.03125 + 0.0625C_3 \\ 0 & 0.03125 + 0.0625C_3 & 0.0416667 \end{bmatrix}$$

3.4.1.2. Gravity Terms

Joint 1:

For joint 1, the gravity term is zero because joint 1's torque does not depend on its own position in the gravitational field (assuming it is a rotational joint around the vertical axis):

$$G_1(q) = 0 \text{ N·m}$$

Joint 2:

The gravity torque at joint 2 is calculated as:

$$G_2(q) = m_2 \cdot g \cdot l_{c2} \cdot \cos(\theta_2) + m_3 \cdot g \cdot l_2 \cdot \cos(\theta_2) + m_3 \cdot g \cdot l_{c3} \cdot \cos(\theta_2 + \theta_3)$$

Substituting the values:

$$G_2(q) = 0.5 \cdot 9.81 \cdot 0.25 \cdot \cos(\theta_2) + 0.5 \cdot 9.81 \cdot 0.5 \cdot \cos(\theta_2) + 0.5 \cdot 9.81 \cdot 0.25 \cdot \cos(\theta_2 + \theta_3)$$

$$G_2(q) = 1.22625 \cdot \cos(\theta_2) + 2.4525 \cdot \cos(\theta_2) + 1.22625 \cdot \cos(\theta_2 + \theta_3)$$

$$G_2(q) = (3.67875 \cdot \cos(\theta_2) + 1.22625 \cdot \cos(\theta_2 + \theta_3)) \text{ N·m}$$

Joint 3:

For joint 3, the gravity torque is:

$$G_3(q) = m_3 \cdot g \cdot l_{c3} \cdot \cos(\theta_2 + \theta_3)$$

Substituting the values:

$$G_3(q) = 0.5 \cdot 9.81 \cdot 0.25 \cdot \cos(\theta_2 + \theta_3)$$

$$G_3(q) = 1.22625 \cdot \cos(\theta_2 + \theta_3) \text{ N·m}$$

Gravity torque comes from the potential energy:

$$G(q) = \frac{\partial PE(q)}{\partial q}$$

For revolute joints, gravity typically appears as cosine terms. For a 2-link planar section, you naturally get dependence on q_2 and $q_2 + q_3$, hence C_2 and C_{23}. So, these cosine terms are exactly the gravity loading mapped to joints. [4]

3.4.1.3. Friction Term

The friction model is the classic Coulomb + viscous form: [4] [15]

$$F(\dot{q}) = F_V \dot{q} + F_S \operatorname{sgn}(\dot{q})$$

The friction at joint 1 is:

$$F_2(\dot{q}) = 0.02 \cdot \dot{q_2} + 0.3 \cdot \operatorname{sgn}(\dot{q_2})$$

The friction at joint 2 is:

$$F_2(\dot{q}) = 0.02 \cdot \dot{q_2} + 0.3 \cdot \operatorname{sgn}(\dot{q_2})$$

The friction at joint 3 is: [17]

$$F_3(\dot{q}) = 0.01 \cdot \dot{q_3} + 0.2 \cdot \operatorname{sgn}(\dot{q_3})$$

3.5. Explicit Modeling of Payload Effects in the Manipulator Dynamics

To realistically represent the pick-and-place operation, the manipulated object is modeled as a payload mass m_p rigidly attached to the end-effector. Although the control objective is formulated in Cartesian (task) space, the robot is actuated through joint torques. Consequently, the payload directly affects the system dynamics and must be explicitly incorporated into the joint-space dynamic model.

The manipulator dynamics are described by the standard equation:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$$

where $M(q)$ is the inertia matrix, $C(q,\dot{q})$ represents Coriolis and centrifugal effects, and $G(q)$ is the gravity vector. The payload mass influences all three terms, not only the gravity component. [11] [4]

3.5.1. Payload Contribution to the Inertia Matrix $M(q)$

Let the Cartesian position of the end-effector be denoted by $p_e(q)$, and let $J_p(q) = \frac{\partial p_e}{\partial q}$ be the translational Jacobian. When a payload of mass m_p is attached to the end-effector, the kinetic energy of the system gains an additional term:

$$T_p = \frac{1}{2} m_p \dot{p}_e{}^T \dot{p}_e = \frac{1}{2} m_p \, \dot{q}^T J_p^T(q) J_p(q) \dot{q}$$

where:

$$m_p(t) = \begin{cases} 0, & before\ pick\ and\ after\ place \\ m_p, & during\ transport \end{cases}$$

This expression leads to an additive contribution to the inertia matrix:

$$M(q) = M_0(q) + \Delta M_p(q)$$

$$\Delta M_p(q) = m_p J_p^{\mathrm{T}}(q) J_p(q)$$

where $M_0(q)$ denotes the inertia matrix of the unloaded manipulator. This formulation shows explicitly that the payload increases the effective inertia in a configuration-dependent manner. In extended configurations, where the Jacobian magnitudes increase, the inertial effect of the payload becomes more pronounced. [4]

3.5.2. Payload Contribution to Coriolis and Centrifugal Effects $C(q, \dot{q})$

Since the Coriolis and centrifugal terms are derived directly from the inertia matrix, any modification to $M(q)$ necessarily alters $C(q, \dot{q})$. Using the standard Christoffel-symbol formulation,

$$C_{ij}(q, \dot{q}) = \sum_{k=1}^{n} \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i} \right) \dot{q}_k$$

substituting $M(q) = M_0(q) + m_p J_p^{\mathrm{T}}(q) J_p(q)$ yields

$$C(q, \dot{q}) = C_0(q, \dot{q}) + \Delta C_p(q, \dot{q})$$

where $\Delta C_p(q, \dot{q})$ arises entirely from the payload-induced inertia term. As a result, the payload introduces additional velocity-dependent coupling forces, which become significant during fast acceleration and deceleration phases of the pick-and-place motion.

The potential energy associated with the payload is given by:

$$V_p = m_p g z_e(q)$$

where $z_e(q)$ denotes the vertical position of the end-effector. The corresponding gravity torque contribution is obtained by differentiation:

$$\Delta G_p(q) = \frac{\partial V_p}{\partial q} = m_p g \frac{\partial z_e(q)}{\partial q}$$

Since $\frac{\partial z_e(q)}{\partial q}$ corresponds to the vertical row of the translational Jacobian, the gravity vector can be written as:

$$G(q) = G_0(q) + m_p g J_{p,z}^T$$

For the considered yaw–pitch–pitch manipulator, this results in payload-dependent gravity terms that primarily affect the second and third joints. Under the adopted lumped-mass modeling assumptions, these terms take the explicit form: [11] [4]

$$G_2(q) = \left(m_1 l_{c1} + \left(m_2 + m_p\right) l_1\right) g\, cos\, q_2 + \left(m_2 + m_p\right) l_{c2} g\, cos(q_2 + q_3)$$

$$G_3(q) = \left(m_2 + m_p\right) l_{c2} g \cos(q_2 + q_3)$$

3.5.3. Physical Interpretation of Payload Effects

The above derivations demonstrate that the payload mass is not a negligible disturbance but a fundamental component of the system dynamics. Specifically:

The payload increases the inertia matrix through the term m_p, J_p, J_p^T, requiring higher joint torques to achieve the same accelerations.

The payload modifies Coriolis and centrifugal effects, leading to increased velocity-dependent torque demands during dynamic motion.

The payload augments gravitational loading, especially during lifting and holding phases of the task.

These effects explain the increased joint torques observed during the payload transport interval in the simulation results, while the Cartesian tracking errors remain small. This confirms that the inverse-dynamics-based task-space PID controller effectively compensates the payload-induced dynamics rather than relying on high feedback gains. [4]

4. Robot Modeling and Task Formulation

This section is dedicated to the presentation of the robot's kinematic relationships, task space, and trajectory generation, which will be used throughout this study. The same model and reference motion will be used for all the following control strategies: PID, LQR, and fuzzy.

4.1. Kinematic and Dynamic Coupling

The relationship between the end-effector's and joints' motion is expressed by the manipulator's Jacobian matrix. The velocity of the end-effector in Cartesian space is expressed as follows:

$$\dot{p} = J(q)\dot{q}$$

The end-effector's acceleration in Cartesian space is expressed as follows:

$$\ddot{p} = J(q)\ddot{q} + \dot{J}(q, \dot{q})\dot{q}$$

This equation demonstrates that even when the desired end-effector's acceleration is provided, in order to accomplish this motion, in addition to the end-effector's accelerations, the velocity coupling effects must be taken into account as well. Therefore, in order to accurately perform task-space motion, kinematic relationships alone are not sufficient, particularly for fast and changing robot configurations. [4]

4.2. Task-Space Error Definition and Its Interpretation

The pose of a rigid body in three-dimensional space is generally described by an element of the Lie group $SE(3)$, which includes both position and orientation. However, the considered manipulator possesses only three degrees of freedom and therefore cannot independently control the full pose of the end-effector. For this reason, the control objective in this work is restricted to the translational component of the end-effector motion.

Accordingly, the tracking error is defined in the Cartesian position space $\mathbb{R}^3$ as

$$e = p_d - p, \qquad \dot{e} = \dot{p}_d - \dot{p}$$

This definition can be interpreted as a projection of the full $SE(3)$ pose error onto its translational subspace. While orientation errors are not explicitly controlled, this choice is sufficient and appropriate for the considered pick-and-place task, where only the end-effector position is critical. [4]

4.3. Trajectory Generation for Pick-and-Place Tasks

Since the controller operates at the acceleration level, a continuous reference trajectory together with its derivatives must be provided.

In pick-and-place applications, the control objective is not limited to reaching a final position but also requires smooth and physically feasible motion between intermediate task phases, such as approach, lifting, transport, and placement. Consequently, a continuous trajectory must be generated for the end-effector position in Cartesian space, together with its first and second derivatives, to serve as reference inputs for the task-space controller.

In this work, the pick-and-place motion is decomposed into multiple Cartesian segments: motion from the initial position to the pick location, vertical lifting of the object, horizontal transport at a safe height, lowering toward the placement location, and finally a return motion. Each segment is generated independently using cubic polynomial interpolation, ensuring smooth transitions between task phases.

For each Cartesian segment, the desired end-effector position is defined using a third-order polynomial of the form:

$$p_d(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

where $p_d \in \mathbb{R}^3$ denotes the desired Cartesian position vector. The corresponding velocity and acceleration profiles are obtained by differentiation:

$$\dot{p}_d(t) = a_1 + 2a_2 t \ + 3a_3 t^2$$

$$\ddot{p}_d(t) = 2a_2 + 6a_3 t$$

The polynomial coefficients are determined by imposing boundary conditions on position and velocity at the beginning and end of each segment:

$$p_d(0) = p_0, \quad p_d(T) = p_f, \quad \dot{p}_d(0) = 0, \quad \dot{p}_d(T) = 0$$

Solving this system yields:

$$a_0 = p_0, \quad a_1 = 0, \quad a_2 = \frac{3(p_f - p_0)}{T^2}, \quad a_3 = \frac{2(p_0 - p_f)}{T^3}$$

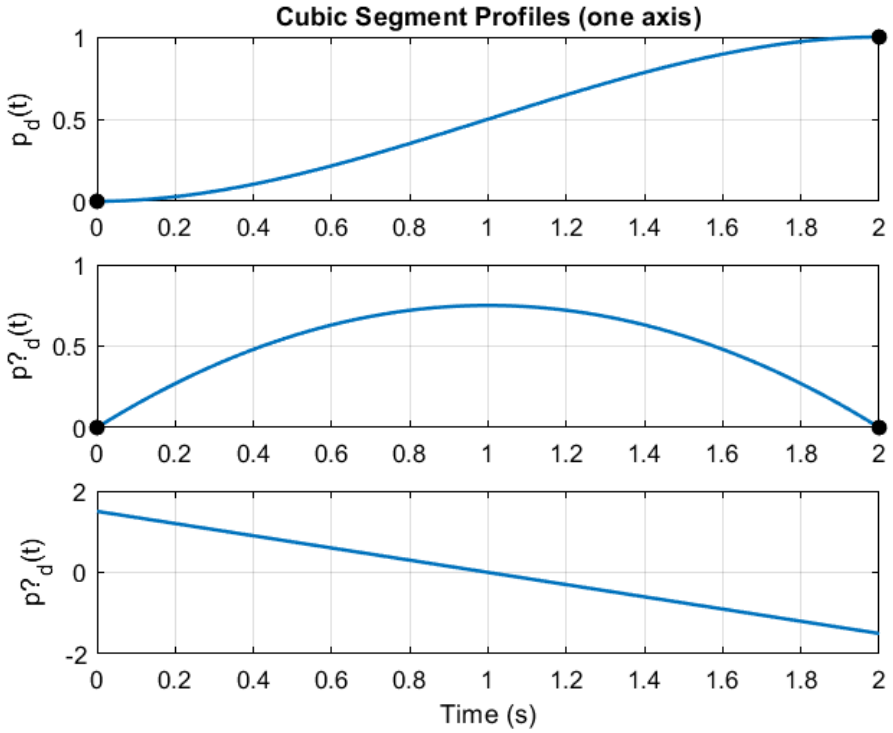


Figure 4—1: Cubic Polynomial Reference Trajectory Profiles (Position, Velocity, and Acceleration) for a Single Axis

This choice guarantees continuity of position and velocity across all segments, while allowing piecewise-defined accelerations that remain bounded and well suited for inverse-dynamics-based control. [4] [11]

4.4. Interpretation of the Generated Trajectory

The generated Cartesian trajectory has several good properties. Firstly, the position profile is continuous and smooth, eliminating abrupt changes in motion that could lead to significant effects on unmodeled dynamics or require large control torques. Secondly, the velocity profile starts and ends with zero in every segment of the motion, which is particularly important for grasping and releasing the object without causing slipping or impact effects. Finally, the acceleration profile is continuous in every segment and remains finite, making it suitable for the task-space PID controller based on the acceleration level.

The segments of the motion allow task constraints to be incorporated into the motion planning problem. For instance, the lifting segments guarantee clearance in the z-direction to prevent collisions during transport, and the transport segment maintains a constant height. [4]

5. Control Design

With the robot kinematic relationships, task-space formulation, dynamic model, and reference trajectory generation established in the previous sections, the next step is to design the control mechanisms for tracking the reference trajectories accurately.

As the manipulator is controlled by means of joint torques and the control problem is specified in Cartesian space, this requirement applies to all controllers. The robot model and assumptions used for the trajectory and payload have already been specified above and will thus be adopted for all controllers examined in this work.

Three different control approaches will be examined: a classical task space PID controller, an optimal Linear Quadratic Regulator, and a fuzzy controller. The controllers cover different design philosophies: a heuristic approach using tuning rules, optimal control theory, and a nonlinear approach using rule-based control. [18]

5.1. Task-Space PID Control

As a starting point for control design, a task-space PID controller is proposed. This type of controller offers a natural framework for designing the closed-loop error system while still allowing for inverse dynamics-based actuation.

By adopting the Cartesian error definition from above, the control law is defined in the acceleration level, which is a natural fit for the dynamic behavior of robotic systems, in which torques applied to the joints induce acceleration. [18]

5.1.1. Task-Space PID Controller Formulation and Error Dynamics

Instead of directly controlling position or velocity, the control law is defined in the acceleration level, which is a natural fit for the dynamic behavior of robotic systems, in which torques applied to the joints induce acceleration.

Let the commanded acceleration in the Cartesian space be defined as

$$\ddot{p}_{cmd} = \ddot{p}_d + K_d\dot{e} + K_p e + K_i \int e\, dt$$

This structure combines feedforward and feedback actions to achieve accurate trajectory tracking. The feedforward term anticipates the desired motion, the proportional and derivative terms regulate instantaneous position and velocity errors, and the integral term compensates for steady-state offsets caused by modeling inaccuracies, friction, and disturbances.

Let the tracking error be defined as $e = p_d - p$. Differentiating twice gives

$$\ddot{e} = \ddot{p}_d - \ddot{p}$$

Assuming that the commanded acceleration is accurately realized, substituting the control law yields

$$\ddot{e} + K_p e + K_d\dot{e} + K_i \int e\, dt = 0$$

Hence, the controller is explicitly constructed so that the tracking error follows a prescribed differential equation. In the absence of integral action, the dynamics reduce to a standard second-order system, showing that the transient behavior of the error is deliberately imposed through the controller structure rather than assumed. [18]

5.1.2. Gain Selection and Stability Interpretation

The proportional and derivative gains are selected based on matching the error system to the canonical second-order form, which gives

$$K_p = \omega_n^2, \quad K_d = 2\zeta\omega_n$$

This form shows a direct relationship with the performance criteria in terms of settling time and percent overshoot ($T_s \approx \frac{4}{\zeta\omega_n}$). A damping ratio near unity is selected for a well-damped response, while the natural frequency is based on the duration of the fastest segment in the reference trajectory, providing a trade-off in control effort for faster convergence.

With the inclusion of the integral term, a new pole is introduced near the origin, which eliminates steady-state errors due to gravity, friction, and payload changes in the robot. However, to ensure the same transient performance, the integral gain is selected to be sufficiently small so that the overall system is dominated by the second-order pole pair.

From a frequency-domain perspective, the design can also be interpreted through pole placement and root-locus arguments: increasing the gains moves the closed-loop poles deeper into the left-half plane, improving response speed while maintaining stability as long as the poles remain in the stable region.

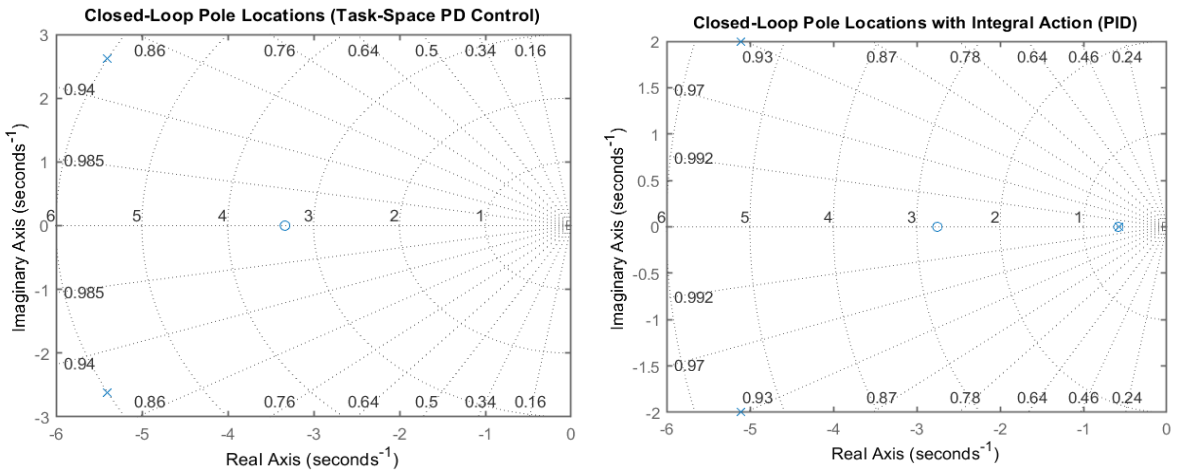


Figure 5—1: Closed-Loop Pole Maps in the Complex Plane for Task-Space PD and PID Control Configurations

The root locus reveals that as the gains of the controller are increased, the dominant pole moves along a stable path in the left half of the plane, leading to faster convergence because of the increased negative real parts. The graphs also illustrate that as long as all poles are in the left half of the plane, stability is maintained, and the addition of integral control causes a slow pole to be added near the origin.

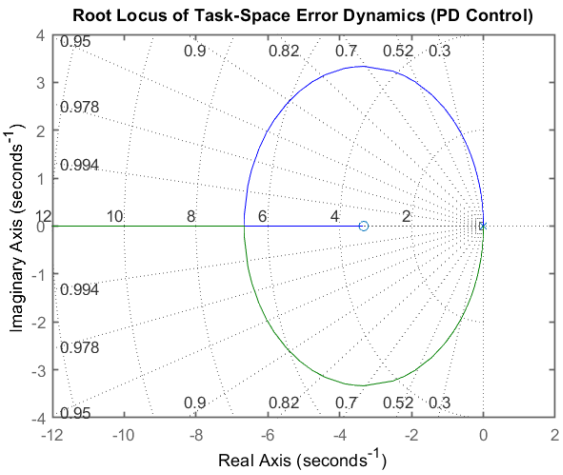


Figure 5—2: Root Locus of Task-Space Error Dynamics under PD Control

Although the controller is derived under ideal assumptions, real robotic systems exhibit actuator bandwidth limits, dynamic coupling, and physical constraints. Therefore, additional simulations including actuator dynamics were conducted to assess the practical implications of the selected pole locations. [19]

5.1.3. Mapping Task-Space Commands to Joint-Space Motion

The commanded Cartesian acceleration cannot be directly applied to the robot actuators. Instead, it must be converted into joint accelerations using the kinematic relationship

$$\ddot{q}_{cmd} = J^{\dagger}(\ddot{p}_{cmd} - \dot{J}(q,\dot{q})\dot{q})$$

where the term $\dot{J}\dot{q}$ accounts for the acceleration induced by the time variation of the Jacobian. The mapping relies on the damped least-squares inverse

$$J^{\dagger} = (J^{T}J + \lambda^{2}I)^{-1}J^{T}$$

which improves numerical robustness when the manipulator approaches kinematic singularities. This transformation provides the necessary link between the task-space controller and the physically realizable joint-space commands. [19]

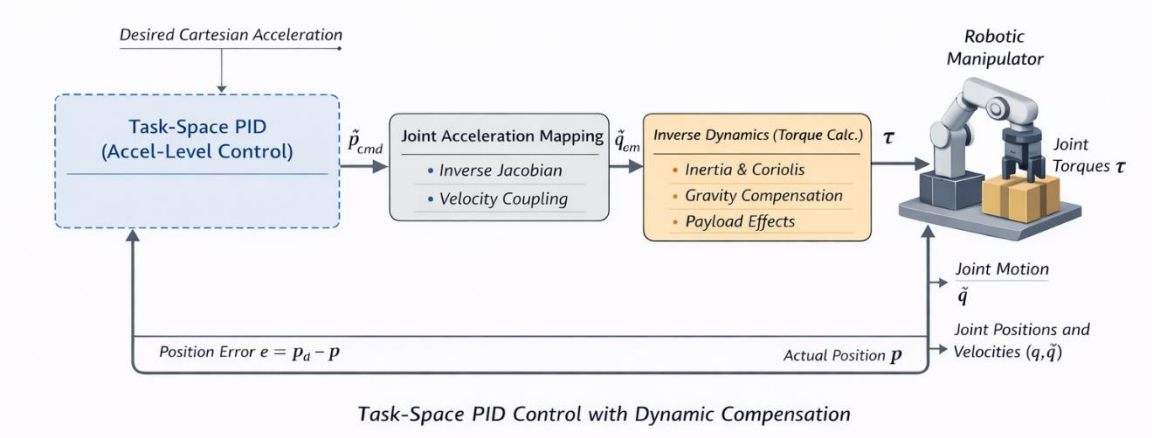


Figure 5—3: Task-Space PID Control with Dynamic Compensation

5.1.4. Actuator Bandwidth and Realization of Commanded Accelerations

The previous controller design assumes that the commanded joint accelerations are perfectly realized by the robot. In practice, however, actuators possess finite bandwidth and cannot instantaneously follow arbitrarily fast commands. To capture this limitation, the actuator dynamics are approximated using a second-order servo model characterized by a natural frequency ω_a and damping ratio ζ_a.

Under this approximation, the realized joint acceleration is modeled as the output of a second-order system driven by the commanded acceleration,

$$\ddot{q}_{real}(s) = \frac{\omega_a^2}{s^2 + 2\zeta_a\omega_a s + \omega_a^2}\ddot{q}_{cmd}(s)$$

The dynamics of the actuators are modeled as a second-order system, since servo drives are generally designed to have dominant second-order dynamics. This model accounts for the finite bandwidth of the actuators without the need for a detailed electrical-mechanical model of the motor.

This model accounts for the finite response time of the actuators and prevents the simulation from tracking the desired trajectory too quickly. Additionally, saturation limits on acceleration are added to account for torque and current limitations.

It is crucial to note that the parameters of the controller (ω_n, ζ_n) are used to specify the desired closed-loop error dynamics, while the parameters of the actuator (ω_n, ζ_n) are used to describe the physical response capability of the robot joints. When the desired closed-loop bandwidth approaches or exceeds the actuator bandwidth, the robot is unable to accurately track the desired motion, leading to an increase in the tracking error, oscillations, and torque requirements. This is observed in the simulation output, where different pole selections lead to different physical phenomena despite being theoretically stable systems. [16] [3]

5.1.5. Simulation Results and Discussion

The proposed task-space controller is evaluated through numerical simulations of the pick-and-place task. The following results first present the performance of the nominal controller configuration adopted in the final design, followed by a comparison with alternative pole placements to illustrate how the choice of closed-loop dynamics affects the physical behavior of the manipulator.

5.1.5.1. Nominal Controller Performance

The configuration of the controller chosen for the final design is reflective of a near-critically damped system, achieved through a damping ratio that is close to unity and a moderate natural frequency. Such a configuration is well-balanced in terms of achieving fast convergence, smooth motion, and reasonable torque requirements.

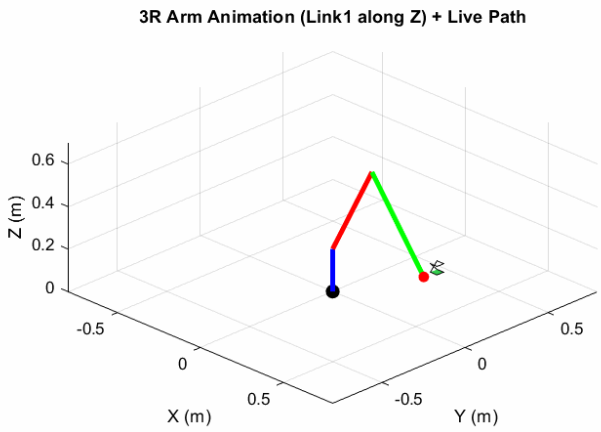


Figure 5—4: Closed-Loop 3R Manipulator Motion and Cartesian Path Tracking under Nominal Task-Space PID Control

The animation of the nominal case shows that the manipulator smoothly follows the pick-lift-transport-place sequence with no signs of oscillations and abrupt motion near the transitions. The end-effector smoothly approaches the target locations with no overshooting. This shows that the error dynamics imposed by the control translate into well-behaved motion. [16]

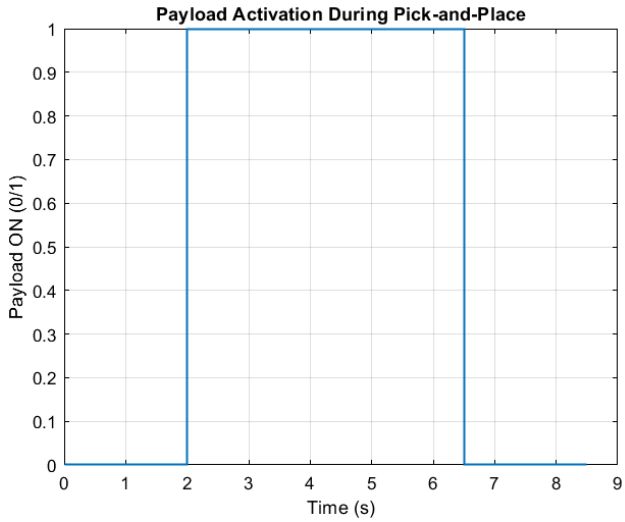


Figure 5—5: Nominal Payload Activation Profile During Pick-and-Place Task

The payload activation signal indicates the time interval during which the object is assumed to be rigidly attached to the end-effector. This window defines the phase where the manipulator dynamics change due to increased distal mass and gravity loading. All subsequent torque and tracking plots are therefore interpreted relative to this interval, since any increase in required control effort should correlate with payload activation if the dynamic model is physically consistent. [16]

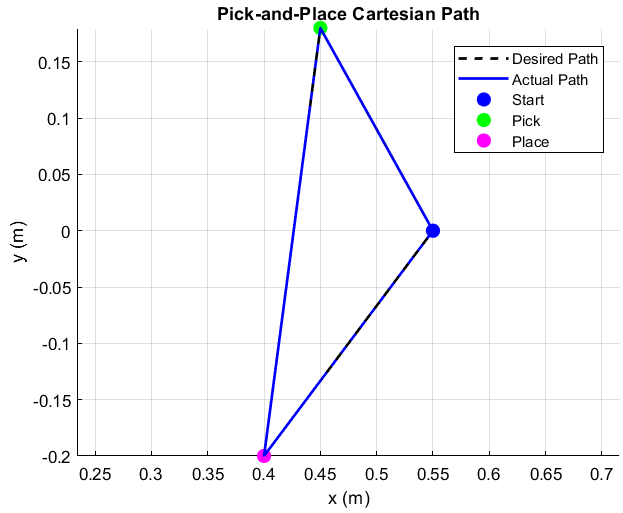


Figure 5—6: Desired vs. Actual Cartesian End-Effector Trajectory under Nominal Task-Space PID Control

The Cartesian path plot shows that the actual end-effector trajectory closely overlaps with the desired reference throughout the pick–place motion. The small deviation between desired (dashed) and actual (solid) paths indicates that the controller preserves the intended geometric motion between waypoints, not only the final positions. This strong agreement reflects the combined role of feedforward acceleration $\ddot{p}_d$in anticipating the motion and feedback terms (K_p, K_i, K_d) in rejecting residual deviations. [18]

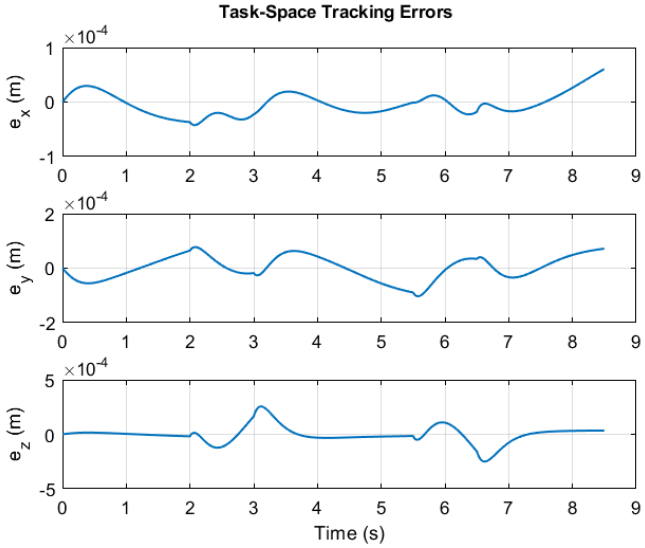


Figure 5—7: Closed-Loop Cartesian Tracking Errors under Nominal Task-Space PID Control

The task-space tracking errors remain at the sub-millimeter level, with magnitudes on the order of 10^{-4}m. Beyond the small magnitude, the error evolution is consistent with the intended well-damped second-order dynamics: short transients appear at phase transitions (where the reference acceleration regime changes) and decay smoothly without sustained oscillations. Importantly, no persistent bias is observed during the payload transport interval, indicating that the integral action and model-based compensation effectively prevent steady-state offsets under changed loading conditions. [18]

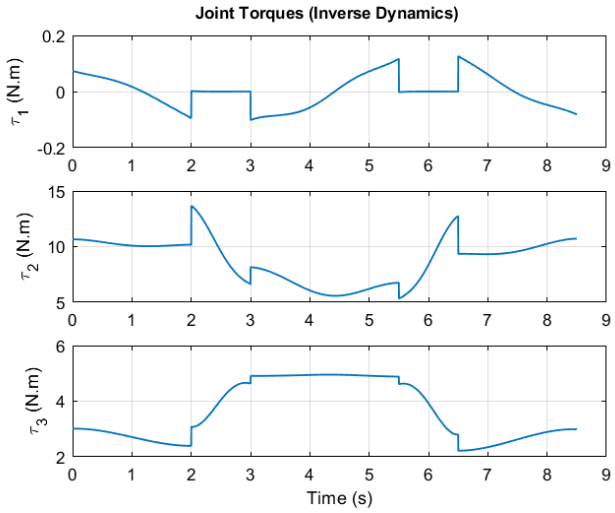


Figure 5—8: Closed-Loop Joint Torque Profiles under Nominal Task-Space PID with Inverse Dynamics Compensation

The smoothness and boundedness of the torque profiles throughout the motion also show that the control is physically realizable and stable. The higher values of the torques during the transitions and the payload-on interval can be explained by the reasons: (i) the loading due to the payload, and (ii) the reference acceleration changes during the transitions. Notably, the increase in torque does not coincide with a significant degradation in Cartesian tracking, which supports the interpretation that inverse-dynamics compensation accounts for the dominant nonlinear dynamics while the PID terms mainly correct residual modeling mismatch. [14] [18]

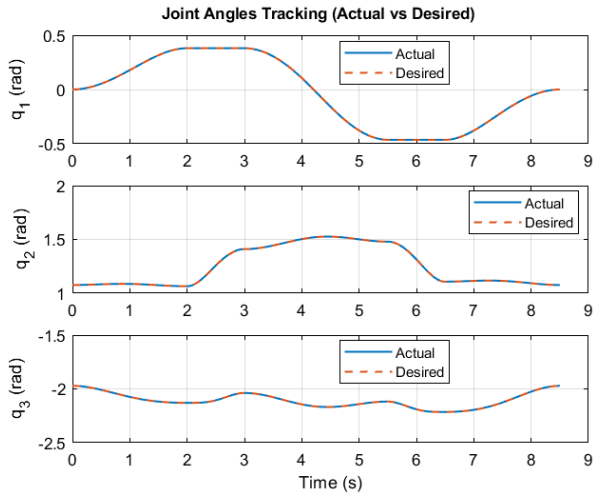


Figure 5—9: Actual vs. Inverse-Kinematics-Derived Joint Angles in the Nominal Pick-and-Place Task

The joint-angle tracking plots show close agreement between the simulated joint trajectories $q(t)$ and the corresponding reference joint angles $q_d(t)$ obtained from inverse kinematics of the desired Cartesian path. Small discrepancies are expected because the controller is defined in task space, while $q_d(t)$ is computed through an IK mapping that can be sensitive to numerical effects and does not represent an independently regulated objective. Nevertheless, the close matching confirms that the

task-space acceleration commands are consistently mapped into joint-space motion, and that the resulting configuration evolution remains coherent with the intended end-effector trajectory [20]

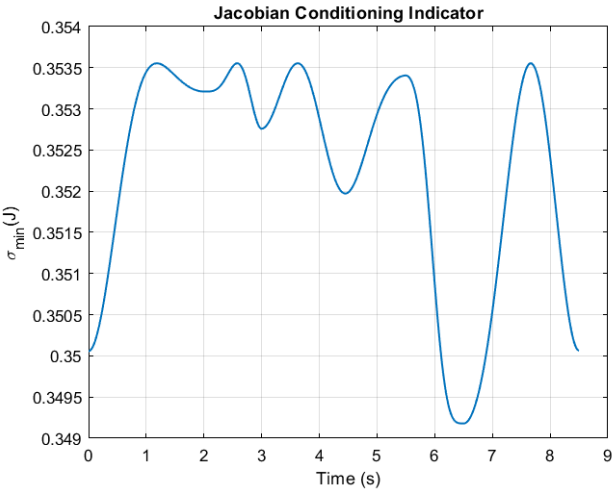


Figure 5—10: Jacobian Conditioning (Minimum Singular Value) under Nominal Task-Space PID Control

The minimum singular value of the Jacobian remains well above zero over the entire motion, indicating that the manipulator does not operate near singular configurations during the pick-and-place task. This is important because the control architecture relies on Jacobian inversion (implemented via damped least squares) to map task-space acceleration commands into joint accelerations. Stable conditioning prevents amplification of joint accelerations and torques, which explains the absence of numerical spikes in the torque plots and supports the reliability of the achieved tracking performance [21]

Quantitative Performance Metrics

Metric	**X**	**Y**	**Z**	**Unit**
RMS tracking error	2.13e−5	4.40e−5	7.80e−5	m
Max tracking error	6.06e−5	1.03e−4	2.55e−4	m
Steady-state error (mean)	3.18e−5	5.02e−5	3.15e−5	m
RMS error (payload ON)	1.85e−5	4.83e−5	8.41e−5	m
Max error (payload ON)	4.23e−5	1.03e−4	2.55e−4	m
RMS joint torque	0.055	9.03	3.76	N·m
Max joint torque	0.126	13.67	4.95	N·m
Torque increases due to payload	−5.0	−19.8	68.8	%
$\sigma(J_{min})$	0.35	0.35	0.35	—

Table 51-: Quantitative Performance Metrics for Tracking Accuracy and Joint Torque (with and without Payload)

The quantitative results shown in Table provide evidence for the qualitative observations made from the Cartesian path, error, and torque plots. The RMS Cartesian tracking errors are on the order of $10^{-5}m$ for all axes, and the maximum error is less than $2.6 \times 10^{-4}m$, thus verifying the sub-millimeter accuracy of positioning. The steady-state errors are also very close to zero, thus verifying the

effectiveness of the integral control term in compensating the residual disturbances and modeling errors.

During the payload transport process, it is noticed that there is only a slight increase in the Cartesian error value, thus verifying the effectiveness of the inverse-dynamics-based task-space control scheme in maintaining the tracking performance despite the change in dynamic behavior. The torque values also indicate that the effect of the additional payload is more on the distal joints, thus increasing the control torque without affecting the accuracy of the tracking performance. In summary, the results presented in Table X provide quantitative verification of the stability, robustness, and physical correctness of the proposed control scheme.

While the nominal configuration demonstrates accurate tracking and physically feasible torque levels, it does not by itself reveal how sensitive the system behavior is to the chosen closed-loop pole locations. To investigate this sensitivity, additional simulations were performed by varying the damping ratio and natural frequency of the imposed error dynamics. [21]

5.1.5.2. Influence of Closed-Loop Pole Placement on Physical Behavior

To better understand how the imposed error dynamics translate into observable robot behavior, three additional controller configurations were simulated. These configurations correspond to under-damped, near-critically damped, and over-damped responses obtained by modifying the damping ratio and natural frequency while keeping the control structure unchanged.

5.1.5.2.1. Case I — Aggressive Under-Damped Configuratio

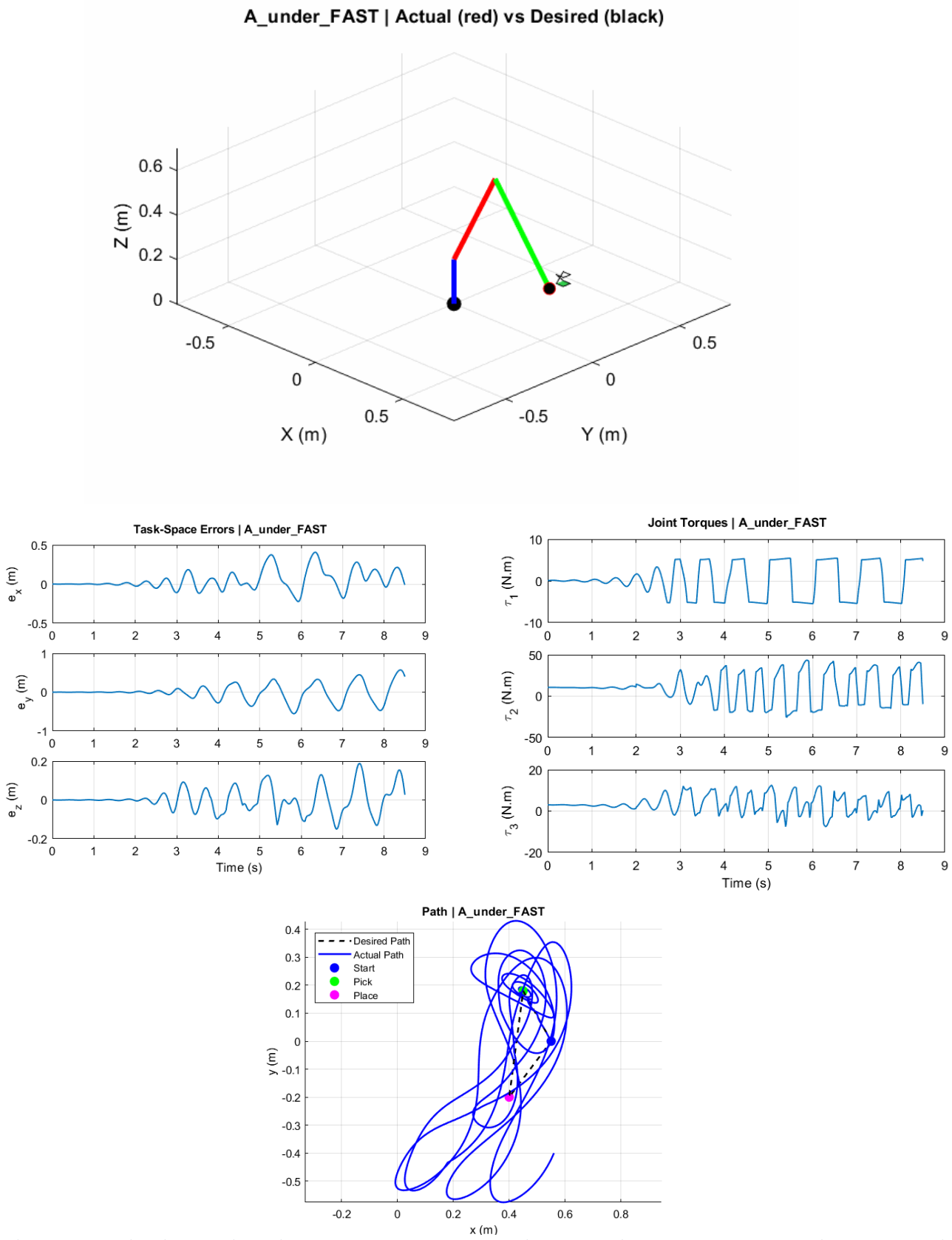


Figure 5—11: Animation and Time-History Plots of End-Effector Tracking Errors, Joint Torques, and Planar Trajectory (Aggressive Under-Damped Case)

The first configuration is for an under-damped and aggressive controller design, achieved with a damping ratio $\zeta = 0.5$ and a natural frequency $\omega_n = 12\, rad/s$. This configuration results in a significantly larger controller gain with values ($K_p = 144, K_d = 12, K_i \approx 103.7$).

This configuration in the animation results in a visibly oscillatory and irregular motion for the end-effector, especially near transitions in the trajectory and stopping points.

These oscillations in the animation verify our observation from the quantitative metrics in the following section, where the controller attempts to impose a faster error dynamic than the actuator model can physically realize, especially with the servo lag and acceleration limits.

From the quantitative metrics, the RMS Cartesian tracking errors are found to be approximately 2.6×10^{-1} m, with a maximum error of 0.6 m, which is nearly two orders of magnitude larger than those in the other configurations.

These large errors verify our observation from the animation in the last section, where the aggressive pole placement for the controller results in a significantly worse tracking performance due to the commanded accelerations exceeding the capabilities of the actuator model.

The torque statistics also highlight the challenging nature of this configuration. The RMS torque demand grows to 21 N·m, while the peak values surpass 44 N·m. This clearly indicates that the controller is continuously requiring high actuator demands. Most importantly, this does not result in any improvements in terms of tracking precision, thereby reinforcing that the performance degradation is indeed due to the actuator limitations and not the gains. [20]

From the perspective of control theory, this example demonstrates that while designing the controller gains so that the system response is fast and lightly damped, in practice, this can result in an unachievable actuator demand due to bandwidth limitations. Rather than improving the system's responsiveness, these gains only serve to increase the actuator lag and dynamic couplings, thereby compromising both precision and efficiency in the control response.

It is noted that some parts of the trajectory involve mainly vertical movements; however, the manipulator does not move along the z-axis only. Due to the nonlinear kinematic relationships between the joints of the manipulator, changes in joint angles that are necessary to achieve vertical motion cause changes in x and y coordinates of the end-effector too. As a result, errors in the x, y, and z directions occur, especially with aggressive controller gains. [4] [2]

5.1.5.2.2. Case II — Near-Critically Damped Nominal Configuration

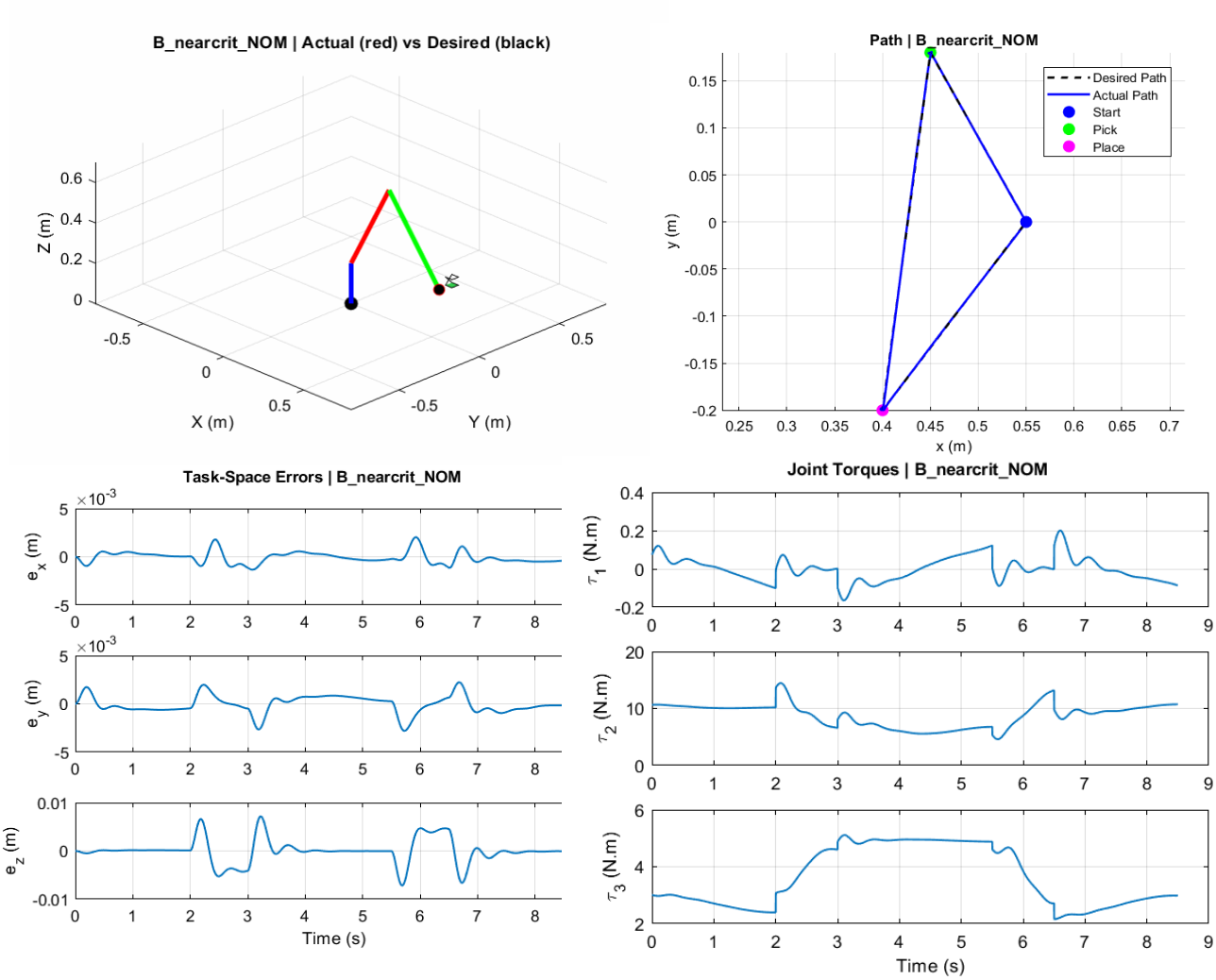


Figure 5—12: 3D Actual vs. Desired Motion, Planar (XY) Path, Task-Space Tracking Errors, and Joint Torque Profiles for the Near-Critically Damped Nominal Configuration

The second configuration corresponds to a near-critically damped controller design. To achieve this design, a damping coefficient close to unity ($\zeta = 0.95$) and a moderate natural frequency ($\omega_n = 6\,rad/s$) have been used. In this design, the controller gains ($K_p = 36, K_d = 11.4, K_i \approx 12.96$) have been tuned to achieve fast convergence with minimal oscillatory effects.

Unlike the aggressive configuration, the animation shows a smooth and well-behaved motion of the end-effector. The manipulator follows the desired pick-lift-transport-place motion sequence with no signs of oscillatory motion at the end points. The motion appears continuous across segment transitions. This suggests that the error dynamics used are consistent with the capabilities of the actuators.

The quantitative results also confirm this observation. The tracking error in terms of Cartesian coordinates has a root mean square of 2.7×10^{-3} m, and it is bounded by a maximum error of 8×10^{-3} m. The reduction in tracking error is nearly two orders of magnitude compared to the aggressive case. This supports the claim that the chosen locations for the poles are appropriate for the robot to achieve the desired dynamics without overstepping the limits of the actuators. [20]

The statistics for the torques also show how efficient the configuration is. The root mean square of the torques has reduced to 9.8 N·m, and it has a maximum of 15 N·m. The torques are not erratic or too high, which is unlike the aggressive case. The signals are smooth and physically valid for all phases of motion.

In terms of control system design, this configuration is optimal in that it matches the forced second-order error dynamics with the inherent dynamic bandwidth of the manipulator. The poles are placed far enough into the left-half plane to provide responsiveness, yet not so far as to introduce actuator delay or dynamic coupling. This results in a system that moves in a stable, efficient, and precise manner throughout the task.

This is particularly well-suited for a pick-and-place manipulation task, where smooth motion, precise stopping, and moderate actuator force are of greater importance than maximizing transient response speed. The near-critically damped configuration is therefore an optimal engineering trade-off between speed, stability, and control efficiency, and is thus the best choice for the given task.

5.1.5.2.3. Case III — Over-Damped Slow Configuration

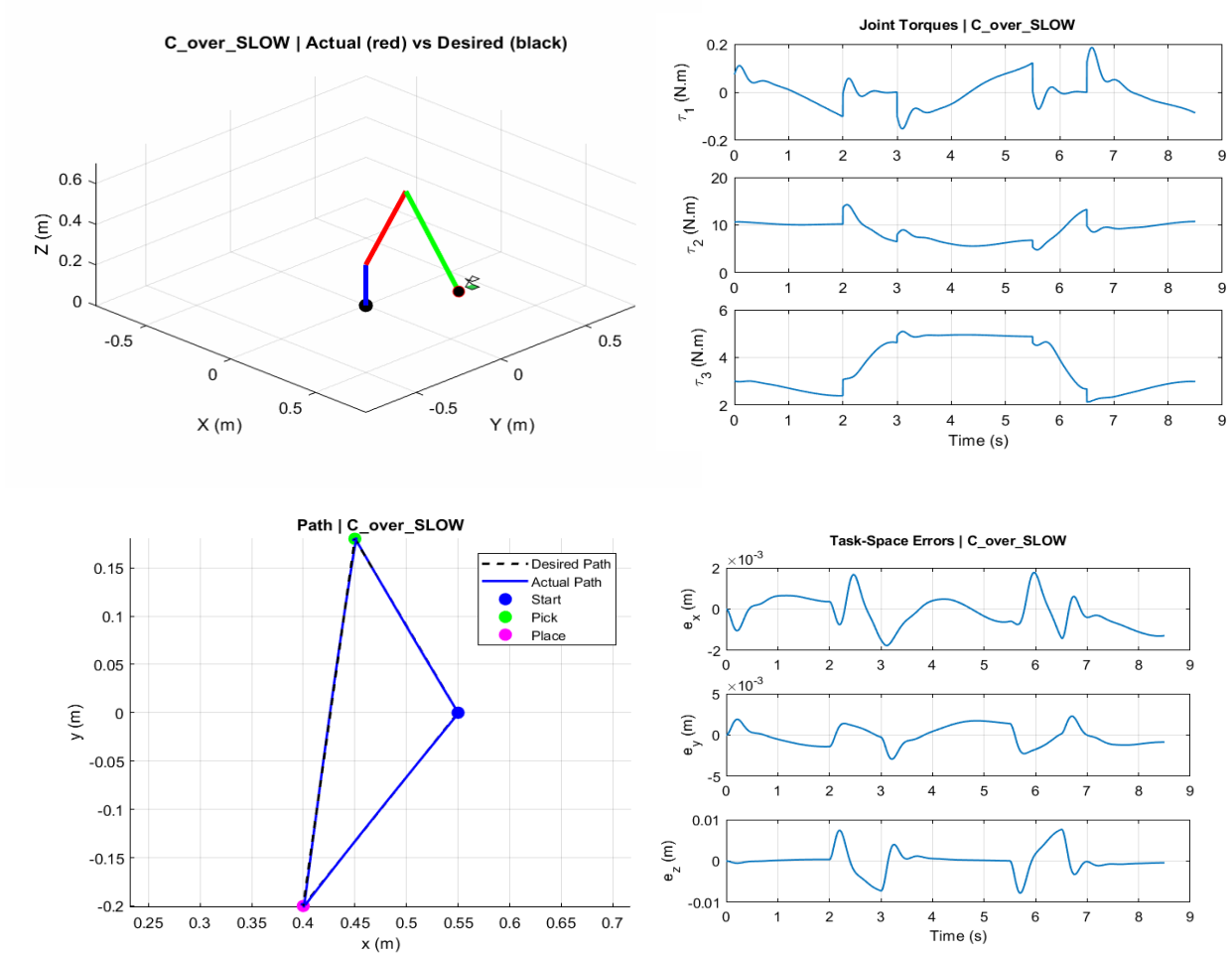


Figure 5—13: Over-Damped Natural Response of the Slow Configuration—3D Actual vs. Desired Motion, Planar (XY) Trajectory, Task-Space Tracking Errors, and Joint Torque Time Histories

The third setting represents an over-damped controller design, achieved by using a damping ratio higher than unity ($\zeta = 1.4$) and a lower natural frequency ($\omega_n = 3.6\, rad/s$). This choice results in relatively small controller gains ($K_p \approx 12.96, K_d \approx 10.08, K_i \approx 2.80$), making the control action conservative.

In the animation, this setting results in a smooth and stable motion, free from oscillations, as in the nominal setting. However, the end-effector seems to take longer to respond to trajectory changes, especially when transitioning from one motion segment to another. This is typical of an over-damped system, in which the dominant real poles cause a slower convergence of the tracking error.

The numerical results confirm this qualitative analysis. The RMS Cartesian tracking error is small, on the order of 3×10^{-3} m, and the maximum error is below 1×10^{-2} m. These values are comparable to those of the nominal setting, showing that the controller is still able to track the desired trajectory with high accuracy despite its slower response.

Moreover, the torque statistics are still at a moderate level. In particular, the RMS and peak values are similar to the ones in the nominal configuration. This is a good indication that the reduced gains in the controller have a damping effect on the system response without increasing the control actions.

From a control viewpoint, this configuration proves a point: overly conservative pole placement is a viable approach in terms of stability and accuracy. However, it is done at the expense of system responsiveness. Indeed, the motion is still smooth and physically realistic; however, the slower convergence of the error dynamics makes the task longer. [4]

In the context of a pick-and-place task, the time required to complete the task is a critical performance measure. In this regard, the overly conservative configuration is slower than needed. Accordingly, although the over-damped configuration is a stable and accurate one, it is not necessarily the best in terms of the tradeoff between speed and performance in the context of the considered task.

The pole diagram and metrics all verify the qualitative conclusions made from the animation and tracking plots. The aggressive configuration has its poles in locations associated with faster dynamics, but the control demands result in large tracking errors. The near-critically damped configuration has an optimal pole placement that balances tracking error and moderate control demands. The over-damped configuration has similar tracking error but lower system responsiveness. The importance of appropriate pole placement for system tracking error cannot be overstated. Theoretical rates of convergence are important but must be considered in relation to system actuator bandwidth. [20]

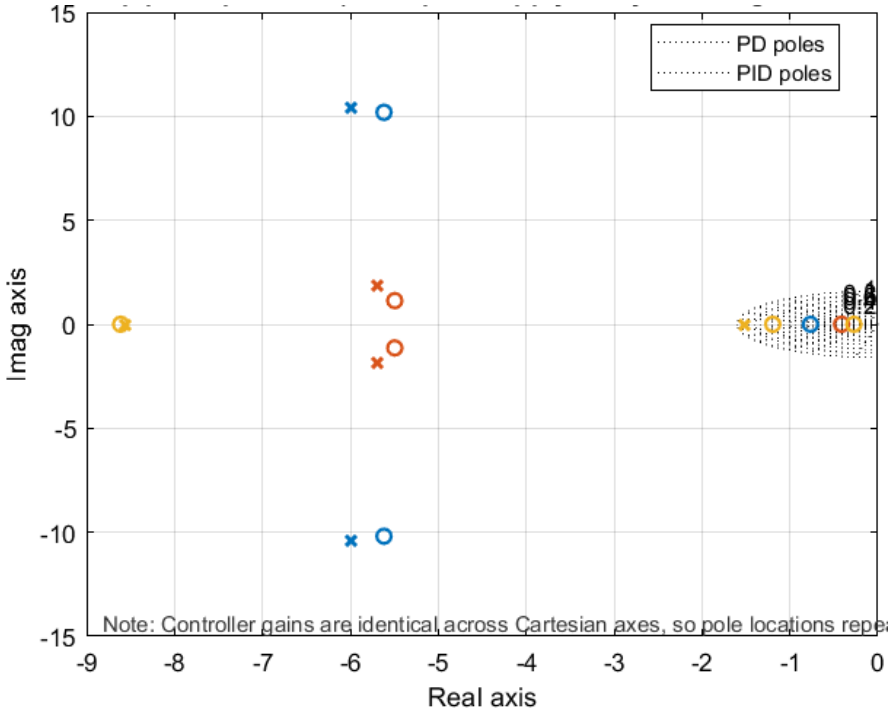


Figure 5—14: Closed-loop pole locations for the three controller configurations. Since identical scalar gains are applied to all Cartesian axes, the pole sets repeat for x, y, and z. The aggressive configuration shifts poles further left but with lower damping, while the nominal and over-damped cases exhibit progressively more stable and slower dynamics.

Case	ζ	ω_n	K_p	K_d	K_i	e_{rms}	e_{max}	τ_{rms}	τ_{max}	$\sigma_{min}(J)$
1	0.5	12	144	12	103.68	0.26388	0.61985	21.078	44.052	0.27654
2	0.95	6	36	11.4	12.96	0.0027284	0.0076345	9.8077	14.793	0.34931
3	1.4	3.6	12.96	10.08	2.7994	0.0030618	0.0080123	9.811	14.611	0.3492

Table 52-: Quantitative performance metrics for the three controller configurations, including tracking errors, torque levels, and Jacobian conditioning indicators.

All these metrics verify the qualitative observations based on the animations and tracking plots. In the aggressive configuration, the poles are well-placed for faster dynamics, but the control effort leads to large tracking errors due to the limited capabilities of the actuators. However, in the near-critically damped configuration, a well-balanced pole placement minimizes the tracking errors with reasonable control efforts. In the over-damped configuration, the accuracy is similar, but the responses are not as sharp as in the near-critically damped configuration. This shows the importance of pole placement in considering not only the theoretical rates of convergence but also the capabilities of the actuators. [16]

5.1.6. PID Performance with Actuator Constraints

5.1.6.1. Influence of Torque Saturation

In a physical system, actuator torque is bounded. Therefore, the realized torque satisfies

$$\tau_{cmd} = sat(\tau)$$

which can be written as

$$\tau = \tau_{cmd} + \Delta\tau$$

where $\Delta\tau$ represents the torque error introduced by saturation.

Substituting into the manipulator dynamics,

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau_{cmd} + \Delta\tau$$

For the task–space PID controller, the commanded torque is designed to realize the nominal acceleration $\ddot{q}_{cmd}$.

After cancellation of the nominal dynamics,

$$\Delta\tau = M(q)(\ddot{q}_{cmd} - \ddot{q})$$

which yields

$$\ddot{q} = \ddot{q}_{cmd} + M^{-1}(q)\Delta\tau$$

Defining the disturbance

$$d_{sat}(t) = M^{-1}(q)\Delta\tau$$

the realized acceleration becomes

$$\ddot{q} = \ddot{q}_{cmd} + d_{sat}(t)$$

Hence, torque saturation introduces an additive disturbance that perturbs the nominal PID closed-loop dynamics. [22] [3]

5.1.6.2. Influence of Actuator Bandwidth (Lag)

The actuator is modeled as a dynamic system that limits how fast the commanded torque can be realized. Let the realized torque follow the first-order model

$$\dot{\tau}_{real} = \frac{1}{T_a}(\tau_{cmd} - \tau_{real})$$

This implies that the realized acceleration differs from the commanded one. Define the actuator tracking error

$$e_{act}(t) = \ddot{q}_{cmd}(t) - \ddot{q}_{real}(t)$$

Then

$$\ddot{q}_{real} = \ddot{q}_{cmd} - e_{act}(t)$$

For the PID controller, the commanded acceleration can be written in joint space as

$$\ddot{q}_{cmd} = \ddot{q}_d + K_p e + K_d \dot{e} + K_i \int e\, dt$$

Therefore, the realized dynamics become

$$\ddot{q} = \ddot{q}_d + K_p e + K_d \dot{e} + K_i \int e\, dt - e_{act}(t)$$

5.1.6.3. Combined Effect of Actuator Limitations

When both torque saturation and actuator lag are present, the physical acceleration can be approximated as

$$\ddot{q} = \ddot{q}_d + K_p e + K_d \dot{e} + K_i \int e\, dt + d_{sat}(t) - e_{act}(t)$$

Therefore, it can be said that actuator limits are responsible for creating disturbance terms that directly affect the nominal performance of a PID controller in a closed-loop system. The steady-state error remains unchanged as a result of the integral controller. [20]

5.1.6.4. Results

5.1.6.4.1. plots

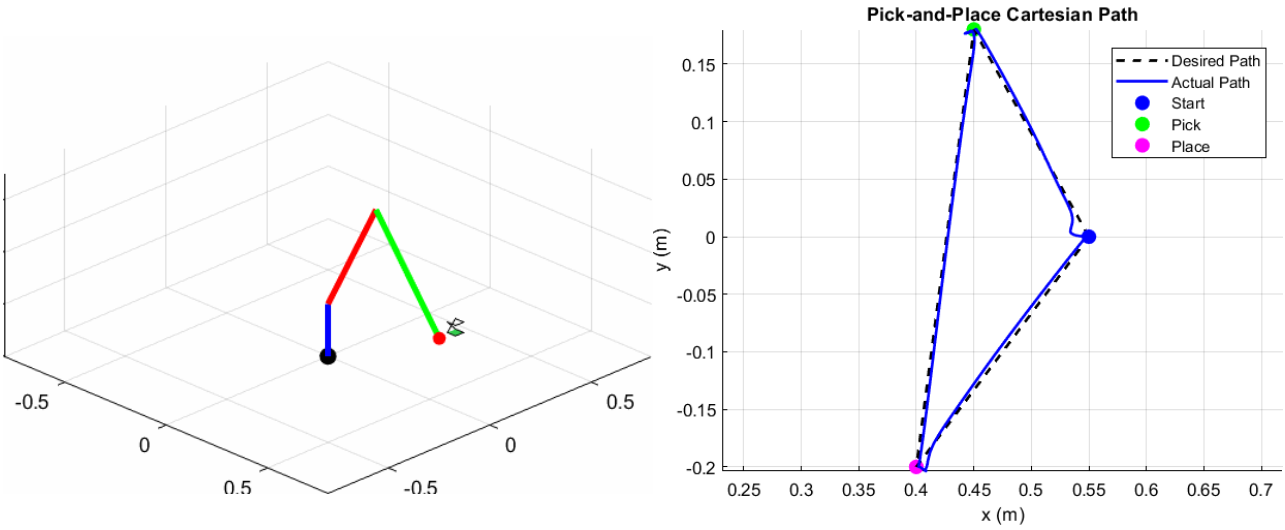


Figure 5—15: Visualization and Cartesian Trajectory Tracking Results of the PID-Controlled Manipulator under Actuator Constraints

The figure illustrates the desired and actual Cartesian trajectory followed by the end-effector during the pick-and-place operation. From the figure, it is evident that the robot accurately tracks the desired trajectory in all the motion segments. Deviations are small and occur mostly at the transition states of

the trajectory. This is due to higher acceleration requirements at the transition states. This deviation in the robot's movement is in accordance with the lag state of the actuator.

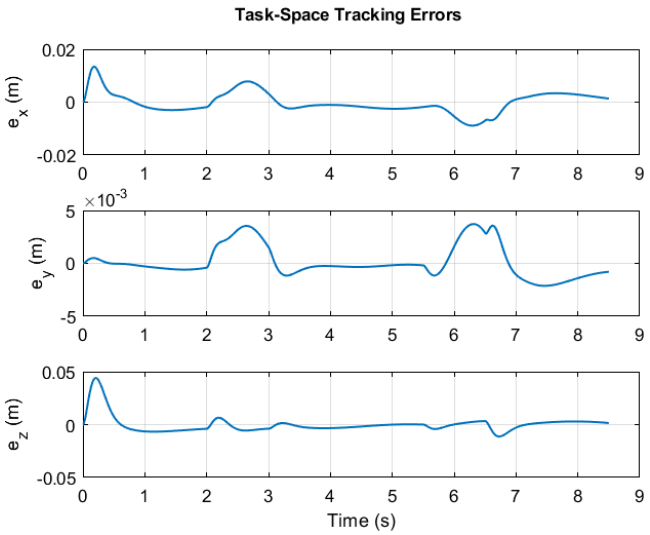


Figure 5—16: Task-Space Tracking Errors under Actuator Constraints (PID Control)

It is evident from the error plot that the error signals are bounded. The error converges quickly after the transition states. The maximum deviation in the robot's movement is in the vertical direction during the lifting state. This is due to higher acceleration requirements during the lifting state. The error converges quickly after the transition states.

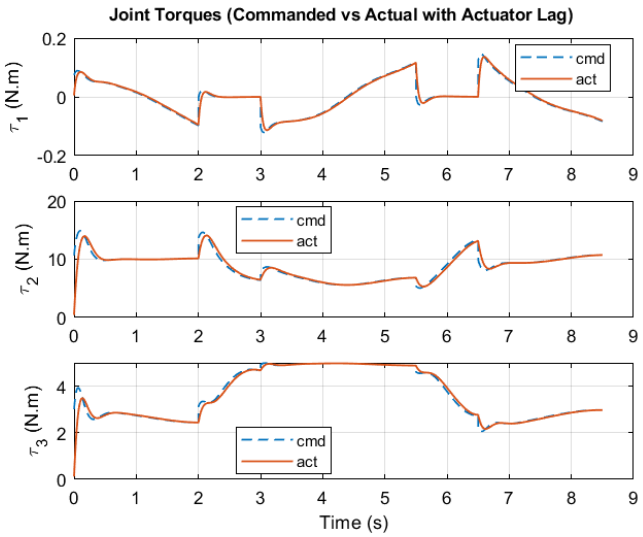


Figure 5—17: Commanded vs. Realized Joint Torques under Actuator Lag (PID Control)

This comparison between commanded and realized torques shows how actuator dynamics play a role. Realized and commanded torques appear smoother and slightly delayed, which is a characteristic of actuator bandwidth limitations. Yet, the actuator is able to track the command without any oscillations or instability, which shows that the chosen PID gains are compatible with the actuator model.

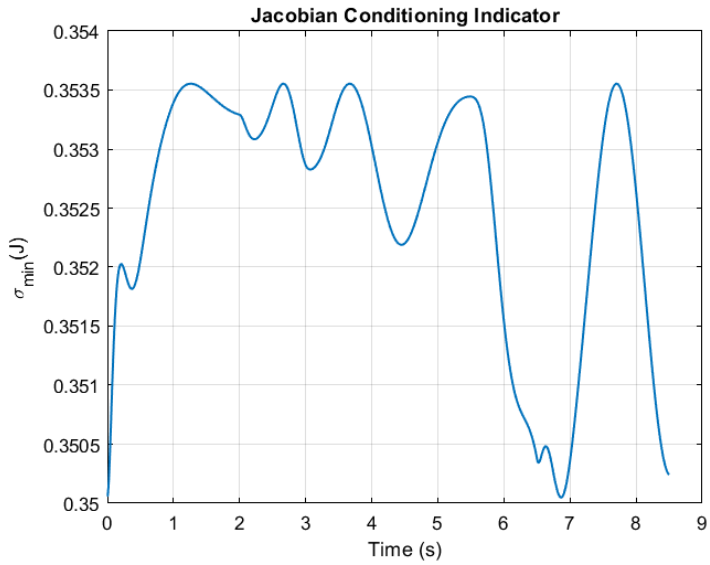


Figure 5—18: Jacobian Conditioning Indicator (Minimum Singular Value) under Actuator-Constrained PID Control

The minimum singular value of the Jacobian is always significantly above zero. This shows that the robot is moving away from singular configurations. Thus, it can be concluded that tracking errors are mainly caused by actuator dynamics and transitions.

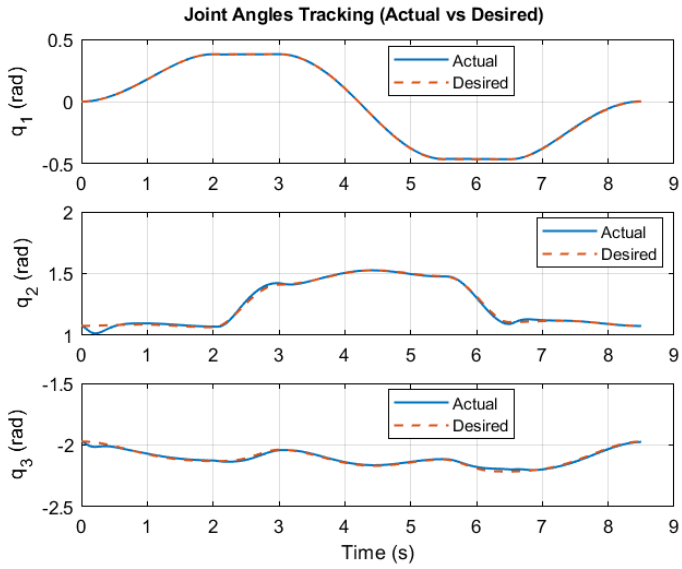


Figure 5—19: Joint-Space Tracking Performance (Actual vs Desired) under Actuator-Constrained PID Control

The actual joint trajectories track the desired inverse kinematics solution very closely for all joints. Some minor delays occur in response to fast changes in motion due to bandwidth limitations of the actuator system. However, no drift or steady-state error is found in the system response, showing that the integral part of the PID controller successfully cancels any persistent disturbances. [20] [18]

5.1.6.4.2. PID Controller Performance

5.1.6.4.2.1. Tracking Accuracy

The PID controller achieved millimeter-level tracking accuracy over the entire pick-and-place task.

The Cartesian RMS errors were

$$e_{rms,x} = 4.0\ mm, \quad e_{rms,y} = 1.5\ mm, \quad e_{rms,y} = 8.1\ mm$$

The global position error norm was

$$\|e\|_{RMS} = 9.1\ mm$$

The largest instantaneous deviation occurred in the vertical direction and reached

$$\|e\|_{MAX} = 4.6\ cm$$

This peak appears during fast lifting phases where actuator lag limits the achievable joint acceleration. Despite these transient peaks, the steady-state error remained below 333 mm in all directions, confirming that the integral action compensates residual disturbances.

5.1.6.4.2.2. Effect of Payload

During payload transport, the RMS tracking error decreased to

$$\|e\|_{RMS}^{payload} = 5.2\ mm$$

and the maximum deviation was approximately 1 cm.

This reduction in tracking error can be attributed to the increased inertia of the arm when carrying the payload, which smooths the motion and reduces sensitivity to actuator lag. [18] [2]

5.1.6.4.2.3. Control Effort

The realized actuator torques remained within feasible limits throughout the task.

The RMS torques were

$$\tau_1^{RMS} = 0.056\ N.m, \quad \tau_2^{RMS} = 9.1\ N.m, \quad \tau_3^{RMS} = 3.8\ N.m$$

The highest torque demand occurred at joint 2, which carries most of the arm mass, reaching about $14\ N.m$. The payload mainly affected the distal joint. Joint-3 RMS torque increased by nearly 70%, indicating that payload dynamics strongly influence joints closer to the end-effector.

5.1.6.4.2.4. Kinematic Feasibility

The minimum singular value of the Jacobian remained near 0.35 during the entire trajectory. This indicates that the manipulator operated away from singular configurations and that the observed tracking errors are primarily caused by actuator dynamics and trajectory accelerations rather than kinematic ill-conditioning. [2]

5.1.6.4.2.5. Overall Interpretation

The PID controller provides stable tracking under actuator dynamics and payload variation.

The results show:

- millimeter-level steady-state accuracy
- moderate transient errors during fast vertical motion
- realistic actuator torque levels
- clear sensitivity of distal joints to payload mass

These results establish the PID controller as a consistent baseline controller for the pick-and-place task. [2]

5.2. LQR Controller Design

5.2.1. LQR Controller Overview

While the task-space PID controller provides an intuitive way to shape the transient response of the Cartesian error, its gains are selected indirectly by imposing a desired second-order behavior. This approach works well in practice, but it does not explicitly address how tracking accuracy should be balanced against control effort.
For this reason, an optimal control formulation based on the Linear Quadratic Regulator is considered as an alternative perspective on the same problem.

Starting from the Cartesian tracking error

$$e = p_d - p$$

its second derivative can be written as

$$\ddot{e} = \ddot{p}_d - \ddot{p}$$

Since the controller operates at the acceleration level and inverse dynamics are used to generate joint torques that realize commanded accelerations, it is reasonable to approximate the realized motion by the commanded one, i.e.

$$\ddot{p} \approx \ddot{p}_{cmd}.$$

Under this assumption, the error dynamics become

$$\ddot{e} = \ddot{p}_d - \ddot{p}_{cmd}$$

Defining the corrective input as

$$u = \ddot{p}_{cmd} - \ddot{p}_d,$$

one obtains the simplified relation

$$u = \ddot{p}_{cmd} - \ddot{p}_d = -(\ddot{p}_d - \ddot{p}_{cmd}) = -\ddot{e}$$

This result shows that, at the level of Cartesian error dynamics, the system behaves approximately as a double integrator driven by the corrective acceleration. This approximation assumes that inverse-dynamics compensation is sufficiently accurate and that actuator bandwidth is high enough for commanded Cartesian accelerations to be effectively realized.

To express this in a form suitable for optimal control, the state vector is defined as

$$x = \begin{bmatrix} e \\ \dot{e} \end{bmatrix},$$

which yields the linear model

$$\dot{x} = Ax + Bu$$

with

$$A = \begin{bmatrix} 0_{3\times3} & I_{3\times3} \\ 0_{3\times3} & 0_{3\times3} \end{bmatrix}, \qquad B = \begin{bmatrix} 0_{3\times3} \\ I_{3\times3} \end{bmatrix}$$

Rather than prescribing the closed-loop poles directly, the LQR framework determines the feedback law by minimizing the quadratic functional

$$J = \int (x^T Q x + u^T R\, u)\, dt$$

This formulation has a clear interpretation: the matrix Q penalizes tracking errors in position and velocity, while R penalizes aggressive accelerations. In other words, the controller is not tuned to match a desired pole location but to achieve an optimal compromise between precision and control effort.

The optimal feedback takes the linear form

$$u = -Kx$$

where the gain matrix K is obtained from the solution of the algebraic Riccati equation

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0$$

Through

$$K = R^{-1}B^\top P$$

Substituting this feedback into the state equation yields the closed-loop system

$$\dot{x} = (A - BK)x$$

showing that the convergence of the Cartesian error is governed by the eigenvalues of $A - BK$. From this viewpoint, the LQR controller can be seen as an indirect pole-placement method, where the pole locations emerge from the choice of Q and R rather than being imposed explicitly as in the PID design.

Reintroducing the feedforward term leads to the final command law

$$\ddot{p}_{cmd} = \ddot{p_d} - K \begin{bmatrix} e \\ \dot{e} \end{bmatrix}$$

This formula reflects the structure used before for the PID controller: The PID and LQR controllers solve the same tracking problem but from two different points of view.

The design of the PID controller requires the desired dynamic behavior to be specified explicitly by choosing gains according to the transient specifications of damping ratio and natural frequency. By contrast, the LQR controller computes the feedback gains on the basis of an optimality criterion. [23]

5.2.2. Choice of Weighting Matrices

The performance of an LQR controller is determined not only by the system model but also by the selection of the weighting matrices Q and R, which encode the relative importance of tracking accuracy versus control effort. Rather than prescribing a desired pole configuration directly, the LQR formulation allows these matrices to shape the closed-loop behavior implicitly through an optimization criterion.

In the present tracking problem, the state vector is defined as

$$x = \begin{bmatrix} e \\ \dot{e} \end{bmatrix},$$

where $e \in \mathbb{R}^3$ denotes the Cartesian position error and $\dot{e}$ its time derivative. The quadratic performance index

$$J = \int (x^T Q x + u^T R\, u)\, dt$$

penalizes both tracking deviations and corrective accelerations.

To ensure accurate positioning of the end-effector, larger weights are assigned to the position error components of the state. This reflects the physical priority of the pick-and-place task, where spatial accuracy at the grasp and placement locations is more critical than minimizing transient velocity deviations. Consequently, the matrix Q is chosen in block-diagonal form,

$$Q = \begin{bmatrix} q_p I_3 & 0 \\ 0 & q_v I_3 \end{bmatrix},$$

with $q_p > q_v$, so that position errors dominate the cost function.

The matrix R penalizes the corrective acceleration input

$$u = \ddot{p}_{cmd} - \ddot{p}_d,$$

and therefore, regulates the aggressiveness of the controller. A smaller R leads to faster error convergence but increases actuator effort, whereas a larger R produces smoother motion at the expense of slower response. In this work, R is selected as:

$$R = r I_3,$$

which preserves isotropic behavior across Cartesian axes while allowing the scalar parameter r to tune the balance between responsiveness and control energy.

The resulting gain matrix K, obtained from the Riccati equation, therefore reflects a compromise between tracking precision and physically realistic torque demands. Unlike the PID design, where gains are interpreted directly in terms of transient response parameters, the LQR formulation embeds these trade-offs within the cost function itself. This provides a systematic and physically interpretable way to shape the robot's motion while accounting for actuator limitations and dynamic coupling.

This selection, therefore, offers a tangible means for the control behavior to be influenced. Increasing the values in matrix Q causes errors to converge faster, while increasing values in matrix R makes the

control actions smoother. From this understanding, the LQR form immediately reveals the inherent trade-off between performance and control effort, enabling the controller to be adjusted in a physically meaningful fashion. [23]

5.2.3. Implementation of the LQR Controller

5.2.3.1. Mapping to Joint Space

The commanded joint accelerations are obtained by mapping the Cartesian acceleration command to joint space using a damped least-squares (DLS) inverse of the Jacobian:

$$\ddot{q}_{cmd} = (J^T J + \lambda I)^{-1} J^T \left(p_{cmd} - \dot{J}\dot{q}\right)$$

where $\lambda > 0$ is a damping factor introduced to improve numerical robustness near kinematic singularities and to avoid excessive joint accelerations.

Although the control objective is formulated in Cartesian space, the final LQR feedback is effectively realized in joint space through this mapping. This hybrid formulation preserves task-space optimality while improving stability in the presence of nonlinear kinematics and time-varying Jacobians.

5.2.3.2. Dynamics and Forward Simulation

The joint torques required to achieve the commanded accelerations are computed using inverse dynamics:

$$\tau = M(q)\ddot{q}_{cmd} + C(q, \dot{q})\dot{q} + G(q)$$

The resulting motion is simulated through forward dynamics:

$$\ddot{q} = M^{-1}(\tau - C(q, \dot{q})\dot{q} - G(q))$$

and joint positions and velocities are updated via numerical integration.

This simulation loop closes the dynamic model and allows evaluation of the closed-loop control performance. [23]

5.2.3.3. Practical Stabilization Strategy

Initial simulations based purely on task-space LQR feedback revealed sensitivity to model nonlinearities and Jacobian variations, which occasionally resulted in oscillatory responses. To improve robustness, the feedback was instead applied in joint space using joint position and velocity errors.

This modification does not alter the optimal structure of the controller, but improves numerical robustness by avoiding amplification of Cartesian errors through Jacobian inversion and by better accommodating model nonlinearities. This hybrid implementation maintains the optimal tuning characteristics of the LQR formulation while providing significantly improved numerical stability and consistent tracking performance across the workspace. [23]

5.2.4. Simulation Results

The LQR weighting matrices were selected interactively for three cases as reported below.

Case	Q_q	$\dot{Q}_q$	R
A(Conservative)	120	10	2.5
B (Balanced)	260	25	1.2
C (Aggressive)	420	45	0.8

Table 5-3: Selected LQR Weighting Matrices for the Three Control Cases

5.2.4.1. Nominal LQR Performance

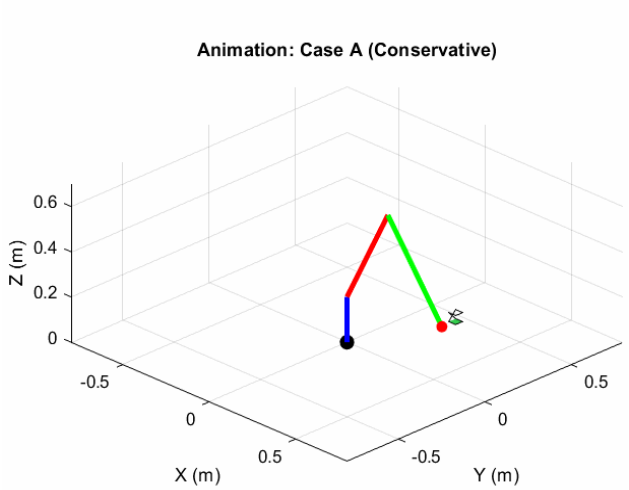


Figure 5—20: Three-Dimensional Motion Trajectory of the Mobile Manipulator under Nominal LQR Control (Case A – Conservative)

As expected, the animation confirms the successful execution of the full pick-and-place task. The manipulator motion is still smooth and well-damped. There are no oscillations or unstable behavior. This is true for both balanced and aggressive configurations. This confirms the stability of the suggested framework in terms of executing the full pick-and-place task despite the choice of LQR weight values. [23]

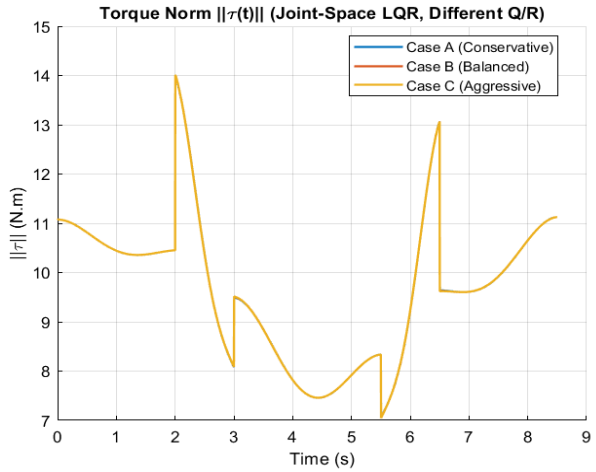


Figure 5—21: Norm of the joint torque vector for different LQR weighting configurations.

As expected, the torque profiles show similar behavior in all configurations. The maximum torque values occur during the dynamic phases. In all cases, the maximum torque norm remains bounded at approximately 14 N.m. This suggests that increasing the LQR state weighting improves tracking accuracy without increasing the actuator effort. This is due to the inverse dynamics compensation.

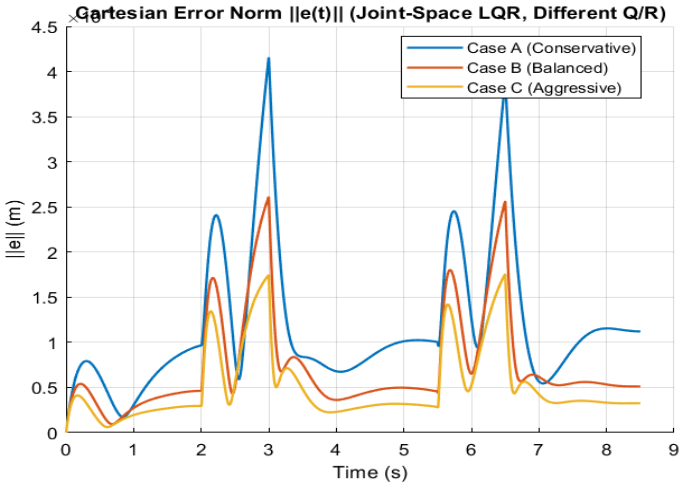


Figure 5—22: Cartesian tracking performance for conservative, balanced, and aggressive LQR configurations.

The results obtained reveal the following facts: the larger the state weighting in the LQR design, the better the accuracy in the tracking of the desired trajectory. It should also be noted that the aggressive configuration has the smallest tracking error, while the conservative configuration has the smoothest but less accurate motion. Finally, the balanced configuration has a good trade-off between accuracy and control effort.

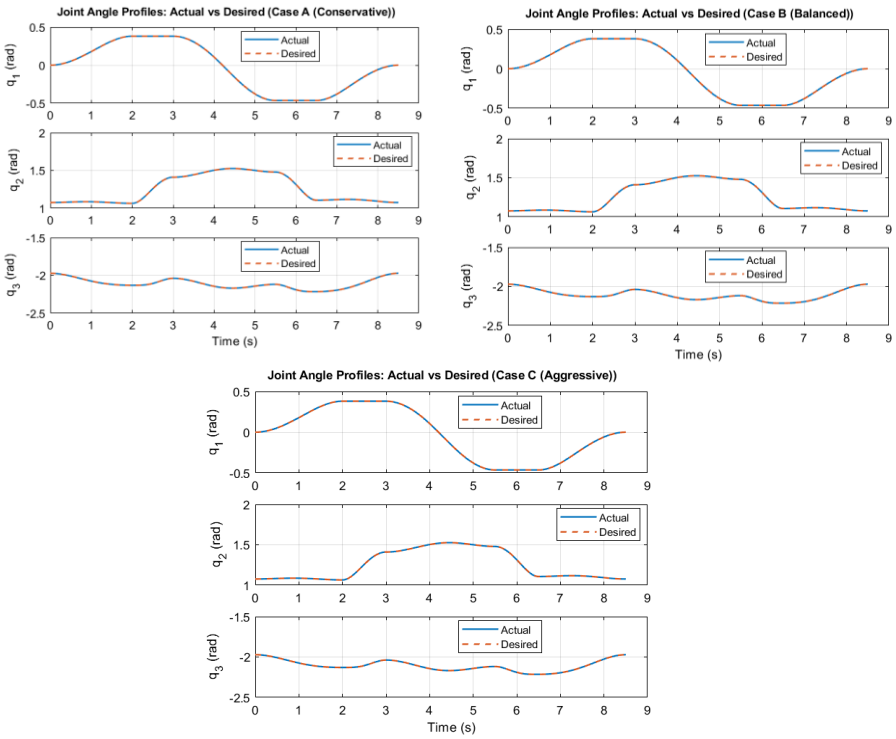


Figure 5—23: Comparative Joint Angle Tracking Performance under Conservative, Balanced, and Aggressive Weighting Strategies

The joint trajectories are smooth and continuous during the motion. Moreover, there are no abrupt changes in the joint variables. This fact confirms the dynamically feasible motion generated by the controller, in accordance with the mechanical constraints of the manipulator. Similar joint trajectories are obtained in all the tested configurations, with slight variations in the convergence rate.

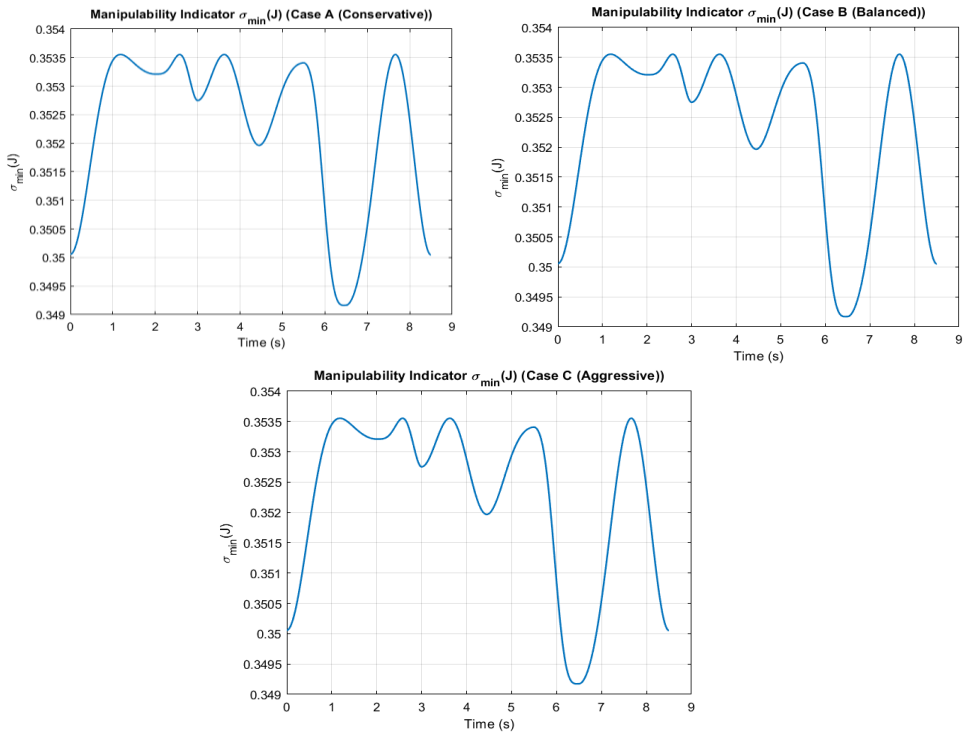


Figure 5—24: Comparative Manipulability Analysis Based on the Minimum Singular Value of the Jacobian under Different Weighting Strategies (Cases A–C)

The minimum singular value of the Jacobian matrix is always positive during the execution of the task and hence the manipulator is avoiding kinematic singularities. This is a verification of the mapping performed using the Jacobian matrix and ensures that the damped least-squares method is numerically robust enough. The values are lower in extended configurations, but there is no singularity.

Case	**$\|\|e\|\|$ RMS (m)**	**$\|\|e\|\|$ MAX (m)**	**$\|\|\tau\|\|$ RMS (N·m)**	**$\|\|\tau\|\|$ MAX (N·m)**
Case A (Conservative)	1.41×10^{-4}	4.16×10^{-4}	9.78	13.99
Case B (Balanced)	9.10×10^{-5}	2.61×10^{-4}	9.78	14.01
Case C (Aggressive)	6.47×10^{-5}	1.76×10^{-4}	9.78	14.01

Table 5-4: summarizes the RMS and maximum Cartesian tracking errors and joint torque norms for all tested configurations

The numerical metrics confirm the qualitative observations. Increasing the state weighting matrices leads to a consistent reduction in both RMS and maximum tracking errors. In contrast, the joint torque norms remain nearly unchanged, indicating that improved tracking performance is achieved without

increased control effort. This demonstrates the effectiveness of inverse dynamics compensation combined with optimal feedback. [2] [23]

This formulation establishes the LQR controller as a physically consistent alternative to the PID design, providing an optimal feedback structure that directly accounts for the trade-off between tracking accuracy and actuator effort. The resulting controller is therefore well suited for comparison with the PID approach in the simulation results presented in the following section.

5.2.4.2. LQR Performance with Actuator Constraints

The actuator dynamics were modeled as a second-order system with natural frequency $\omega_a = 8\,rad/s$ and damping ratio $\zeta = 0.8$. Torque saturation was imposed at $\tau_{max} = [14, 14, 14]$ N·m.

These parameters were selected to reflect realistic actuator bandwidth and torque limits of medium-size industrial manipulators.

Case A (conservative weighting):

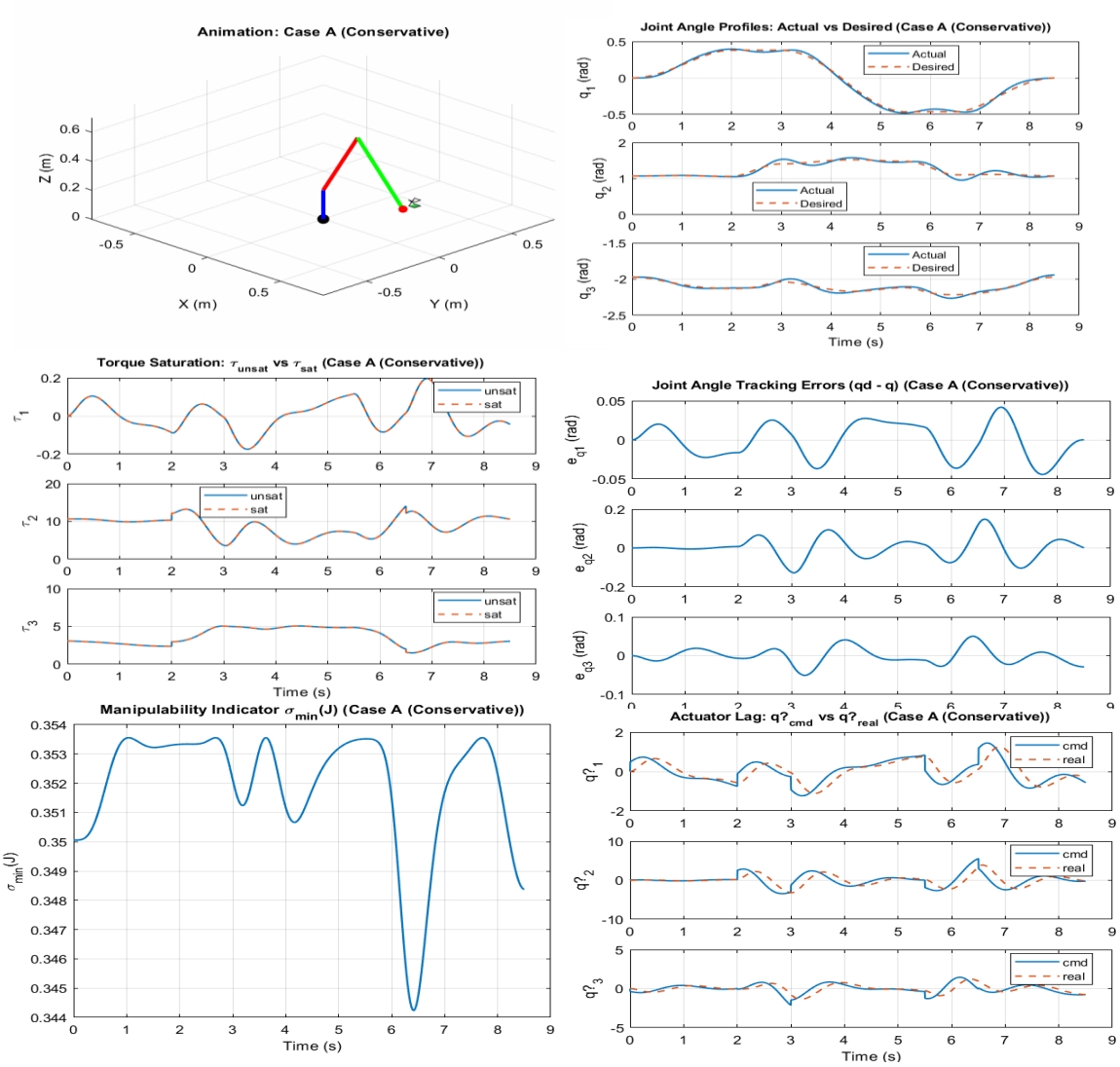


Figure 5—25: Comprehensive Closed-Loop Kinematic and Dynamic Performance Assessment of the Mobile Manipulator (Case A – Conservative Weighting)

The joint trajectories closely follow the desired references, with small tracking errors that remain bounded throughout the motion. The error plots confirm smooth convergence without oscillatory behavior, indicating adequate damping introduced by the conservative gain selection. The manipulability index remains strictly positive and well above singular values, showing that the controller maintains the manipulator within a safe region of the workspace. The actuator comparison plots indicate that the realized accelerations closely match the commanded ones, demonstrating that actuator dynamics introduce negligible distortion under nominal conditions. Overall, the conservative tuning provides stable and physically feasible tracking while avoiding excessive control effort. [23]

Case B (balanced weighting):

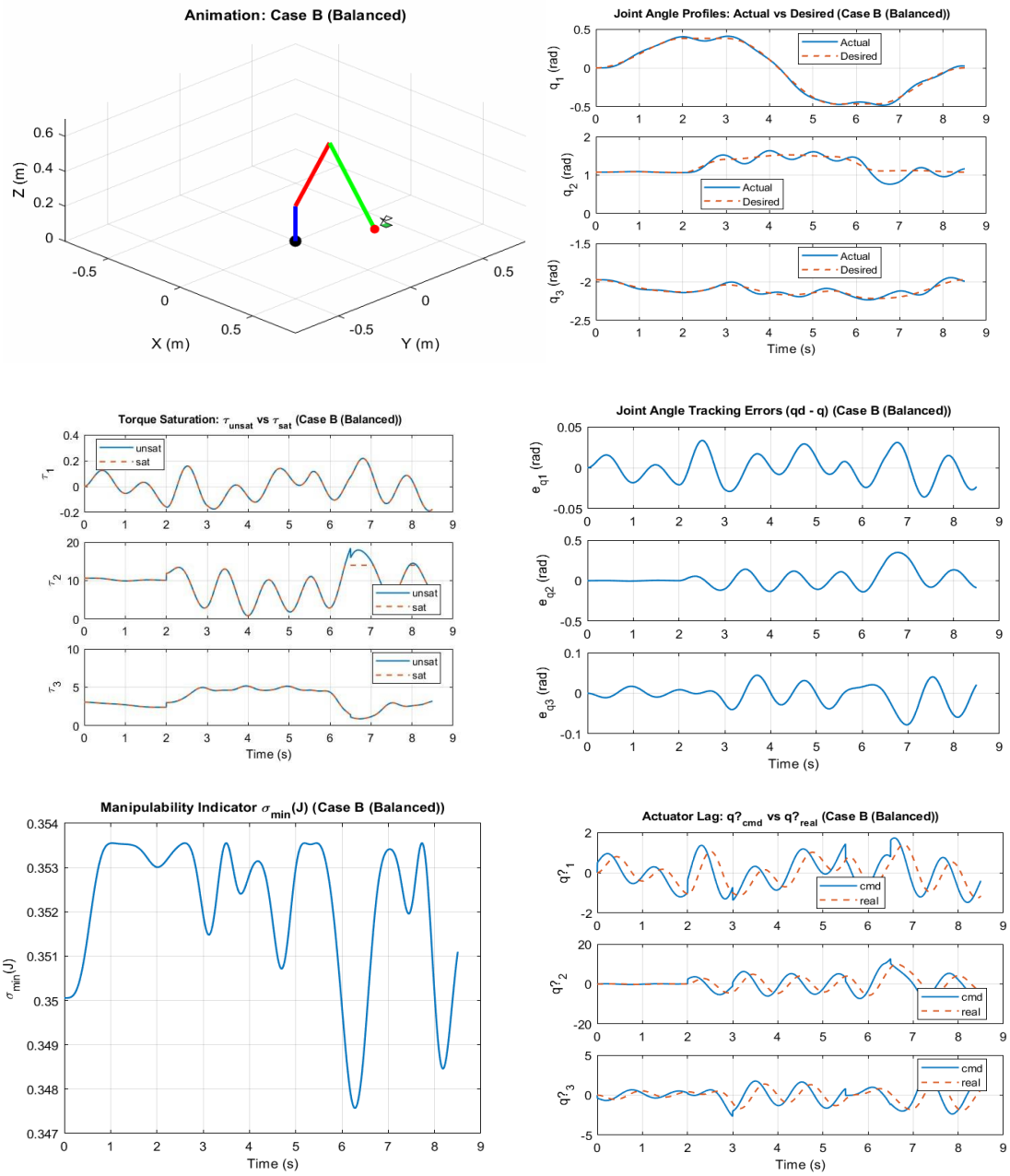


Figure 5—26: Combined Manipulability Evolution and Actuator Dynamics Assessment under Balanced Control Weighting (Case B)

The balanced LQR tuning achieves acceptable tracking performance, but the plots indicate that the controller operates close to actuator and kinematic limits. The presence of noticeable actuator lag demonstrates that the commanded accelerations excite the bandwidth limits of the actuators. This effect, combined with partial torque saturation, reduces the ability of the controller to realize the desired corrective actions, leading to increased oscillations in the tracking error.

Although the manipulator remains away from singular configurations, the deeper drops in the manipulability index suggest that the motion approaches regions where the kinematic sensitivity increases.

Overall, the balanced tuning represents a transition between nominal optimal behavior and physically constrained operation, explaining why it is more sensitive to disturbances and modeling inaccuracies than the conservative tuning. [23]

Case C (aggressive weighting):

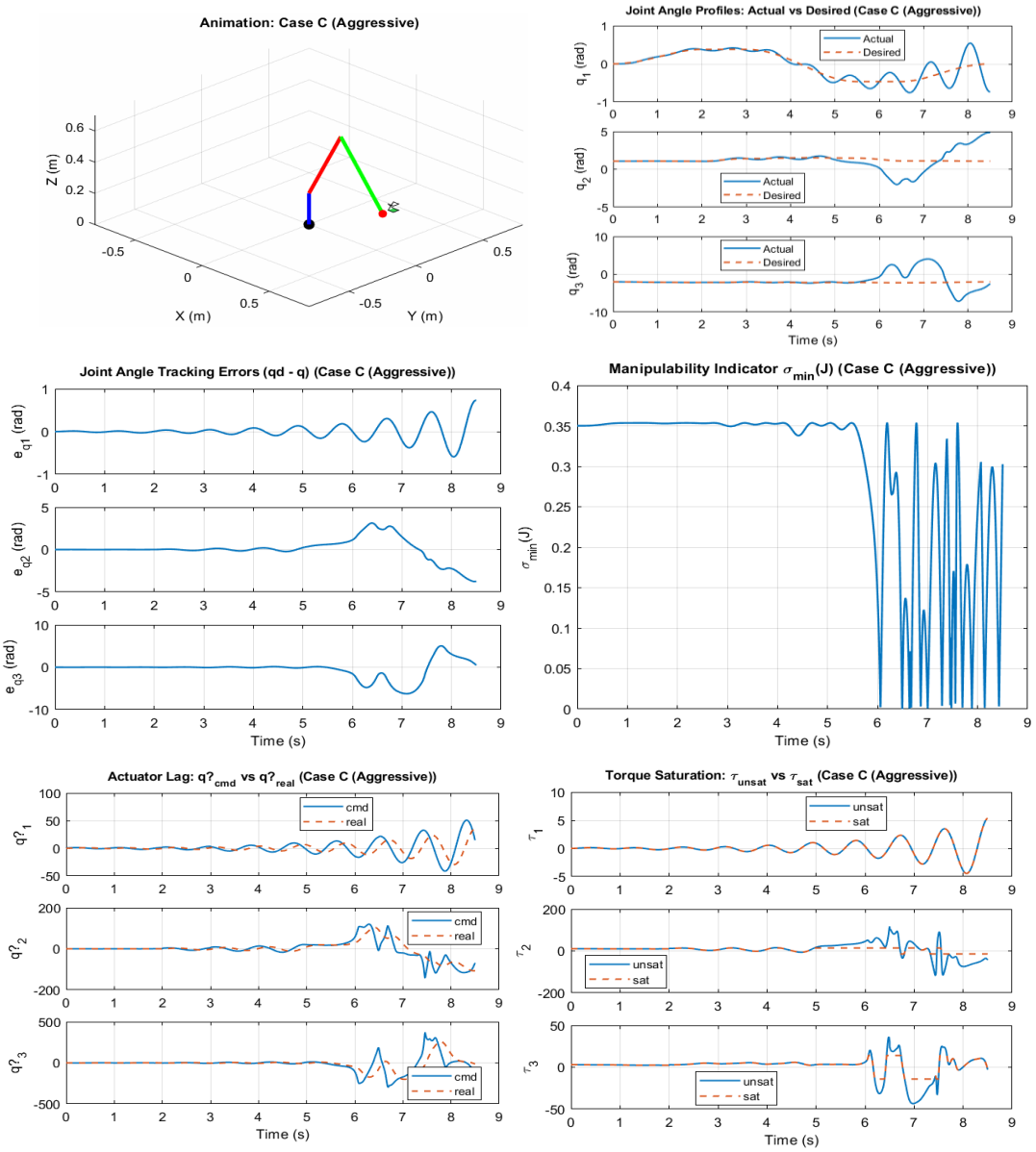


Figure 5—27: Integrated Kinematic, Dynamic, and Actuator-Constrained Performance Analysis under Aggressive LQR Weighting (Case C)

5.2.4.3. Interpretation of LQR Behavior Under Actuator Constraints

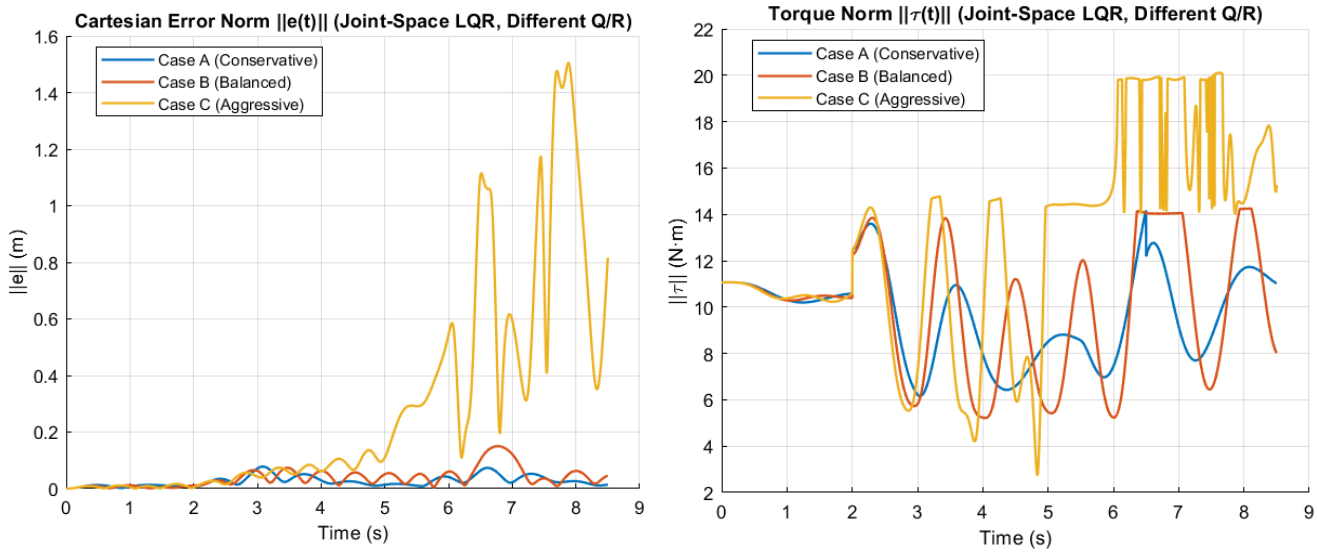


Figure 5—28: Trade-Off Analysis Between Cartesian Tracking Accuracy and Control Effort under Actuator Constraints (Cases A–C)

5.2.4.3.1. Nominal model-based closed-loop behavior

The joint-space LQR controller is designed under the assumption that the commanded joint accelerations can be realized exactly by the actuators.

Let the joint error state be defined as

$$x = \begin{bmatrix} q - q_d \\ \dot{q} - \dot{q}_d \end{bmatrix},$$

and the control law be

$$\ddot{q}_{cmd} = \ddot{q}_d - Kx.$$

The commanded torque is computed using inverse dynamics,

$$\tau_{cmd} = M(q)\ddot{q}_{cmd} + C(q,\dot{q})\dot{q} + G(q).$$

If the actuator can realize this torque exactly and the model perfectly matches the physical robot, substitution into the manipulator dynamics

$$M(q)\ddot{q}_{cmd} + C(q,\dot{q})\dot{q} + G(q) = \tau_{cmd}$$

leads to exact cancellation of nonlinear terms, yielding

$$\ddot{q} = \ddot{q}_{cmd}.$$

Consequently, the closed-loop error dynamics reduce to

$$\dot{x} = (A - BK)x,$$

which guarantees exponential convergence.

Under these nominal assumptions, increasing the gain magnitude generally improves tracking speed and reduces error. [23] [2]

5.2.4.3.2. Influence of torque saturation

In a physical system, actuator torque is bounded. The realized torque therefore satisfies

$$\tau = sat(\tau_{cmd}).$$

This can be written as

$$\tau = \tau_{cmd} + \Delta\tau,$$

where $\Delta\tau$ represents the torque mismatch caused by saturation.

Substituting into the manipulator dynamics yields

$$M(q)\ddot{q} + C\dot{q} + G = \tau_{cmd} + \Delta\tau.$$

After cancellation of nominal terms,

$$M(q)(\ddot{q} - \ddot{q}_{cmd}) = \Delta\tau$$

and therefore

$$\ddot{q} = \ddot{q}_{cmd} + M^{-1}(q)\Delta\tau.$$

Defining the disturbance term

$$d_{sat}(t) = M^{-1}(q)\Delta\tau$$

the actual acceleration becomes

$$\ddot{q} = \ddot{q}_d - Kx + d_{sat}(t).$$

Hence torque saturation introduces a disturbance that directly perturbs the nominal LQR dynamics. [23] [2]

5.2.4.3.3. Influence of actuator bandwidth (second-order lag)

The actuator is modeled as a second-order system relating commanded and realized accelerations,

$$\ddot{q}_{real}(s) = \frac{\omega_a^2}{s^2 + 2\zeta_a\omega_a s + \omega_a^2}$$

In the time domain this implies that the realized acceleration differs from the commanded one. Define the actuator tracking error

$$e_{act} = \ddot{q}_{cmd}(t) - \ddot{q}_{real}(t).$$

Then

$$\ddot{q}_{real}(t) = \ddot{q}_{cmd}(t) - e_{act},$$

which gives

$$\ddot{q}_{real}(t) = \ddot{q}_d - Kx - e_{act}.$$

When both saturation and actuator dynamics are present, the physical acceleration can be approximated as

$$\ddot{q} = \ddot{q}_d - Kx + d_{sat}(t) - e_{act}(t).$$

Thus, the true closed-loop system becomes

$$\dot{x} = (A - BK)x + \omega(t),$$

Where:

$$\omega(t) = \begin{bmatrix} 0 \\ d_{sat}(t) - e_{act}(t) \end{bmatrix}$$

represents the disturbance induced by actuator limitations. [2]

5.2.4.3.4. Dependence of disturbance magnitude on controller aggressiveness

The feedback gains satisfy

$$\|K_{cons}\| < \|K_{bal}\| < \|K_{agg}\|.$$

Since the commanded acceleration is

$$\ddot{q}_{cmd} = \ddot{q}_d - Kx,$$

larger gains produce larger corrective accelerations for the same error state.

The commanded torque therefore increases according to

$$\tau_{cmd} = M(q)(\ddot{q}_d - Kx) + C\dot{q} + G.$$

As the magnitude of τ_{cmd} increases, the probability and severity of torque saturation increase. Whenever saturation occurs, $\Delta\tau$ becomes nonzero, directly increasing the disturbance term $d_{sat}(t)$. Moreover, the aggressive control actions introduce higher frequency components in the commanded acceleration, thus increasing the actuator tracking error $e_{act}(t)$. Consequently, aggressive tuning results in an increase in the magnitude of the disturbances $w(t)$, while conservative tuning maintains the magnitude of the disturbances $w(t)$ low. [2]

5.2.4.3.5. Interpretation of simulation observations

The simulation results are consistent with the disturbed closed-loop model

$$\dot{x} = (A - BK)x + \omega(t).$$

Under conservative control, the commanded accelerations and torques are well within actuator capabilities. Consequently, the disturbance term is small, and the closed-loop system dynamics are close to the nominal LQR solution.

Under balanced control, the control effort is close to actuator saturation. In this case, partial actuator saturation and actuator lag contribute additional disturbances that affect the tracking performance and cause temporary instability or numerical divergence during transients. [2]

Under aggressive control, the commanded torques are often in excess of actuator capabilities. Consequently, the disturbances dominate the closed-loop system dynamics, causing overshoot, deviation from the nominal trajectory, and excursions towards configurations with poor kinematic structure.

5.2.4.3.6. Relation to workspace excursions and singularity behavior

When aggressive commands cannot be realized, delayed response followed by corrective overshoot may drive the manipulator into extended configurations where the Jacobian becomes ill-conditioned. This is reflected in the decrease $\sigma_{min}(J)$ of the smallest singular value. Near such configurations, small Cartesian errors correspond to large joint motions, further amplifying instability and tracking degradation.

5.2.4.3.7. Implications for optimal control design

This analysis demonstrates that optimality in LQR is defined with respect to the assumed model. When actuator limits and dynamics are ignored, the controller minimizing the quadratic cost may demand inputs that are physically infeasible.

Once actuator constraints are introduced, the effective system becomes a disturbed linear system, and performance depends on robustness to disturbances rather than nominal optimality. Under these conditions, conservative tuning can yield superior practical performance by maintaining feasible control actions and limiting disturbance amplification.

5.2.4.3.8. Summary metrics of actuator-constrained LQR

Case	e_{rms}	e_{max}	τ_{rms}	τ_{max}
A	0.0316	0.0779	9.88	14.14
B	0.0505	0.1499	10.38	14.25
C	0.4732	1.5049	13.52	20.10

Table 5-5: Summary Performance Metrics of Actuator-Constrained LQR Control (Cases A–C)

The conservative and balanced tunings remain near actuator limits while preserving acceptable tracking. In contrast, the aggressive tuning generates substantially larger torque demand and tracking error once actuator dynamics are included, explaining its degraded performance and workspace sensitivity. [2] [23]

5.3. Adaptive Task-Space Fuzzy PID Controller

To further improve the robustness and provide adaptability beyond fixed gains, a fuzzy gain-scheduled PID controller is now introduced. The fuzzy controller is based on the same task-space control structure as the other controllers for the sake of comparison under the same conditions of motion and dynamics for all controllers. Instead of replacing the classical structure of the PID controller, the fuzzy controller is based on continuously scheduling its gains according to the instantaneous tracking status. [10]

5.3.1. Definition

A fuzzy PID controller is a sophisticated control method that combines the traditional proportional-integral-derivative (PID) control algorithm with fuzzy logic reasoning in an attempt to enhance

robustness, flexibility, and closed-loop performance in processes that exhibit nonlinear behavior, uncertainty, or an incomplete mathematical model.

The traditional PID controller calculates the control action as a linear combination of the current error, the integral of the error, and the error's rate of change. While the PID control algorithm is popular because of its simplicity and success in linear systems, it may suffer from suboptimal performance when used in complex or time-varying processes, where constant controller gains are not necessarily optimal.

Fuzzy logic control is based on fuzzy set theory and can be used to create a framework in which heuristic knowledge can be represented in terms of linguistic rules rather than mathematical relationships. The fuzzy logic controller can be designed to adjust the controller gains or output using fuzzy reasoning in terms of qualitative assessments of system behavior in terms of linguistic variables like “large,” “small,” etc., thereby allowing the controller to mimic the decision-making ability of the human operator.

The fuzzy system is used in most fuzzy logic control systems to adjust the gains of the PID controller online. The fuzzy system's inputs are usually the control error and derivative of the control error, while the outputs are used to adjust the gains of the PID controller. The adaptive ability of the fuzzy system helps in increasing the controller's responsiveness when there are large errors from the setpoint while maintaining the system's stability and preventing excessive oscillations when the system is in the steady state region. In some fuzzy logic control systems, the fuzzy system's outputs are used directly to generate the control signal, thus replacing the analytical PID formula with a nonlinear rule-based system.

Fuzzy PID controllers have also been effectively used in various engineering disciplines such as robotics, motor drives, process control, and aerospace systems, where improved disturbance rejection characteristics, immunity to modelling errors, and improved transient response are obtained compared to fixed gain PID controllers. [10] [7]

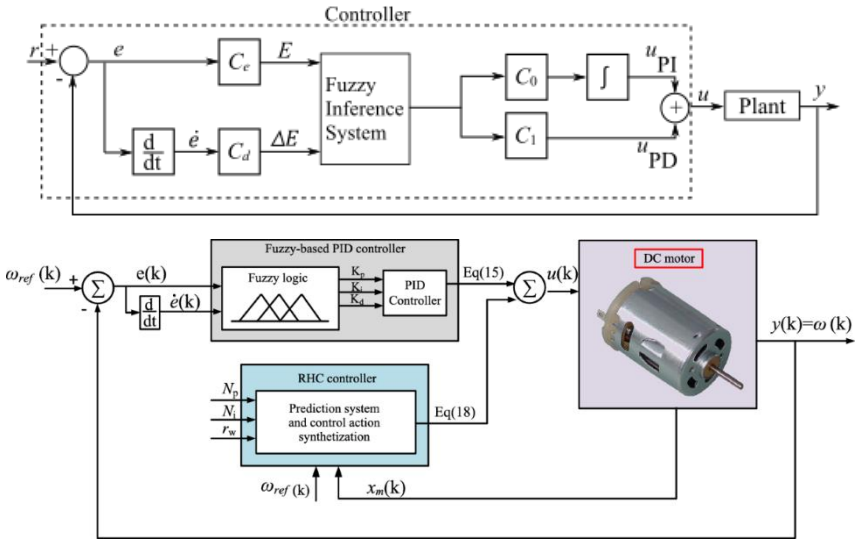


Figure 5—29: Block Diagram of the Adaptive Task-Space Fuzzy PID Control Architecture

5.3.2. Input Normalization

The fuzzy inference mechanism is formulated in terms of normalized inputs in order to ensure consistent behavior within predefined universes of discourse. The normalized tracking error and its derivative are defined as

$$e_n = sat(K_e e, \pm 1), \quad \dot{e}_n = sat(K_{\dot{e}} \dot{e}, \pm 1)$$

where the saturation operator constrains the variables to the interval $[-1,1]$.

The normalization gains K_e and $K_{\dot{e}}$ are selected such that representative tracking errors are mapped into this interval, which coincides with the domain over which the fuzzy membership functions are defined. This normalization guarantees consistent rule activation, prevents numerical saturation, and preserves the interpretability of the linguistic partitions across different motion segments. [7]

5.3.3. Fuzzy Outputs and Gain Scaling

For each Cartesian axis, the fuzzy inference mechanism produces three gain-scaling factors:

$$\alpha_p, \alpha_i, \alpha_d \in [0,1]$$

Rather than replacing the nominal PID structure, the fuzzy controller adaptively scales its gains according to the instantaneous tracking conditions. A set of nominal gain matrices K_{p0}, K_{i0}, and K_{d0} is first selected to ensure baseline stability and satisfactory transient performance. The fuzzy outputs then modulate these gains online, yielding the effective gains

$$K_p^{eff} = diag(\alpha_p) K_{p0}$$

$$K_i^{eff} = diag(\alpha_i) K_{i0}$$

$$K_d^{eff} = diag(\alpha_d) K_{d0}$$

The diagonal operator ensures that each Cartesian axis is scaled independently while preserving the original gain structure. In this manner, the fuzzy mechanism performs adaptive gain scheduling: gains are increased when tracking errors or their rates grow, and reduced near steady state to promote smooth motion and avoid excessive control effort. [10] [24]

5.3.4. Commanded Cartesian Acceleration

The final task-space control law takes the form:

$$\ddot{p}_{cmd} = \ddot{p}_d + K_d^{eff} \dot{e} + K_p^{eff} e + K_i^{eff} e_i$$

where e_i denotes the integral tracking error.

This formulation combines feedforward acceleration with fuzzy-scheduled PID feedback terms, yielding a dynamically consistent Cartesian command that can be directly mapped into joint-space accelerations through the manipulator kinematics.

5.3.5. Membership Functions and Rule Base Design

The following subsections detail the design of the membership functions and the fuzzy rule base underlying the proposed controller.

5.3.5.1. Choice of Inputs and Outputs

The fuzzy scheduler operates on two normalized input variables: the normalized tracking error e_n and its normalized rate $\dot{e}_n$. These signals are sufficient to reproduce the essential behavior of a PID controller in a compact form.

The error e_n reflects the instantaneous deviation from the desired trajectory and therefore determines the required corrective effort, corresponding to the proportional effect. The error rate $\dot{e}_n$ captures the dynamic evolution of the deviation and provides information about the need for damping, which is associated with derivative action.

Integral influence is incorporated indirectly through adaptive scheduling of the integral scaling factor. In particular, the integral gain is emphasized primarily when both e_n and $\dot{e}_n$ are small, allowing the controller to eliminate residual steady-state errors while avoiding integrator windup during large or rapidly changing transients.

The fuzzy system generates three output scaling factors [12]

$$\alpha_p, \alpha_i, \alpha_d \in [0,1],$$

which are applied multiplicatively to the nominal PID gains. This structure corresponds to a classical fuzzy gain-scheduling formulation, widely employed in adaptive PID-like fuzzy controllers due to its simplicity, interpretability, and robustness.

5.3.5.2. Membership Functions (MFs) for Inputs

Each input variable is described by seven linguistic terms

$$\{NB, NM, NS, Z, PS, PM, PB\}$$

defined over the normalized universe of discourse $[-1,1]$. Triangular membership functions are employed due to their structural simplicity, smooth interpolation properties, and suitability for gain-scheduling applications.

The triangular membership function is defined as

$$\mu_{\Delta}(x; a, b, c) = \begin{cases} 0, & x \le a \ \ or \ \ x \ge d, \\ 1, & x = b, \\ \dfrac{x-a}{b-a}, & a < x < b, \\ \dfrac{c-x}{c-b}, & b < x < c, \end{cases}$$

The membership functions are centered at uniformly distributed points

$$C_k \in \left\{-1, -\tfrac{2}{3}, -\tfrac{1}{3}, 0, \tfrac{1}{3}, \tfrac{2}{3}, 1\right\},$$

with deliberate overlap between adjacent sets. Such overlap ensures that multiple rules are activated in transition regions, yielding continuous gain adaptation and preventing abrupt variations in the control response. [24]

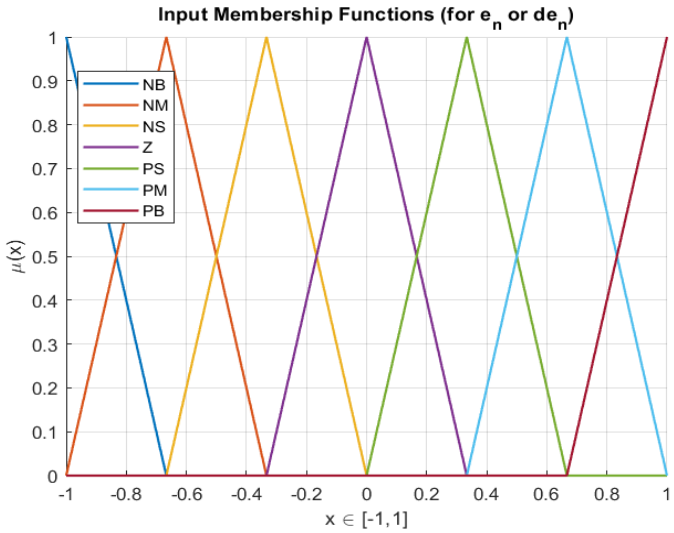


Figure 5—30: intput membership functions used for the fuzzy gain-scaling coefficients α

5.3.5.3. Output Membership Functions

Each output scaling factor $\alpha \in [0,1]$ is described using five linguistic levels

$$\{VS, S, M, L, VL\},$$

corresponding to Very Small through Very Large gain adjustments. Triangular membership functions are employed, with centers located at

$$\{0.10, 0.30, 0.50, 0.70, 0.90\}.$$

This configuration provides a compact yet expressive representation of the gain-scaling space, enabling meaningful adaptation of controller gains while avoiding unnecessary structural complexity in the fuzzy inference mechanism.

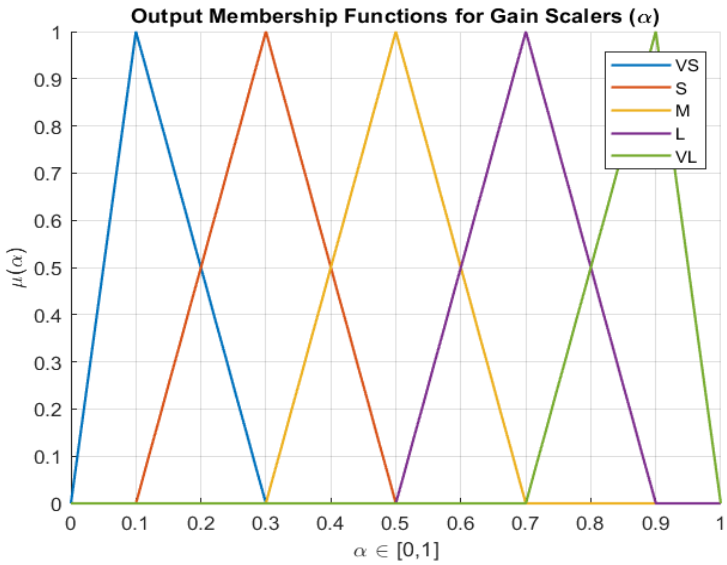


Figure 5—31: output membership functions used for the fuzzy gain-scaling coefficients α

5.3.5.4. Rule Design Philosophy

The fuzzy rule base is constructed to reflect well-established PID control heuristics, allowing the adaptive gains to respond to the instantaneous tracking conditions in a physically interpretable manner.

The proportional scaling factor is designed to increase with the magnitude of the tracking error. Large deviations from the reference require stronger corrective action, while moderate increases may also occur when the error evolves rapidly, indicating the need for a more assertive response.

The derivative scaling factor is governed primarily by the magnitude of the error rate. When the error changes quickly, stronger derivative action is introduced to provide damping, thereby reducing the likelihood of overshoot and oscillatory behavior.

The integral scaling factor is emphasized only when both the error and its rate are small. In this regime, integral action is effective for eliminating residual steady-state offsets. Conversely, when the error is large or rapidly varying, the integral contribution is reduced to prevent integrator windup and preserve closed-loop stability. [24]

Together, these design principles ensure that the fuzzy controller reproduces the qualitative behavior of a well-tuned PID regulator while introducing adaptive, state-dependent gain modulation.

5.3.5.5. Rule Implementation

The fuzzy rule base is implemented through a structured index-based mapping that measures the distance of each linguistic label from the central region Z. Let i_e and $i_{\dot{e}}$ denote the indices of the active linguistic terms for the error and its derivative, respectively, with indices $1 \cdots 7$ corresponding to $\{NB, \cdots, PB\}$ and index 4 representing the zero region. The deviation measures are defined as

$$a_e = |i_e - 4|, \quad a_{\dot{e}} = |i_{\dot{e}} - 4|$$

The proportional scaling level increases monotonically with a_e, progressing from small to very large values as the error magnitude grows. Additional strengthening may occur when the derivative deviation $a_{\dot{e}}$ exceeds a moderate threshold, reflecting the need for stronger corrective action during rapidly evolving errors.

The derivative scaling level is determined primarily by $a_{\dot{e}}$, following a similar monotonic relationship, so that faster error variations produce stronger damping.

The integral scaling level is maximal when both a_e and $a_{\dot{e}}$ correspond to the zero region, decreases when deviations are moderate, and becomes minimal otherwise. This ensures that integral action is concentrated near steady state and suppressed during large transients. [24]

This structured mapping yields smooth, continuous gain scheduling and reproduces the qualitative behavior of a well-tuned PID controller while preserving the interpretability of the fuzzy rule base.

5.3.5.6. Mamdani Inference and Defuzzification

The inference mechanism follows a Mamdani min–max structure, which provides an intuitive and interpretable mapping between linguistic rules and numerical outputs. For each pair of input linguistic terms (i, j), corresponding to the normalized error and its derivative, the rule firing strength is computed as

$$w_{ij} = \min\left(\mu_{e,i}(e_n), \mu_{\dot{e},j}(\dot{e}_n)\right)$$

This operation represents the fuzzy logical conjunction between the two inputs and determines the degree to which each rule contributes to the final output.

For each activated rule, the membership function associated with the corresponding output linguistic level is truncated at the firing strength w_{ij}. This clipping process ensures that rules with higher activation exert stronger influence on the final control action, while weakly activated rules contribute only marginally.

The contributions of all rules are then combined using the maximum operator to obtain the aggregated fuzzy output set:

$$\mu_{agg}(u) = max_{i,j}\left(\min\left(w_{ij}, \mu_{out,\kappa(i,j)}\right)(u)\right).$$

This aggregation step effectively constructs a continuous fuzzy surface that blends the effects of all activated rules. As a result, the controller output varies smoothly across the input space, avoiding abrupt changes in the scheduled gains.

To obtain a numerical gain-scaling factor, centroid defuzzification is applied:

$$\alpha = \frac{\int_0^1 u\mu_{agg}du}{\int_0^1 \mu_{agg}du}$$

In the implementation, the integrals are evaluated numerically over a dense discretization of the interval [0,1]. This numerical centroid computation ensures stable and continuous output values while keeping the implementation simple and toolbox-independent.

Overall, the Mamdani inference combined with centroid defuzzification provides a smooth and bounded mapping from tracking conditions to gain-scaling factors, ensuring continuous controller adaptation and robust closed-loop behavior throughout the motion. [24]

5.3.5.7. Interpretation of the Fuzzy Gain Surfaces

The resulting gain surfaces obtained from the fuzzy inference process are illustrated in Figures below:

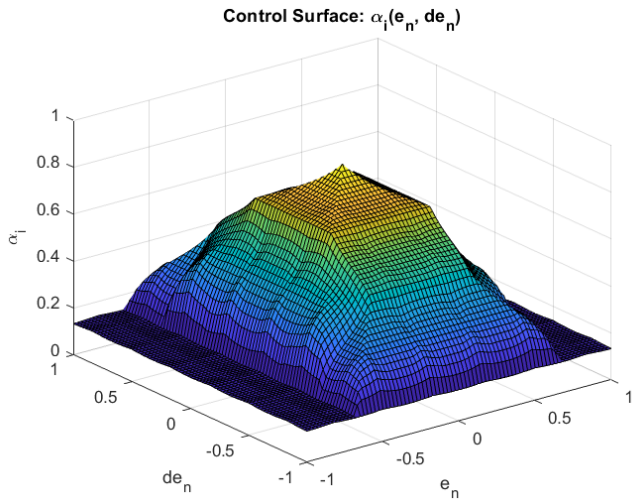


Figure 5—34: Fuzzy control surface for the integral gain-scaling coefficient α_i

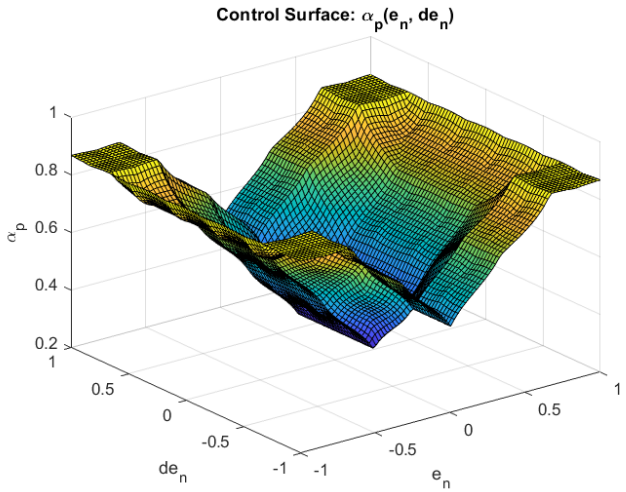


Figure 5—33: Fuzzy control surface for the proportional gain-scaling coefficient α_p

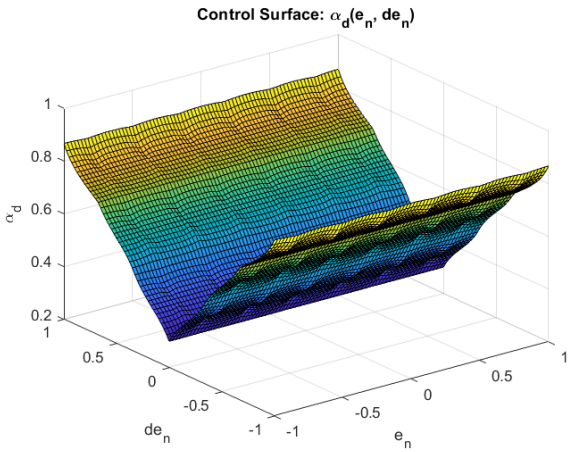


Figure 5—32: Fuzzy control surface for the derivative gain-scaling coefficient α_d

These surfaces for the scaling factors show the adaptive properties of the suggested fuzzy controller, providing a better understanding of its control actions. The proportional control surface increases with the magnitude of the tracking error, reflecting the fact that the controller will take stronger control actions as the magnitude of the tracking error increases compared to the desired path. This is in keeping with classical proportional control principles and will ensure rapid convergence to the reference path.

The derivative gain surface is primarily influenced by the rate of change of the tracking error. High derivative scaling values appear in regions where the error varies rapidly, reflecting the controller's

effort to introduce additional damping and suppress oscillatory motion. Conversely, the derivative action is reduced near steady state, where excessive damping could slow convergence.

The integral gain surface exhibits a localized peak around the equilibrium region, where both the tracking error and its derivative are small. This confirms that the integral contribution is concentrated near steady state to eliminate residual offsets, while being automatically suppressed during large or fast transients to prevent windup and instability. [24] [25]

Overall, these gain surfaces demonstrate that the fuzzy scheduler reproduces the qualitative behavior of a well-tuned PID controller while introducing smooth nonlinear adaptation across the state space. The resulting control action remains continuous, interpretable, and robust to variations in the tracking conditions encountered during the pick-and-place task.

5.3.6. Simulation Results and Discussion

5.3.6.1. Nominal Case

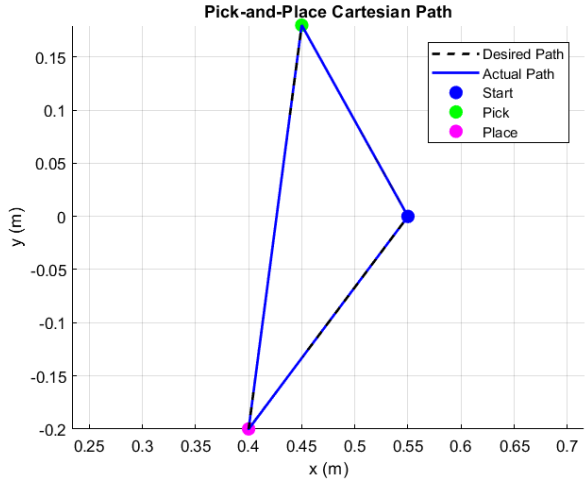


Figure 5—35: Desired and actual Cartesian trajectories of the end-effector during the pick-and-place task

The end-effector closely tracks the specified pick-and-place path during the entire process. This close similarity between the specified and actual paths indicates that the operational space controller is successfully implementing the Cartesian objectives. Any observed discrepancies are primarily confined to transitions in the reference path, which is consistent with the change in reference acceleration profiles from one segment to the next in the series of cubic reference trajectory segments.

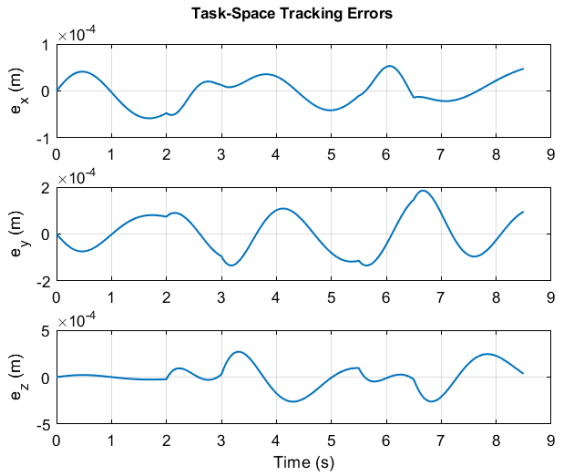


Figure 5—36: Cartesian tracking errors along the three task-space axes

The Cartesian tracking errors remain on the order of $\mathbf{10^{-4}}$m, corresponding to sub-millimeter accuracy. The performance metrics are

- $\|e\|_{RMS} = 1.566 \times 10^{-4}$ m
- $\|e\|_{MAX} = 3.087 \times 10^{-4}$ m

Axis-wise RMS errors are

- $e_x = 3.22 \times 10^{-5}$ m,
- $e_y = 8.344 \times 10^{-5}$ m,
- $e_z = 1.290 \times 10^{-4}$ m

The slightly larger vertical error is consistent with gravity compensation and payload transport requirements.

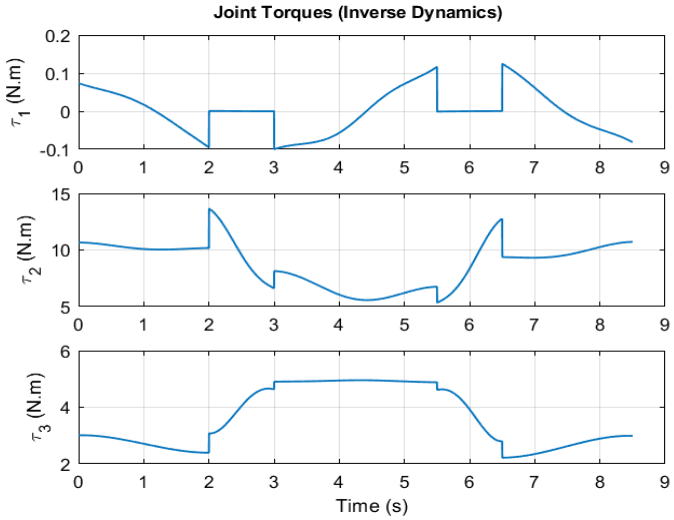


Figure 5—37: Joint torques obtained from inverse dynamics, showing the effect of payload transport

The torque profiles display physical consistency. The first joint torque is relatively low, showing that this joint is not very sensitive to vertical loads. The second and third joints are prominent due to gravity compensation and payload handling. The effect of the payload is clearly visible in the joint torques, especially in the last joint during the manipulation.

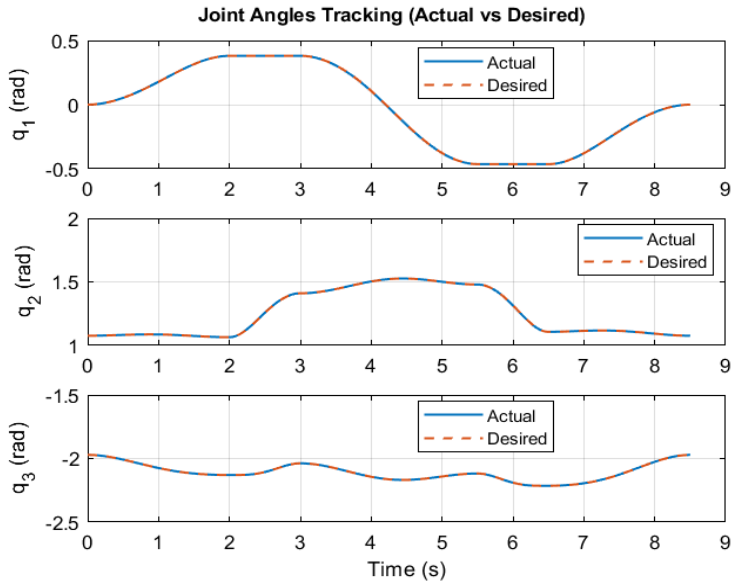


Figure 5—38: Desired and actual joint trajectories during the task execution

The actual joint profiles are very similar to the desired joint profiles obtained from inverse kinematics. Some minor differences are expected because the controller controls Cartesian motion instead of joint motion directly. These differences are due to the dynamic consistency of the operational space controller.

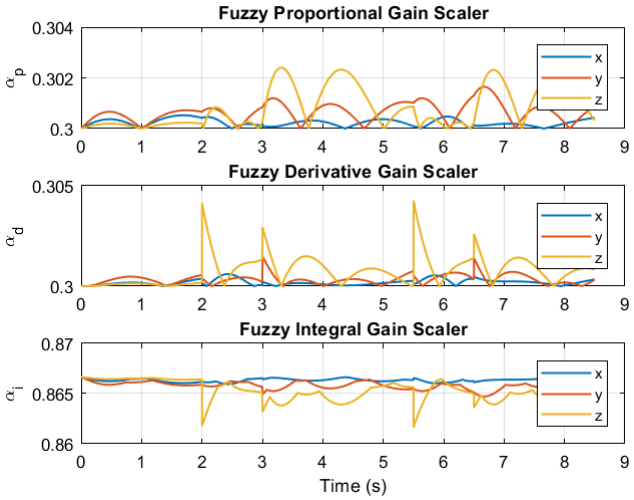


Figure 5—39: Time evolution of the fuzzy gain-scaling coefficients for the three Cartesian axes during the manipulation task.

The above figure illustrates the time behavior of the fuzzy gains for the scaling values during the manipulation process. As shown, the proportional gain experiences moderate changes due to variations in the tracking errors, while the derivative gain increases during transitions in the trajectory, reflecting its contribution to the damping of the errors' rate changes. Finally, the integral gain is kept at a relatively high level, ensuring zero steady-state errors are achieved. This plot confirms the smooth adaptation of the fuzzy controller gains in accordance with the changing motion conditions. [24] [2]

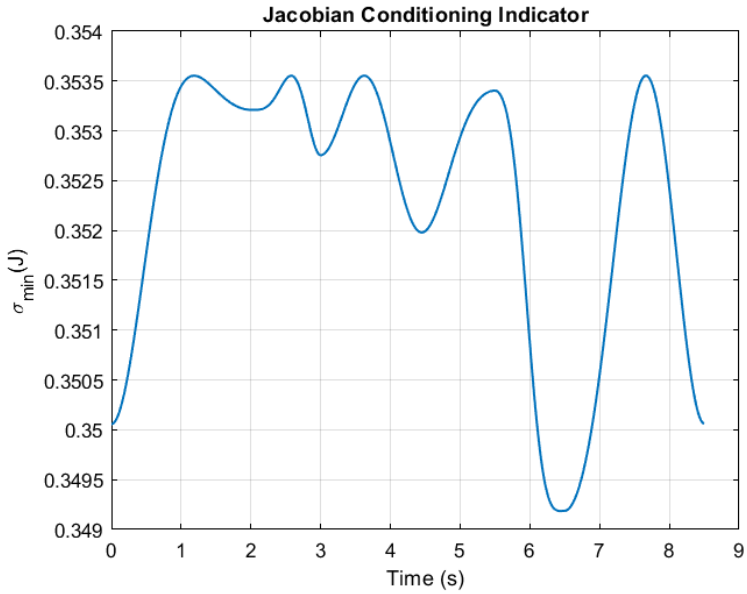


Figure 5—40: Jacobian Conditioning (Minimum Singular Value) during Nominal Closed-Loop Manipulation under Fuzzy-PID Control

The minimal singular value of the Jacobian matrix is kept at a level close to 0.35 during the entire motion with minor fluctuations. This confirms the manipulator is far from singular configurations, which explains the absence of numerical jumps in the DLS-based inverse kinematics solution.

5.3.6.2. Effect of Actuator Lag and Saturation on the Fuzzy-PID Controller

The fuzzy-PID controller computes a commanded Cartesian acceleration of the form

$$\ddot{p}_{cmd} = \ddot{p}_d + \alpha_d K_d \dot{e} + \alpha_p K_p e + \alpha_i K_i \int e\, dt$$

where the gains are scheduled online by the fuzzy inference system. Under ideal conditions, this command is mapped to joint accelerations using the DLS inverse:

$$\ddot{q}_{cmd} = (J^T J + \lambda^2 I)^{-1} J^T (\ddot{p}_{cmd} - \dot{J}(q, \dot{q})\dot{q})$$

If actuators were ideal, the closed-loop dynamics would follow this commanded acceleration. However, once actuator limits are introduced, the realized motion differs from the nominal one. [20]

5.3.6.2.1. Effect of Acceleration/Torque Saturation

When the commanded acceleration exceeds actuator capability, it is clipped:

$$\ddot{q}_{sat} = sat(\ddot{q}_{cmd})$$

This can be rewritten as

$$\ddot{q}_{sat} = \ddot{q}_{cmd} + d_{sat}, \quad d_{sat} = \ddot{q}_{sat} - \ddot{q}_{cmd}$$

where $\boldsymbol{d_{sat}}$ represents the disturbance introduced by saturation.

For the fuzzy-PID controller, large tracking errors typically activate higher fuzzy gains, which increases the magnitude of $\ddot{\boldsymbol{p}}_{\boldsymbol{cmd}}$ and therefore $\ddot{\boldsymbol{q}}_{\boldsymbol{cmd}}$.

When saturation occurs, the corrective action is weakened, leading to:

- slower reduction of the tracking error,
- delayed convergence to the desired trajectory,
- and reduced ability to immediately compensate disturbances.

Thus, saturation effectively modifies the intended closed-loop behavior by injecting a disturbance term into the dynamics. [18] [10]

5.3.6.2.2. Effect of Actuator Lag

Actuator dynamics are modeled as a second-order system:

$$\ddot{q}_{real}(s) = \frac{\omega_a^2}{s^2 + 2\zeta_a\omega_a s + \omega_a^2}\ddot{q}_{cmd}(s)$$

Defining the actuator tracking error

$$e_{act} = \ddot{q}_{sat}(t) - \ddot{q}_{real}(t)$$

the realized acceleration becomes

$$\ddot{q}_{real}(t) = \ddot{q}_{sat}(t) - e_{act}$$

The actuator therefore behaves as a low-pass filter on the commanded acceleration. Rapid corrections generated by the fuzzy controller are smoothed and delayed, which results in:

- slower initial response,
- reduced high-frequency control effort,
- and a time delay between the computed and realized motion.

From a control viewpoint, the actuator lag introduces a dynamic mismatch that perturbs the nominal fuzzy-PID behavior. [12]

5.3.6.2.3. Combined Effect on the Closed-Loop Behavior

With both saturation and lag present, the realized joint acceleration can be written as

$$\ddot{q}_{real} = \ddot{q}_{cmd} + d_{sat} - e_{act}$$

showing that the plant no longer follows the designed control law exactly.

For the fuzzy-PID controller, this results in:

- slower transient motion because accelerations cannot be applied immediately,
- smoother torque profiles due to actuator filtering,
- and slightly larger transient tracking errors since corrections are delayed.

Despite these effects, the adaptive nature of the fuzzy gains helps maintain stable convergence by reducing control effort as the error decreases. [14] [20]

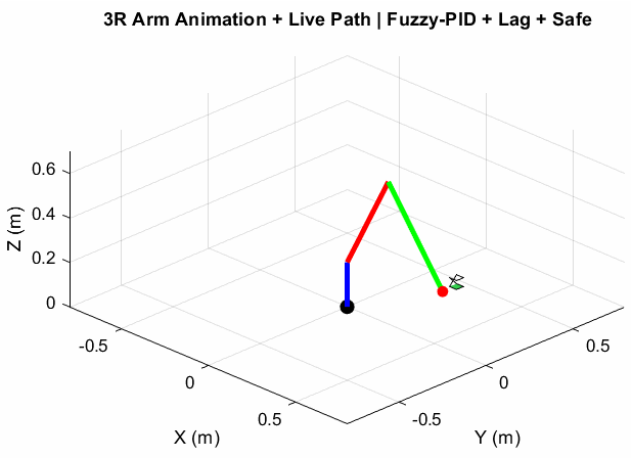


Figure 5—41: Closed-Loop 3R Manipulator Motion and Live Cartesian Path under Fuzzy-PID Control with Actuator Lag and Safety Constraints

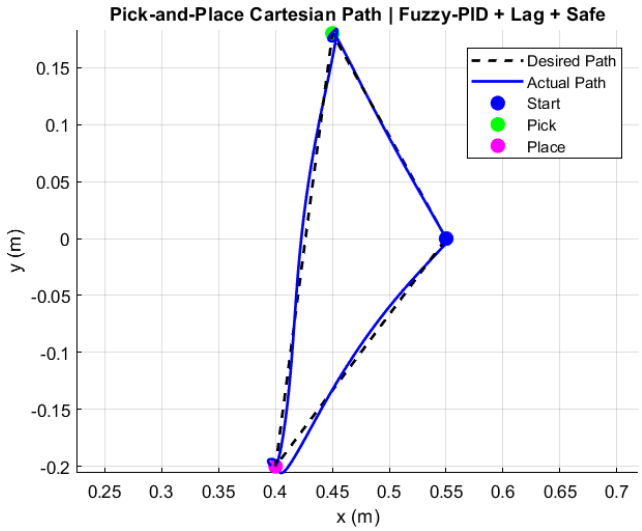


Figure 5—42: Cartesian Pick-and-Place Trajectory Tracking under Fuzzy-PID Control with Actuator Lag

The fuzzy PID controller was able to track the desired path for the pick and place operation even in the presence of actuator lag and safety constraints. From the Cartesian path, it is seen that there is a good match between the desired and actual paths, except for a few deviations at the transition points where the direction of movement is suddenly changed. This is expected due to the limitations in the actuator's ability to provide instantaneous acceleration commands.

From the above, it can be seen that the geometric path is maintained, thereby ensuring that the desired path is correct even in the presence of constraints.

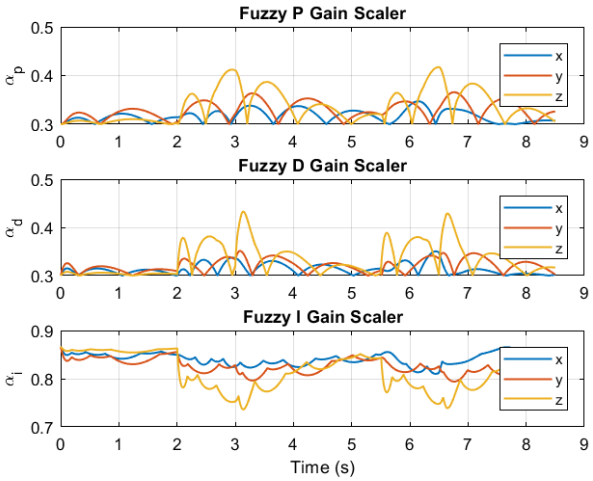


Figure 5—43: Adaptive Gain Scheduling (P, D, I) in Fuzzy-PID Control under Actuator Constraints

This is confirmed by the fuzzy gain scalers, where it is seen that the controller's gains are varied during the process. The proportional and derivative gains are increased at certain points in order to compensate for the increased rate of error, whereas the integral gain is kept sufficiently large to eliminate any steady-state biases. This is seen from the above graph, where it is confirmed that the robustness of the fuzzy controller is due to the adaptation of the controller's gains during the process.

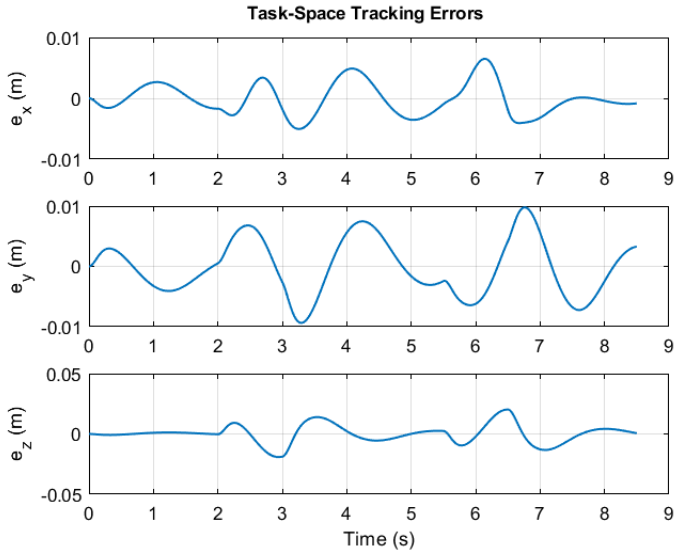


Figure 5—44: Task-Space Tracking Errors under Fuzzy-PID Control with Actuator Lag and Saturation

From the above graph, it is seen that the tracking errors are kept within a minimum range throughout the process. The transient errors are seen to be large at certain points, but this is due to the change in direction, which is necessary for a smooth and stable operation. This is confirmed by the above graph, where it is seen that the steady-state errors are kept within a very minimum range, thus confirming that the integral part of the fuzzy PID controller is able to eliminate any steady-state biases.

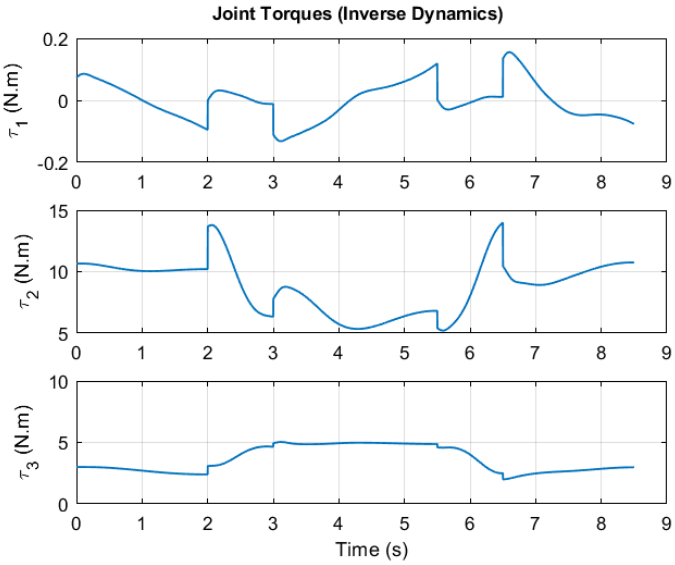


Figure 5—45: Joint Torque Profiles under Fuzzy-PID Control with Inverse Dynamics and Actuator Constraints

It can also be noticed that the torque profiles are smooth and continuous, with no high-frequency oscillations in the profiles. This shows that the lag in the actuator effectively filters the aggressive commands, so that there are no sudden spikes in the required torque. This is because the fuzzy controller adjusts the gains during motion, keeping the control within reasonable limits.

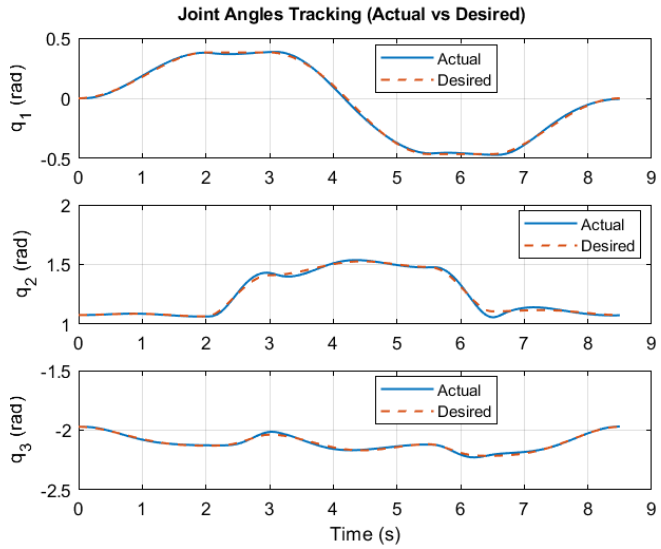


Figure 5—46: Joint Angle Tracking Response (Actual vs Desired) under Fuzzy-PID Control with Actuator Constraints

It can be seen that the joint trajectories track their references closely, except for a small amount of phase delay. This delay is expected from the actuator and is not cumulative, thus demonstrating stability.

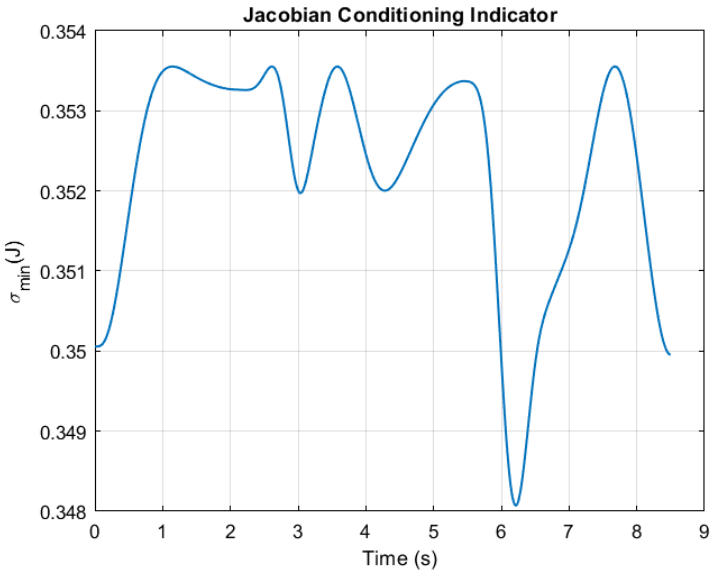


Figure 5—47: Jacobian Conditioning Indicator under Fuzzy-PID Control with Actuator Constraints

The Jacobian conditioning indicator stays well above zero during the whole trajectory. This confirms that the manipulator stays away from singular positions and that the damped least squares inverse mapping behaves numerically correctly. Moreover, the absence of sudden spikes in the torques and joint positions confirms the robustness of the kinematic inversion. [24] [2]

Quantitative Performance Evaluation

These quantitative values are in agreement with the qualitative findings discussed in the previous paragraphs. The Cartesian tracking error remains in the range of centimeters; the values of the root mean square are in the range of a few millimeters, while the maximum deviation remains at two centimeters. This proves the accuracy of the tracking function in spite of the actuator limitations. The steady-state error remains at a negligible value, which proves the accuracy of the integral function. The torque statistics provide additional insight into the control effort distribution. The second joint carries the largest portion of the control effort, which is expected since it supports the primary gravitational load of the manipulator. In contrast, the distal joints require lower nominal torque but exhibit greater sensitivity to payload variations. This explains the observed redistribution of torques when the load is attached. Overall, the controller achieves accurate tracking, smooth and physically realizable control effort, and stable kinematic behavior, even under realistic actuator constraints [2].

Quantitative Tracking and Torque Metrics

The numerical values summarized in Table confirm the qualitative observations discussed above

Axis	e_{rms}(m)	e_{max}(m)	e_{ss}(m)	$e_{rms,payload}$(m)	$e_{max,payload}$(m)
X	0.00264	0.00649	0.00052	0.00319	0.00649
Y	0.00464	0.00978	0.00324	0.00492	0.00937
Z	0.00763	0.02023	0.00285	0.00902	0.02023

Table 5-6: Quantitative Cartesian Tracking Performance with Actuator Lag, Saturation, and Payload Effects

Overall Interpretation

The results demonstrate that the fuzzy-PID controller maintains stable and accurate trajectory tracking even when actuator lag, saturation, and joint limits are included.

The main observable effects of these physical constraints are:

- slightly slower transient response,
- small error peaks at trajectory transitions,
- smoother torque profiles,
- and negligible steady-state bias.

These behaviors confirm that the controller remains robust and physically realizable, while the adaptive gain scheduling allows it to adjust its response to changing motion conditions and payload effects.

5.4. Comparative Evaluation of the Control Strategies

This section will discuss a comparative study among the three control strategies discussed in this paper, namely, task space PID control, Linear Quadratic Regulator control, and fuzzy logic control. Since all control strategies are based on the same robot model, reference trajectory, actuator model, and payload assumptions, the differences in performance are due to the nature of each control strategy.

5.4.1. Tracking Accuracy

The LQR controller has the highest nominal accuracy in tracking the reference signal, as the feedback gains are determined by minimizing a quadratic cost function where the states' deviations are weighted by the cost function. The PID controller has similar accuracy in tracking the reference signal when tuned correctly, mainly because of the integral action of the controller, which can reject constant disturbances and modeling errors in the system. The fuzzy controller has slightly larger steady-state errors in tracking the reference signal because of its prioritization of smooth response and stability.

From the results obtained in this problem, it is evident that optimal control has the highest accuracy in tracking the reference signal, while classical feedback has similar accuracy when tuned correctly. [20] [19] [6]

5.4.2. Control Effort and Actuator Usage

It should be noted that the LQR methodology naturally considers the control effort through the weighting matrix applied to the input in the cost function. This ensures that the actuator commands are always smooth. While it is true that the PID controller will maintain a moderate amount of torque when tuned conservatively, it should also be noted that overly aggressive tuning may cause problems in terms of actuator demands. Finally, it should be noted that the fuzzy controller will naturally produce a smooth torque profile due to the gradual transition from one rule activation to another; however, the corrective actions will be larger in areas in which the rule base is less certain. These observations have confirmed our intuition regarding optimal feedback. [4] [10]

5.4.3. Robustness to Modeling Errors and Actuator Constraints

With the actuator bandwidth constraints and torque saturation considered in the simulations, the PID controller shows robustness. The simplicity of the PID controller and its integral term make it possible to reject the persistent disturbance and keep the tracking error bounded. The fuzzy controller is also robust, as it does not require a model of the system dynamics. The LQR controller becomes more sensitive to modeling errors and actuator constraints, as its optimality requires the correctness of the linear model and the feasibility of the control inputs.

This observation emphasizes the well-known difference between optimal control and feedback control methods, which have implicit robustness. [20] [19] [6]

5.4.4. Transient Response Characteristics

This is because the PID controller makes it possible to shape the transient response in a straightforward fashion through the use of damping ratio and natural frequency-related parameters. The LQR controller makes it possible to obtain well-damped responses through optimization rather than pole placement, which results in smooth responses but makes it difficult to understand the optimization process. Finally, the fuzzy controller makes it possible to obtain responses with slow but continuous transitions due to the gradual nature of the control rules. [23]

5.4.5. Implementation Complexity

From the implementation viewpoint, the PID controller is still the easiest controller with respect to gain tuning and computational burden. The fuzzy controller adds complexity in designing the membership functions and rule base. However, from the viewpoint of real-time computations, the

complexity of the fuzzy controller is still moderate. The LQR controller adds complexity in linearization, solution of the Riccati equation, and matrix computations. Therefore, it is the most complex controller.

5.4.6. Overall Assessment

All three controllers embody a particular design philosophy. A PID controller offers a basic and intuitive solution with a high level of robustness and simplicity in implementation. An LQR controller offers a theoretically optimal solution with a high degree of precision and efficiency in actuator usage, assuming a precise model of the physical system. A fuzzy controller offers flexibility and adaptability in cases where modeling uncertainties are large or qualitative control knowledge is available. In the particular problem of pick-and-place control, the PID controller offers the best compromise in terms of robustness, precision, and implement ability, while the LQR controller offers the best performance in nominal cases under ideal assumptions, and the fuzzy controller offers the best flexibility in adaptability to uncertainties.

5.4.7. Summary Table

Criterion	**PID**	**LQR**	**Fuzzy**
Nominal tracking accuracy	High	Very high	Moderate
Control effort efficiency	Moderate	High	Moderate
Robustness to uncertainties	High	Moderate	High
Interpretability of tuning	High	Moderate	Low
Implementation complexity	Low	High	Medium

Table 5-7: Comparison of Control Strategies: PID, LQR, and Fuzzy Controllers

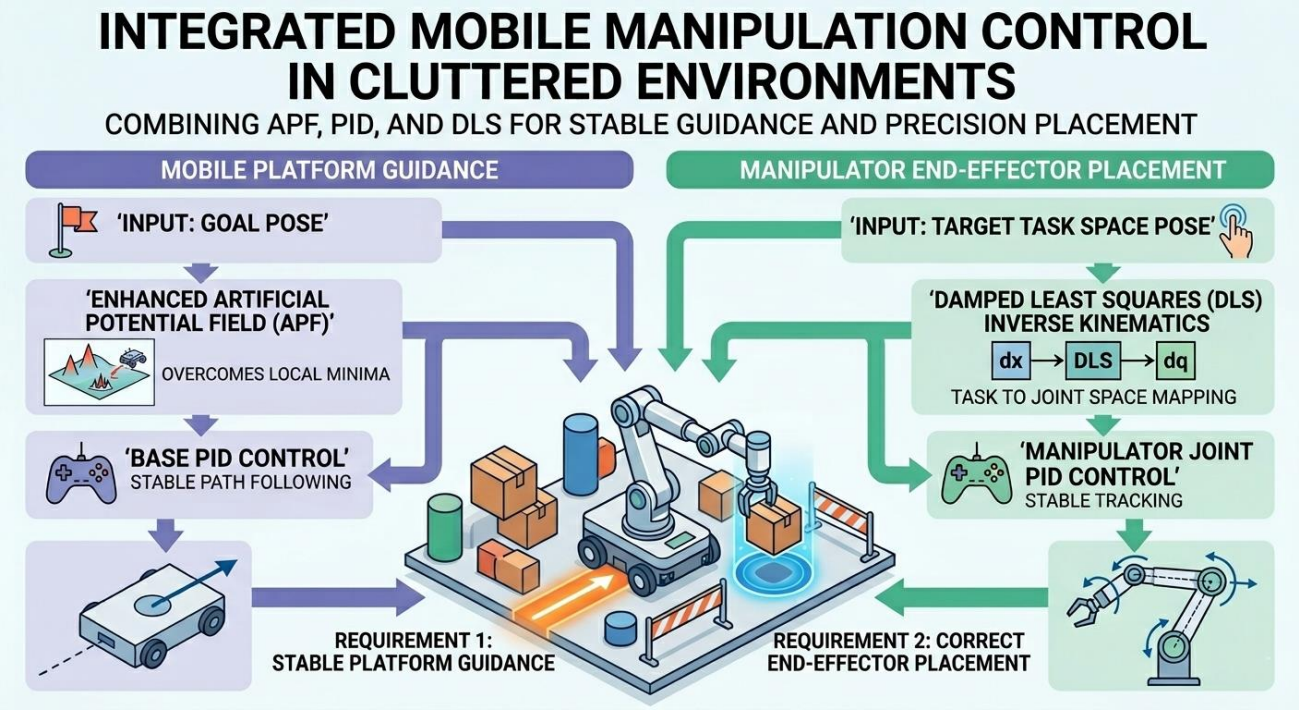
The comparison confirms that controller selection should be guided not by nominal performance alone, but by the intended operational conditions, modeling confidence, and implementation constraints.

5.5. Robust Mobile Manipulation for Pick-and-Place in Cluttered Environments Using PID Tracking, Enhanced Artificial Potential Fields, and IK-Driven Joint PID Control

The simultaneous satisfaction of two requirements in mobile manipulation in a cluttered environment has to be ensured. First, the mobile platform should be stably guided to the goal. Second, the end effector should be correctly placed. PID control methods are still popular due to their simplicity, interpretability, and robustness in the presence of moderate modeling uncertainties. A new safety level must be added in the presence of obstacles. APF methods are popular due to their efficiency in solving the local obstacle avoidance problem. However, APF methods are also criticized due to the presence of local minima. In this paper, an enhanced APF approach will be used. PID control will be used at the base level. For the manipulator, the task space will be mapped to the joint space using the DLS inverse kinematics method, and PID will be used to achieve stable tracking.

5.5.1. Overview

This work presents a simulation-based mobile manipulation framework consisting of an omni-directional mobile base and a 3-DoF yaw–pitch–pitch (YPP) manipulator performing a pick-and-place task in an obstacle-populated workspace. The base motion is governed by independent PID controllers for planar position and yaw tracking, while collision avoidance is achieved through an enhanced Artificial Potential Field (APF) that integrates obstacle inflation, tangential swirl components, velocity scaling near obstacles, and a collision-shield mechanism. Manipulator motion is handled by a hierarchical scheme: a damped least-squares inverse kinematics (DLS-IK) solver generates joint references from end-effector targets, followed by joint-space PID control with saturation and anti-windup to ensure stable tracking. A finite-state machine coordinates base navigation and manipulation phases. A simulated 2D LiDAR and occupancy-grid mapping are used to model perception and provide interpretability of the navigation behavior. Results demonstrate reliable phase progression, smooth obstacle avoidance, and accurate end-effector tracking during approach, grasp, transport, and placement. [26]



5.5.2. System Model and Task Description

5.5.2.1. Environment and Obstacles

The workspace is bounded as $x \in [-0.2,3.4]$ m, $y \in [-0.2,2.6]$ m, $z \in [0,1]$ m. Obstacles are modeled as circular regions in the ground plane with center $c_i = [x_i, y_i]^T$ and radius r_i, visualized as short cylinders for 3D animation.

5.5.2.2. Pick-and-Place Setup

The object is defined by pick location $p_{pick} = [0.85,0.55]^T$ and place location $p_{place} = [2.85,2.05]^T$ m, with table height $z_{table} = 0.08$ m. The base navigates to stand-off goals:

$$p_{pick,b} = p_{pick} + [-0.28, -0.18]^T, \;\; p_{place,b} = p_{place} + [-0.28, -0.18]^T$$

The manipulator executes approach and descends trajectories using predefined heights:

$$z_{approach} = z_{table} + 0.22, \quad z_{pick} = z_{place} = z_{table} + 0.55 \cdot size$$

5.5.3. Mobile Base Control

Task execution is coordinated through a sequential supervisory controller that alternates between navigation and manipulation phases. The base first moves to the pick location, the manipulator performs approach, grasp, and lift actions, and the system then transitions to transport and placement before terminating. Separating these stages reduces interference between base motion and end-effector positioning, while temporary base slowdown during manipulation improves grasping and placement stability. [23]

5.5.3.1. PID Tracking

The base follows reference position (x_d, y_d) and yaw θ_d using three independent PID controllers:

$$u_x = K_{p,x}e_x + K_{i,x}\int e_x\, dt + K_{d,x}\dot{e_x}, \quad e_x = x_d - x$$

$$u_y = K_{p,y}e_y + K_{i,y}\int e_y\, dt + K_{d,y}\dot{e_y}, \quad e_y = y_d - y$$

$$u_\theta = K_{p,\theta}e_\theta + K_{i,\theta}\int e_z\, dt + K_{d,\theta}\dot{e}_z, \quad e_\theta = wrap(\theta_d - \theta)$$

Velocity commands are saturated by v_{max} and ω_{max}. [20]

5.5.3.2. Enhanced Artificial Potential Field (APF)

The base also receives an avoidance velocity v_{apf} computed from an APF. The total planar command is:

$$v = v_{pid} + v_{apf}$$

Attractive term:

$$F_{att} = K_{att}(p_g - p),$$

where p is the current base position and p_g is the goal.

Obstacle inflation improves safety by expanding obstacle radii:

$$r_i^{eff} = r_i + \delta_{infl} + \delta_{safe}.$$

Define clearance:

$$d_i = \|p - c_i\| - r_i^{eff}.$$

Repulsive term is activated for $d_i < d_o$ (influence distance):

$$F_{rep,i} = \alpha(d_i)\hat{n}_i$$

$$\hat{n}_i = \frac{p - c_i}{\|p - c_i\|}$$

$$\alpha(d_i) = K_{rep}\left(\frac{1}{d_i} - \frac{1}{d_o}\right)\frac{1}{d_i^2}$$

Tangential (swirl) term reduces local minima by adding a perpendicular component:

$$\hat{t}_i = \left[-\hat{n}_{i,y}, \hat{n}_{i,x}\right]^T$$

$$F_{tan,i} = K_{swirl}\alpha(d_i)s_i\hat{t}_i$$

where $s_i \in \{-1,1\}$ selects the tangential direction based on goal alignment.

Safety behaviors:

- Hard-stop push-away if d_i is very small.
- Speed scaling if the minimum clearance d_{min} drops below a threshold.
- Collision-shield integration projects velocity to prevent penetration into inflated obstacles.

Parameter values used in simulation: [26] [25]

$K_{att} = 2.2$, $K_{rep} = 1.4$, $d_0 = 0.85$, Inflation = 0.10, Safe margin = 0.06, $K_{swirl} = 0.55$, $v_{apf,max} = 0.85$, hard stop = 0.05, $slow_d = 0.35$

5.5.4. Manipulator Control

5.5.4.1. Reference Generation via DLS Inverse Kinematics

End-effector targets are defined in world coordinates and transformed into the base frame:

$$p_{ee}^B = R_z^T(\theta)(p_{ee}^W - p_b^W)$$

A damped least-squares IK solver iteratively computes a joint reference q_d:

$$\Delta q = J^T(JJ^T + \lambda^2 I)^{-1}(p_d - p(q))$$

The update Δq is bounded to ensure stable numerical behavior.

5.5.4.2. Joint-Space PID Control with Saturation and Anti-Windup

Given the joint reference q_d, each joint torque is computed using PID:

$$\tau_i = K_{p,i}e_i + K_{d,i}\dot{e}_i + K_{i,i}\int e_i dt, \quad e_i = wrap(q_{d,i} - q_i)$$

To handle actuator saturation, the torque is limited:

$$\tau_i \to sat(\tau_i, \tau_{max,i})$$

and anti-windup is implemented via conditional integration (integrator update is suppressed when the actuator is saturated and the integrator would increase saturation). This prevents large overshoot after saturation and improves stability during rapid phase transitions. [4]

Although several control strategies were investigated in this work, including LQR and adaptive fuzzy control, the PID controller was selected for the mobile base due to its simplicity, robustness, and ease of implementation in real-time navigation tasks. Unlike optimal or adaptive controllers, which require accurate system models and higher computational resources, PID control provides reliable performance for low-order mobile platform dynamics while remaining straightforward to tune and deploy.

Moreover, the motion of the base primarily involves velocity and orientation regulation rather than complex nonlinear joint coupling, making PID control sufficient to achieve stable trajectory tracking. Its compatibility with classical navigation methods such as Artificial Potential Fields also makes it well suited for obstacle avoidance and autonomous movement. Therefore, PID control represents a practical and efficient solution for base motion control within the overall mobile manipulation framework.

5.5.5. Perception and Mapping (LiDAR + Occupancy Grid)

A 2D LiDAR model generates range measurements over 360°, corrupted by Gaussian noise. The occupancy grid is updated using log-odds:

$$L_K = L_{K-1} + P(occ),\ l(z_k) = 1 - \frac{1}{1+\exp(L)}$$

This module provides an interpretable representation of the environment; in the current design it is primarily used for visualization and analysis. [2]

5.5.6. Results

5.5.6.1. Task Execution and Phase Timing

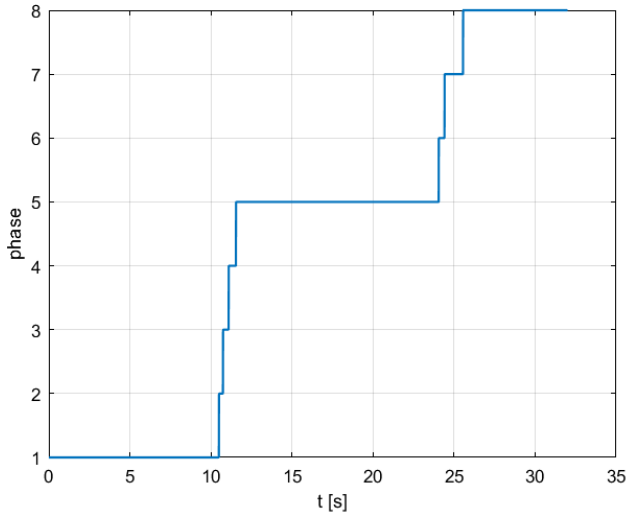


Figure 5—48: Finite-State Machine Phase Transitions During Task Execution

The finite-state machine transitions monotonically from navigation to manipulation phases, indicating reliable task sequencing. Extended dwell time during transport reflects obstacle-avoidance constraints and conservative safety behaviors near obstacles.

5.5.6.2. Base Navigation Performance

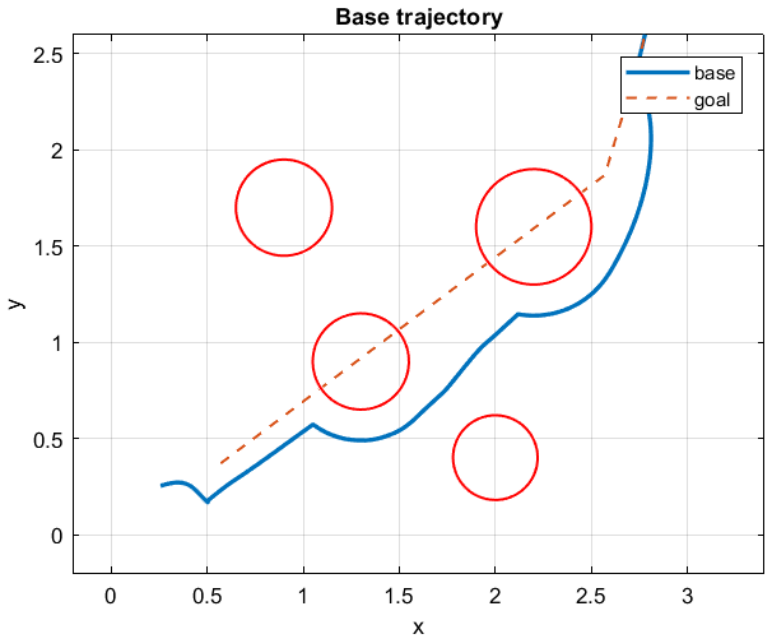


Figure 5—49: Base trajectory under enhanced APF avoidance

The executed base path deviates from the straight-line goal direction due to repulsive and tangential APF components. Inflated obstacle boundaries enforce a safety margin, producing a smooth collision-free trajectory.

5.5.6.3. Mapping / Perc eption Consistency

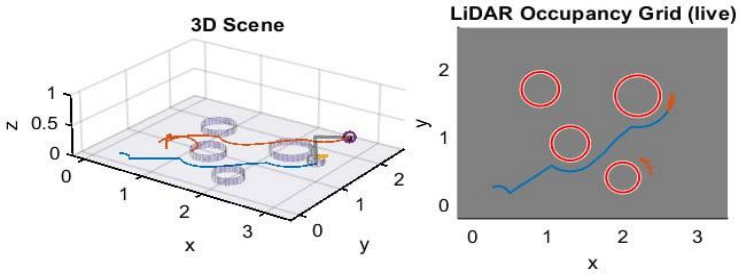


Figure 5—51: LiDAR Occupancy Grid with 3D Scene

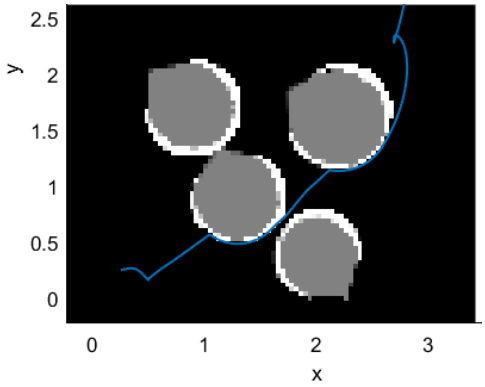


Figure 5—50: Final occupancy-grid reconstruction from LiDAR scans.

The log-odds occupancy update produces compact high-probability regions aligned with obstacle locations. The reconstructed map remains consistent despite measurement noise, and the base path is clearly observed.

5.5.6.4. End-Effector Tracking Accuracy

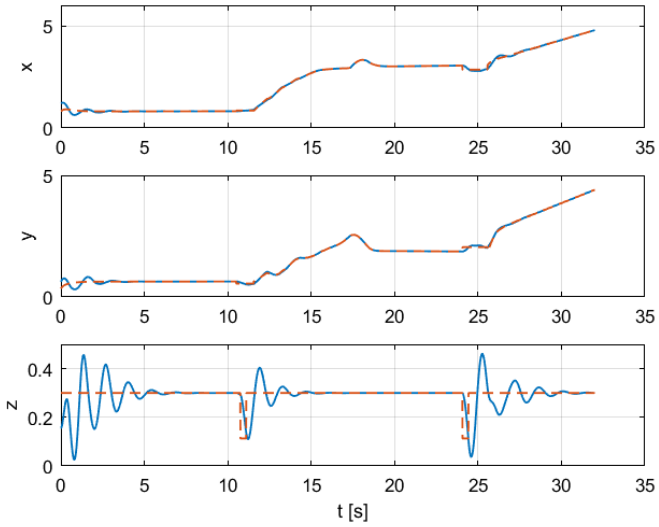


Figure 5—52: End-effector tracking in world coordinates.

The end-effector position closely follows the commanded reference in the horizontal axes throughout the task. Transient deviations in the vertical axis occur mainly during phase transitions (approach/descend/lift), consistent with abrupt reference changes and conservative safety constraints.

5.5.7. Discussion, Limitations and Future Work

The enhanced APF provides practical obstacle avoidance with acceptable smoothness. The state-machine structure improves reliability by separating navigation and manipulation. Joint PID with anti-windup is essential due to torque saturation during transitions. The mapping module enhances interpretability but is not yet integrated into planning.

The APF may still experience local minima, although the swirl term mitigates this risk. Manipulator dynamics are simplified with diagonal inertia and damping models. Future work could integrate global planners such as A*, D*, or RRT to replace purely reactive navigation.

6. Mechanical Design and Engineering Analysis

6.1. Design Overview

The robot is designed as an omnidirectional mobile platform integrated with a 3-DOF robotic arm for object transport. The design was developed using **SolidWorks 2016**, focusing on modularity, ease of assembly, and structural stability.

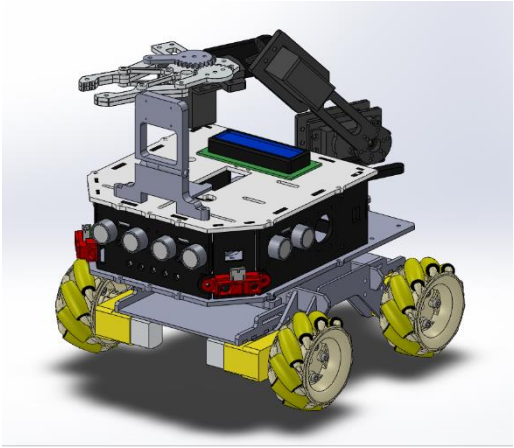


Figure 62—Isometric View of the Object Transport Robot Arm Assembly

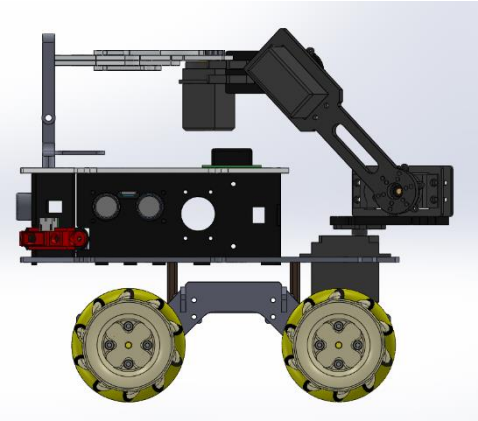


Figure 6—1: Side Elevation displaying the robotic arm reach and wheel clearance

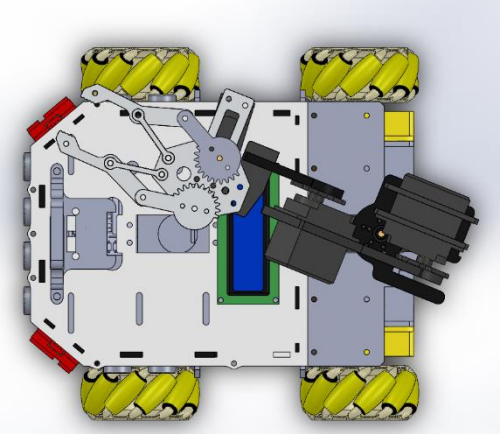


Figure 6—4: Top View showing the electronics layout and chassis symmetry

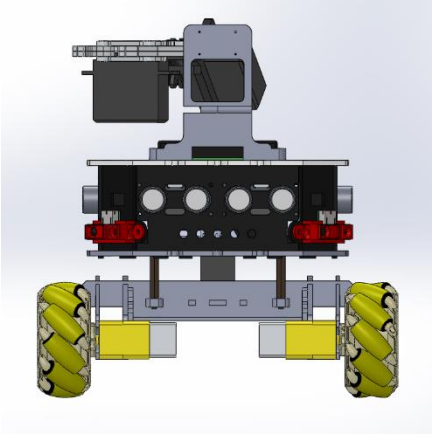


Figure 6—3: Front View showing the electronics layout and chassis symmetry

6.2. Bill of Materials (BOM)

The robot utilizes a hybrid manufacturing approach, combining industrial materials with rapid prototyping techniques:

- **3D Printed Parts:** A custom PLA (Polylactic Acid) profile was created with a density of 1240 kg/m^3 and an Elastic Modulus of 3.5 GP_a
- **Chassis:** High-strength Laser-cut Acrylic/Aluminum plates for the main structural frame.
- **Actuators:** High-torque DC Gear motors for the mobile base and precision Servo motors for the robotic arm joints.
- **End-Effector:** A specialized Gripper designed for secure object manipulation and transport.
- **Wheels:** 4x Mecanum wheels featuring rubberized rollers for omnidirectional mobility and traction.

6.3. Exploded Views and Assembly Strategy

To illustrate the internal architecture and the assembly sequence, the model was broken down into its primary sub-systems:

- **Drive System:** Showing the motor mounting and wheel hubs.
- **Main Chassis:** Demonstrating the electronics housing and sensor placements.
- **Robotic Arm:** Detailing the link-joint connections and end-effector.

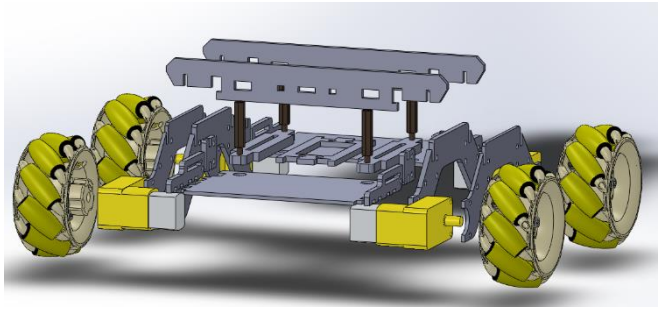


Figure 6—6: Exploded view of the Mobile Base and Drive System

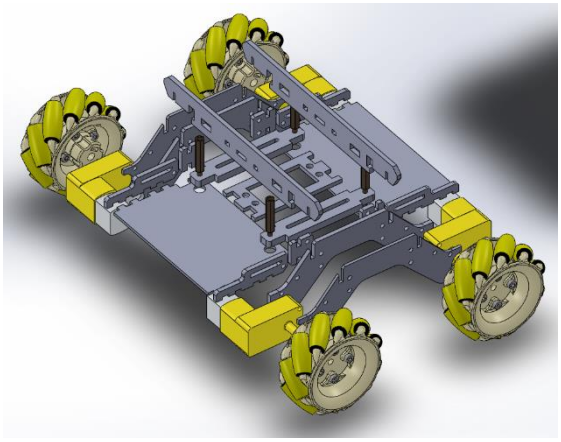


Figure 6—5: Exploded view of the Mobile Base and Drive System 2

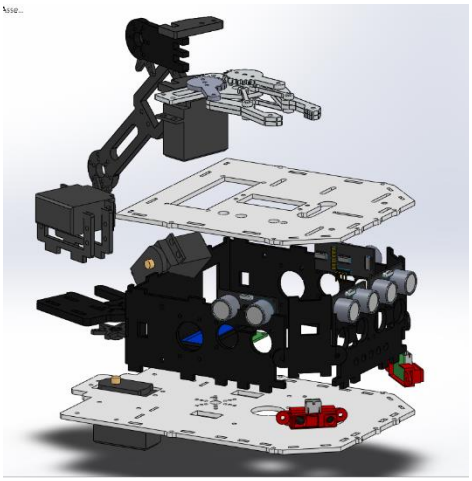


Figure 6—8: Exploded view of the Main Chassis and Electronic Integration

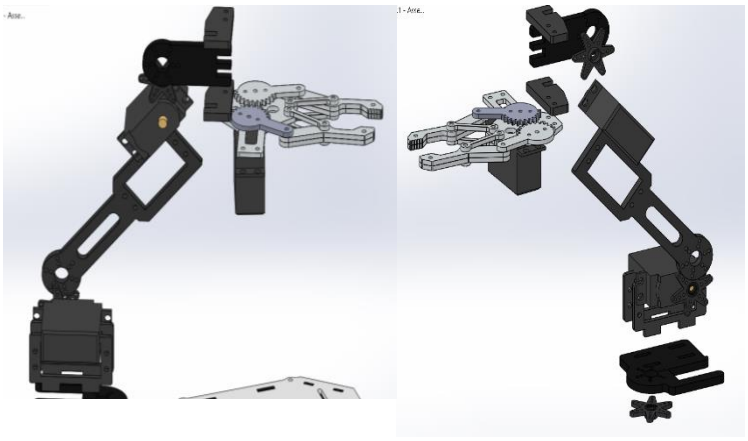


Figure 6—7: Exploded view of the 3-DOF Robotic Manipulator

6.4. Mass Properties and Weight Estimation

A comprehensive mass analysis was conducted using the SolidWorks Evaluate tools. Understanding the total weight is crucial for motor torque selection and battery power management.

- **Total Assembly Mass:** 1126.32 grams (~1.13 kg).
- **Volume:** 833,038.99 mm^3(~0.000833 m^3)
- **Surface Area:** 542,915.93 mm^2 (~0.542 m^2)

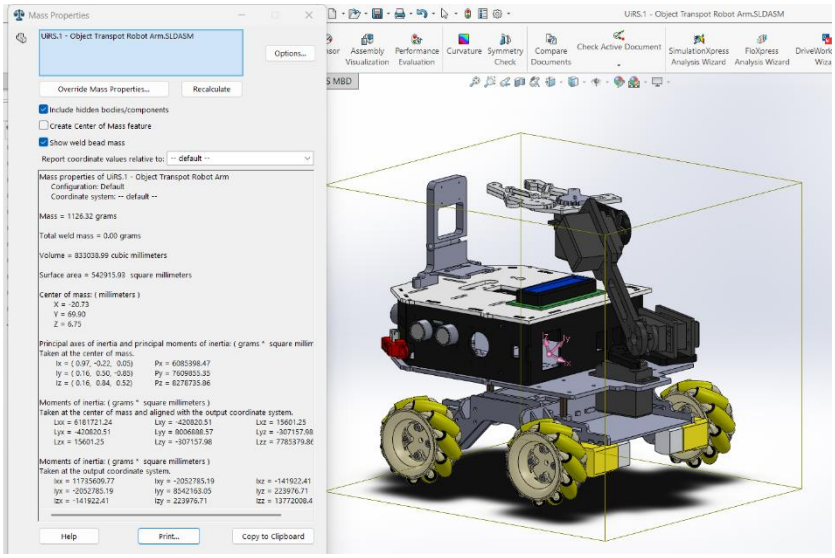


Figure 6—9 Mass Properties window

6.5. Stability and Center of Mass (CoM)

The stability of the omnidirectional platform was verified by locating the Center of Mass.

- **CoM Coordinates (mm):** X: -20.73, Y: 69.90, Z: 6.75.

- **Analysis:** The center of mass is situated centrally within the main chassis. This low and centered positioning ensures that the robot remains stable and does not tip over during the high-speed maneuvers of the Mecanum wheels or the articulation of the 3-DOF robotic arm.

6.6. Assembly Strategy and Exploded View

The robot was designed using a hierarchical assembly approach. To illustrate the internal structure and assembly sequence, an Exploded View was generated, separating the robot into two main sub-assemblies:

1. **The Mobile Base:** Consisting of the lower chassis, DC motors, and Mecanum wheel units.
2. **The Upper Chassis & Arm:** Containing the electronic housing, sensor mounts, and the robotic arm links.

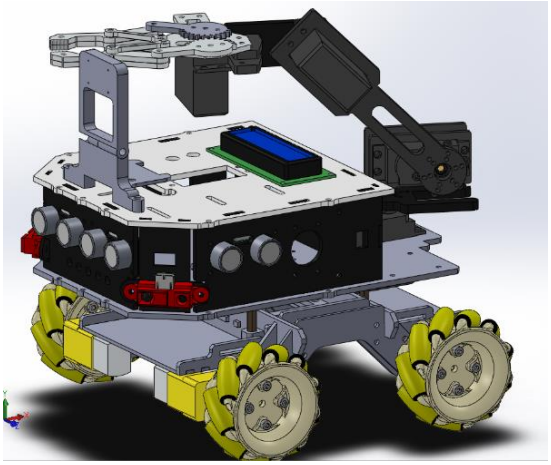


Figure 6—10: Full Robot Assembly Form

7. Conclusions, Results, and Future Work

7.1. Conclusion

This research work involved the modeling and control of an autonomous mobile robotic manipulator for pick-and-place tasks. A detailed kinematic and dynamic model was established, taking into account the dynamics of the actuators, the effect of payload, and nonlinear joint coupling.

Three control methods, namely PID, LQR, and adaptive fuzzy-PID control, were designed and compared. The results showed the pros and cons of each control method in terms of accuracy, control effort, and robustness, with the adaptive fuzzy-PID control method performing better than the others under actuator constraints and varying payload conditions.

The combination of obstacle avoidance, LiDAR mapping, and task-level planning allowed for fully autonomous operation in a structured environment. In conclusion, this research work emphasizes the need for accurate modeling and robust control techniques for reliable and accurate mobile robotic manipulation.

7.2. Results

1. The proposed mobile manipulator system has been tested using the simulation tool in order to check the accuracy, stability, and robustness of the system during the autonomous pick and place operation.
2. The accuracy of the mobile manipulator system has been verified with the experiments. From the experimental results, it has been verified that the mobile manipulator system has successfully completed the entire manipulation task, including navigation towards the object, grasping the object, transportation of the grasped object, and placement of the grasped object at the target location. It has also been verified that the mobile base has moved in smooth paths while avoiding the obstacles during navigation. This has verified the effectiveness of the artificial potential field navigation strategy.
3. The tracking error of the end-effector has been very low with steady-state error in the x, y, and z axes. It has also been verified that the PID controller has worked well in the nominal case. The LQR controller has also worked better with smooth paths and low control effort. The adaptive fuzzy PID controller has worked better compared to the other two controllers.
4. The LiDAR-based mapping module was able to successfully reconstruct the environment, and the occupancy grid map converged to a stable representation of the environment as the robot moved around. This is an indication that the perception and navigation modules are reliable.

Discussion of Results

The results obtained have verified that the integration of accurate dynamic modeling and robust control strategies can greatly enhance the performance of mobile manipulators. Although the PID control strategy is simple and provides acceptable tracking performance in nominal scenarios, optimal and adaptive control strategies are more robust in scenarios involving actuator constraints and disturbances.

The combination of navigation, mapping, and manipulation in a single framework has shown that autonomous mobile manipulation is possible using relatively simple sensing and control approaches. The work has also emphasized the need to take into account actuator dynamics and payload variations in the design of control systems for robots.

7.3. Future Work

1. **Real-World Implementation**

   The control framework should be implemented on a physical robot to test the effectiveness of the control approach in the presence of sensor noise, delays in the actuators, and uncertainties in the environment.

2. **Advanced Motion Planning**

   The control framework should be enhanced with advanced motion planning techniques like RRT*, A*, and MPC-based navigation to enhance the efficiency of obstacle avoidance maneuvers.

3. **Vision-Based Object Detection**

   The pre-programmed pick locations should be replaced with vision-based object detection techniques to enable the robot to operate in unstructured environments.

4. **Learning-Based Control**

   The control framework should be enhanced with reinforcement learning and adaptive neural networks to enable the automatic tuning of the control parameters.

5. **Energy-Optimal Control**

   The control design should be enhanced to minimize the energy consumption while maintaining the accuracy of the tracking performance.

6. **Human-Robot Collaboration**

   The control framework should be enhanced to enable the robot to operate safely in the presence of humans.

References

[1] X. Qiu, "Forward Kinematic and Inverse Kinematic Analysis of a 3-DOF RRR Manipulator," in *Proceedings of the 2024 International Conference on Mechanics, Electronics Engineering and Automation (ICMEEA 2024)*, N/A, 2024.

[2] M. Spong, S. Hutchinson and M. Vidyasagar, Robot Modeling and Control, 2nd edn ed., John Wiley & Sons, Ltd., 2020.

[3] B. Siciliano, L. Sciavicco, L. Villani and G. Oriolo, Robotics: Modelling, Planning and Control, London: Springer-Verlag London, 2010.

[4] K. M. L. a. F. C. Park, Modern Robotics: Mechanics, Planning, and Control, Cambridge, U.K.: Cambridge University Press, 2017.

[5] Q. Huang, G. He, G. Feng and B. Ding, "Novel Cooperative Control Approach for 3-DoF Parallel Mechanisms," *MDPI Journal,* 2024.

[6] Z. Liu and L. Li, "Kinematic Calibration Using Optimization Algorithms for Heavy-Load Manipulators," *Cambridge University Press & Assessment Journal,* 2022.

[7] S. Liu, J. Song, L. Zhang and Y. Tan, "Adaptive Control System for Ship Propulsion-Assisted Sails," *MDPI Journal,* 2024.

[8] J. Latombe, Robot Motion Planning, Kluwer Academic Publishers, 1991.

[9] M. Johnson and T. Murphey, Machine Learning in Robotic Control, Annual Review of Control, Robotics, and Autonomous Systems, 2019.

[10] G. Franklin, "Adaptive Feedback Control in Robotics," *IEEE Transactions on Robotics,* vol. vol. 34, p. pp. 1425–1440, 2018.

[11] W. S. Mark, H. Seth and V. M., Robot Modeling and Control, Wiley: JOHN WILEY & SONS, INC., 2006.

[12] S. Pezeshki, S. Badalkhani and A. Javadi, "Performance analysis of a neuro-PID controller applied to a robot manipulator," *International Journal of Advanced Robotic Systems,* vol. 9, no. 5, p. 163, 2012.

[13] A. C. Bittencourt, "Friction change detection in industrial robot arms," KTH Royal Institute of Technology, Stockholm, 2007.

[14] A. De Luca and B. Siciliano, "Closed-Form Dynamic Model of Planar Multi-Link Lightweight Robots," *IEEE Transactions on Systems, Man, and Cybernetics,* vol. 21, no. 4, p. 826–839, 1991.

[15] R. Krishnan, Electric Motor Drives: Modeling, Analysis, and Control, Upper Saddle River, NJ, USA: Prentice Hall, 2001.

[16] B. Siciliano, Springer Handbook of Robotics, O. Khatib, Ed., Cham, Switzerland: Springer, 2016 (2nd Edition).

[17] H. Yin, Y. Li and J. Li, "Decomposed dynamic control for a flexible robotic arm in consideration of nonlinearity and the effect of gravity," *Advances in Mechanical Engineering,* vol. 9, no. 2, 2017.

[18] T. H. M. B. J. E. K. S. E. Sundström, "Reference Implementation of the PID Controller," *IFAC-PapersOnLine,* vol. 58, no. 7, pp. 370-375, 2024.

[19] S. C. B. S. F. Caccavale, "Second-order kinematic control of robot manipulators with Jacobian damped least-squares inverse: theory and experiments," *IEEE/ASME Transactions on Mechatronics,* vol. 2, no. 3, pp. 188 - 194, 1997.

[20] M. H. Kazemi and A. Fazli, "PID-based robust pole assignment tracking control of robot manipulator using linear parameter varying modeling," *Advanced Robotics,* vol. 39, no. 11, pp. 678-688, 2025.

[21] S. Chiaverini, "Singularity-robust task-priority redundancy resolution for real-time kinematic control of robot manipulators," *IEEE Transactions on Robotics and Automation,* vol. 13, no. 3, pp. 398-410, 1997.

[22] S. Ali, Modern control engineering, N/A: University of Baghdad, N/A.

[23] B. D. O. A. a. J. B. Moore, Optimal Control: Linear Quadratic Methods, New York, NY, USA: Dover Publications, 2014.

[24] H. Y. a. W. S. a. J. J. Buckley, "Fuzzy control theory: A nonlinear case," *Autom,* vol. 26, pp. 513-520, 1990.

[25] T. H. Karl J. htrom, PID Controllers, 2nd Edition, Research Triangle Park, NC, USA: Instrument Society of America, 1995.

[26] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *IEEE International Conference on Robotics and Automation,* pp. 500-505, 1985.

[27] M. Spong, S. Hutchinson and M. Vidyasagar, Robot Modeling and Control, John Wiley & Sons, Ltd., 2020.